

PythonFP

Functional Programming in Python, the Prelude Way

A course in one file and seventy-four problems

J. L. Clements III

September 16, 2026

Preface

This book teaches functional programming in Python through a single artifact: `prelude.py`, a Haskell-style prelude rebuilt in pure Python with no imports, read top to bottom in strict define-before-use order. Around it sit seventy-four problems, each solved by *composing* prelude tools rather than narrating loops.

The course has three movements. Part I teaches how to *think* in this style: what functional Python buys you, and the recognition drill that maps the words of a problem statement to the right tool. Part II walks the prelude section by section – every definition, why it exists, and what it displaces. Part III applies the toolkit to nine classes of problems; every problem appears with its statement, contract, hint, doctests and a worked solution.

The book is generated from the course files themselves, so every listing is the real code. The intended use is active: keep `prelude.py` open beside the book, attempt each problem in its practice file before reading the solution, and let the doctests judge you. Appendix D describes that workflow.

Contents

Preface	iii
I Thinking in PythonFP	1
1 Why Functional Python	3
1.1 One file, no imports	3
1.2 What “functional” buys you	3
1.3 Conventions you must internalise	3
2 The Recognition Drill	5
2.1 Step 0: name three things	5
2.2 Step 1: walk the ladder	5
2.3 Step 2: say it out loud	6
2.4 Step 3: compose, don’t invent	6
II The Prelude, Section by Section	7
3 Functions About Functions	9
4 Arithmetic and Logic	11
5 Lists I: The Basics	13
6 Lists II: Higher-Order	15
7 Folds	17
8 Scans, Streams, and Finite Views	19
9 Order and Text	23
10 Containers: Dicts, Bags, Trees, Heaps	25
11 Nodes, Grids, Windows, Control	29

12 Deep Dives	33
12.1 flip	33
12.2 curry	34
12.3 on	35
12.4 NOTHING	35
12.5 fromMaybe	36
12.6 isNothing	37
12.7 isqrt	37
12.8 transpose	38
12.9 find	38
12.10 partition	39
12.11 mapMaybe	39
12.12 foldl	40
12.13 foldr	41
12.14 compose	42
12.15 subsequences	43
12.16 accumulate	43
12.17 scanl	44
12.18 unfoldr	45
12.19 span	46
12.20 cmp_to_key	46
12.21 merge	47
12.22 bisect_left	47
12.23 defaultdict	48
12.24 Counter	49
12.25 insertWith	49
12.26 fromListWith	50
12.27 unionWith	51
12.28 groupBy	51
12.29 ITree	52
12.30 paths	52
12.31 setpath	53
12.32 deque	54
12.33 dsu	55
12.34 heappush	56
12.35 heappop	56
12.36 tfold	57
12.37 levelorder	58
12.38 reverse_list	58
12.39 middle	59
12.40 has_cycle	59

12.41	merge_lists	59
12.42	neighbors4	60
12.43	longest_window	60
12.44	memo	61
12.45	until	62
 III The Nine Problem Classes		65
 13 Class I — Arithmetic and Unfolds		67
13.1	sign: Sign of a number	67
13.2	rect_area: Rectangle area	68
13.3	reverse_digits: Reverse an integer's digits	68
13.4	to_base: Render n in base b	69
13.5	to_roman: Integer to Roman numeral, $1 \leq n \leq 3999$	69
13.6	from_roman: Parse a valid Roman numeral	70
 14 Class II — Lists and Strings		71
14.1	greet: Greeting string	71
14.2	count_vowels: Count vowels, case-insensitive	71
14.3	fizzbuzz: FizzBuzz as data	72
14.4	word_lengths: Map each word to its length	73
14.5	flatten: Flatten one level of nesting	73
14.6	caesar: Caesar cipher, letters only, case preserved	74
14.7	contains_duplicate: NeetCode: Contains Duplicate	74
14.8	chunks: Split a list into pieces of length k	75
14.9	permutations: All distinct orderings, sorted	75
14.10	reverse_words: Reverse the word order	76
14.11	transpose_matrix: Flip rows and columns	76
14.12	rotate_right: Rotate a list right by k	77
 15 Class III — Folds, Scans and Streams		79
15.1	two_sum: NeetCode: Two Sum	79
15.2	best_time_stock: NeetCode: Best Time to Buy and Sell Stock	80
15.3	maximum_subarray: NeetCode: Maximum Subarray (Kadane)	80
15.4	product_except_self: NeetCode: Product of Array Except Self	81
15.5	moving_average: Average of every k-window	81
15.6	pascal: First n rows of Pascal's triangle	82
15.7	climbing_stairs: NeetCode: Climbing Stairs	82
15.8	primes_up_to: All primes $\leq n$, ascending	83
15.9	totals_and_deltas: A scan and its inverse	83
15.10	paren_depth: Deepest paren nesting	84

16 Class IV — Sorting and Searching	85
16.1 is_anagram_sorted: Anagrams via canonical form	85
16.2 second_largest: Second largest distinct value	86
16.3 binary_search: Index of target in a sorted list, else -1	86
16.4 merge_intervals: Merge overlapping (and touching) intervals	87
16.5 photo_lineup: The reported Anduril prompt, prelude-sized	87
16.6 merge_sort: A real $O(n \log n)$ sort, by hand	88
16.7 lis: Longest strictly increasing subsequence (length)	88
16.8 russian_doll: LC 354: Russian Doll Envelopes	89
16.9 pairs_within_budget: Count pairs whose sum fits a budget	90
16.10 sensor_pairing: The screen's domain problem, function form	90
17 Class V — Bags, Dicts and Grouping	93
17.1 valid_anagram: NeetCode: Valid Anagram	93
17.2 ransom_note: HackerRank: Ransom Note	93
17.3 most_frequent: Most frequent value; ties -> smallest	94
17.4 top_k_words: The k most frequent words	94
17.5 group_words: Group words by first letter	95
17.6 group_anagrams: NeetCode: Group Anagrams	95
17.7 merge_sum: Merge two dicts of numbers, summing shared keys	96
17.8 rle_encode: Run-length encode	97
17.9 lru_class: An LRU cache class, dict-only	97
17.10 first_unique: First non-repeating character	98
17.11 log_report: Error report from messy logs	99
17.12 invert_dict: Swap a dict's keys and values	100
18 Class VI — Tries and Paths	101
18.1 deepest_directory: Challenge: deepest path in a directory forest	101
18.2 autocomplete: Challenge: prefix trie with completion	102
18.3 subset_sum_paths: Challenge: all paths in the decision tree summing to a total .	103
18.4 subset_sum_trie: Challenge: subset-sum, winners recorded in a solution trie . . .	104
19 Class VII — Graphs, Grids, Windows and Heaps	107
19.1 longest_unique: Longest substring without repeats	107
19.2 grid_path: Shortest path through a grid	108
19.3 rate_limiter: Sliding-window rate limiter	108
19.4 running_median: Median after each arrival	109
19.5 dijkstra: Challenge: shortest paths from weighted (from, to, dist) edges	110
19.6 dag_longest_path: Challenge: longest path in a weighted DAG	111
19.7 bellman_ford: Challenge: shortest paths as a fixed point (negative weights ok) . .	112
19.8 course_schedule: NeetCode: Course Schedule (cycle detection via Kahn)	113
20 Class VIII — Dynamic Programming and Control	115

20.1	coin_change: NeetCode: Coin Change	115
20.2	lcs: NeetCode: Longest Common Subsequence	116
20.3	edit_distance: Levenshtein distance	116
20.4	knapsack: 0/1 knapsack	117
20.5	calculator: Integer expression evaluator (capstone)	118
20.6	max_height_cuboids: LC 1691: Maximum Height by Stacking Cuboids	119
20.7	guillotine_cut: Max value cut from a sheet, guillotine cuts only	120
20.8	partition_k_subsets: LC 698: Partition to K Equal Sum Subsets	121
20.9	min_path_sum: Cheapest right/down path through a grid	122
21	Class IX — Stack Folds	125
21.1	balanced_brackets: Matching pairs: valid bracket nesting	125
21.2	eval_rpn: Evaluate Reverse Polish notation	126
21.3	next_greater: First later element that's larger	126
IV	Appendices	129
A	prelude.py, Complete	131
B	Haskell Provenance	143
C	How to Practice	145

Part I

Thinking in PythonFP

Chapter 1

Why Functional Python

1.1 One file, no imports

Everything in this course runs on bare `python3`. That constraint is not asceticism; it is the point. When `collections.Counter` is an import, it is a black box; when it is six lines you have retyped from memory, it is knowledge. The prelude rebuilds the working parts of Haskell’s standard vocabulary – and the useful corners of `Data.List`, `Data.Map`, `Data.Maybe` and friends – as Python one-liners and small definitions you can carry into any interview editor, any locked-down environment, any whiteboard.

1.2 What “functional” buys you

Three habits, none of which require giving up Python:

- **Functions are values.** Sort keys, predicates, combining functions, closures over window state: passing behaviour as data is what lets one skeleton (`longest_window`, `foldl`, `fromListWith`) serve a hundred problems.
- **Data in, data out.** Prefer expressions that build new values to statements that mutate old ones. Not because mutation is sinful – the prelude mutates inside its machines – but because data-in/data-out functions compose, test in one line, and survive a mutating interview prompt.
- **Name the transformation.** A loop narrates *how*; a composition of named tools states *what*. `unwords(reversed(words(s)))` is its own documentation. Vocabulary is leverage: every name you own is a loop you never write again.

1.3 Conventions you must internalise

None is Nothing. Every partial prelude function signals absence with `None`; `fromMaybe` lands defaults, `mapMaybe` and `catMaybes` sweep Nothings out of lists, `isJust` and `isNothing` test a single value. Never overload `0` or `""` to mean “missing”.

Haskell names shadow builtins by design. `id`, `map_`, `filter_`, `sort`: inside prelude code the Haskell vocabulary wins. The trailing underscore marks the few collisions with keywords (`not_` being the survivor; `all/any` serve bare).

Define before use. `prelude.py` reads top to bottom; every name is already defined when you meet it. This is also how you should *study* it: read a section, close the file, retype its one-liners from memory, and check yourself. The definitions are short on purpose.

Chapter 2

The Recognition Drill

Most timed failures are not coding failures; they are *recognition* failures – ten minutes spent hand-rolling a loop that was a `scanl1`. The drill below runs before any typing.

2.1 Step 0: name three things

From the statement, extract: the **input shape** (a list? two lists? pairs? a grid? edges?); the **output shape** (one value? a list? a dict? yes/no? *all* solutions?); and the **quantity word** (the “most”, “fewest”, “longest”, “every” that names what is asked). The output shape is the strongest clue: one value smells like a fold, a dict smells like `fromListWith`, “all of them” smells like enumeration or a tree.

2.2 Step 1: walk the ladder

Match the statement’s words against the trigger column; the first row that fits names your tools.

Shape	Trigger words	Reach for
SORT FIRST	“sorted”, “k-th”, “closest pair”, “can they be arranged”, “intervals”	sort, sortOn, bisect_left, merge
FOLD	“total”, “count”, “largest”, “best single ...”	foldl, minOn / maxOn
SCAN	“running”, “so far”, “at each step”, “prefix sums”	scanl, scanl1
DICTIONARY FOLD	“group by”, “frequency”, “most common”, “anagrams together”	fromListWith, Counter, unionWith
WINDOW	“substring”, “contiguous”, “longest run”, “within the last k”	longest_window, pairwise
STACK	“nested”, “matching pairs”, “most recent open”, “next greater”	list as stack; foldl with stack acc
UNFOLD	“digits of”, “split into pieces”, “repeat until”, “simulate”	unfoldr, iterate + take
ENUMERATE	“all subsets”, “any combination”, “every way to pick”	subsequences, replicateM, cross
TREE / PATHS	“nested”, “prefix” (trie), “directories”, “solutions as paths”	ITree, paths, setpath, tfold
GRAPH	“prerequisites”, “network”, “shortest route”, “reachable”	Tree(1,list) adjacency, deque BFS, heap, until
MEMOIZE	“fewest”, “max value”, “count the ways”, “can it be done”	memo over indices

Cross-cutting, any row: messy input $\rightarrow$ `mapMaybe`; absence $\rightarrow$ `None` + `fromMaybe`; first hit $\rightarrow$ `find`; same-up-to-order $\rightarrow$ a canonical form (`sort` or `Counter`); connectivity $\rightarrow$ `dsu`; repeated best-next $\rightarrow$ the heap.

2.3 Step 2: say it out loud

State the shape, the tools, and the complexity before typing: “group-by-key with `fromListWith`, then best-by-key with `minOn`, $O(n)$.” If two rows both apply, **the row that names the output wins**: “most common word” *mentions* frequency (dict fold) but *asks for* one value (fold) – so `Counter` feeds `minOn`.

Complexity vocabulary to keep loaded: $\sim 10^7$ simple Python operations per second is a safe planning number; $n \leq 20$ licenses 2^n enumeration; $n \leq 10^4$ licenses $O(n^2)$; $n \leq 10^6$ demands $O(n \log n)$ or better.

2.4 Step 3: compose, don’t invent

Solutions in this course are pipelines of two or three prelude names plus one lambda of your own. If you find yourself writing a raw loop, first ask which combinator you are rebuilding. Sometimes the answer is “none – this loop is the algorithm” (BFS, a parser, a heap sift); then write the loop proudly.

Part II

The Prelude, Section by Section

Chapter 3

Functions About Functions

Functional programming begins with one liberty: a function is a value. It can be passed, returned, stored in a dict, and – most usefully – *shaped* by other functions. Section 1 is a kit of shapers. `flip` swaps an argument order so a function fits a slot it was not written for; `curry` and `uncurry` convert between two-argument functions and functions of pairs; `partial` freezes the first arguments; `on` routes both arguments of a comparison through a key first.

Two definitions here are load-bearing conventions rather than tools. `NOTHING` is a private sentinel meaning “no argument was supplied” – it lets `foldl` and `accumulate` distinguish an omitted initial value from a legitimate `None`. And the file-wide contract *None is Nothing*: every partial function in the prelude signals absence with `None`, `fromMaybe` is the standard way to land a default, and `isJust/isNothing` name the `None` test itself – a predicate you can hand to `find`, `filter_` or `all`, and one that asks identity against `None` rather than truthiness, so 0 and "" count as present. Section 2 adds the pair accessors – `fst`, `snd`, `swap` – which read better than `[0]` and `[1]` in code built from tuples.

```
# ===== 1. combinators ===== (functions about functions)

const      = lambda x: lambda _: x                # always returns x, ignoring the
    ↪ second argument
# curry: converts a two-argument function into a chain of two one-argument functions,
    ↪ so instead
# of calling f(x, y), you call curry(f)(x)(y).
curry      = lambda f: lambda x: lambda y: f(x, y)
flip       = lambda f: (lambda x, y: f(y, x))      # flip f x y = f y x
NOTHING    = object()                             # Maybe's Nothing: "no arg
    ↪ given"; test with `is`                               # hoisted ahead of foldl, which
    ↪ needs it as a sentinel

def foldl(f, xs, base=NOTHING): # THE left fold; Python buried its own in functools
    ↪ as reduce
    it = iter(xs)
    if base is NOTHING:
        try:
            acc = next(it)
        except StopIteration:
            raise TypeError("fold of empty sequence with no initial value")
    else:
```

```

    acc = base
    for x in it:
        acc = f(acc, x)
    return acc

foldr  = lambda f, xs, base: foldl(flip(f), reversed(list(xs)), base)  # THE right
    ↪ fold, via foldl
compose = lambda *fns: lambda x: foldr(lambda f, acc: f(acc), fns, x)  # foldr (.)
    ↪ id: RIGHTMOST first
fromMaybe = lambda d, x: d if x is None else x                        # Data.Maybe: default for every
    ↪ None-returning tool
id       = lambda x: x                                                # shadows builtin id() by design
isJust   = lambda x: x is not None                                    # Data.Maybe: None test as a
    ↪ handable predicate
isNothing = lambda x: x is None                                       # Data.Maybe: true when x is
    ↪ Nothing (None)
on       = lambda f, g: lambda x, y: f(g(x), g(y))                  # Data.Function: combine via a
    ↪ key -- cmp `on` fst
partial  = lambda f, *bound: lambda *args: f(*bound, *args)         # fix leading args of
    ↪ f, wait for the rest
uncurry  = lambda f: lambda p: f(*p)                                # inverse of curry: call f with
    ↪ a pair, not two args

# ===== 2. pairs =====

fst = lambda p: p[0]                                                # first element of a pair
snd = lambda p: p[1]                                                # second element of a pair
swap = compose(tuple, reversed)  # Data.Tuple

```

At the REPL:

```

>>> add = lambda a, b: a + b
>>> flip(add)("world", "hello ")
'hello world'
>>> on(add, len)("ab", "c")                                         # combine THROUGH a key
3
>>> fromMaybe(0, None), fromMaybe(0, 5)
(0, 5)
>>> isJust(0), isNothing(None)                                     # identity, never truthiness
(True, True)
>>> swap((1, 2))
(2, 1)

```

Chapter 4

Arithmetic and Logic

Small tools, chosen for their sharp edges. `signum` compresses a three-way comparison into arithmetic. `div/mod` floor like Haskell's `div`; `quot/rem` truncate toward zero – knowing which one your language does to negative numbers has decided real interviews. `gcd` recurses safely because Euclid's depth is tiny even for 64-bit inputs (the worst case is consecutive Fibonacci), and `isqrt` is Newton's method on integers: floor square roots with no floating point in sight.

```
# ===== 3. arithmetic & logic =====

add      = lambda a, b: a + b                # (+) as a
    ↪ value: scanl1(add, xs)
div      = lambda a, b: a // b                # floors,
    ↪ like Haskell div
even     = lambda n: n % 2 == 0               # true for
    ↪ even numbers
gcd      = lambda a, b: abs(a) if b == 0 else gcd(b, a % b) # >= 0 like
    ↪ Haskell; depth <= 90
halve    = lambda n: n // 2                  # e.g. iterate(halve, 1024) -> 1024, 512, 256,
    ↪ ...
hypot    = lambda x, y: (x*x + y*y) ** 0.5    #
    ↪ straight-line distance

def isqrt(n): # floor sqrt without floats (Newton)
    if n < 0:
        raise ValueError("isqrt of negative")
    x = n
    y = halve(x + 1)
    while y < x:
        x, y = y, halve(y + n // y)
    return x if n else 0

lcm      = lambda a, b: abs(a // gcd(a, b) * b) if a and b else 0 # >= 0 like
    ↪ Haskell
mod      = lambda a, b: a % b                 # remainder,
    ↪ sign follows b
not_     = lambda b: not b                    # `not` is a
    ↪ Python keyword
odd      = lambda n: n % 2 == 1               # true for
    ↪ odd numbers
```

```

otherwise = True                                # guard
    ↪ fallback: always true
pred      = lambda x: chr(ord(x) - 1) if isinstance(x, str) else x - 1  # Enum:
    ↪ numbers and Chars
quot      = lambda a, b: -(-a // b) if (a < 0) != (b < 0) else a // b    # truncates,
    ↪ like Haskell quot
rem       = lambda a, b: a - b * quot(a, b)                               # remainder,
    ↪ sign follows a
quotRem   = lambda a, b: (quot(a, b), rem(a, b))                         # (quot, rem)
    ↪ in one call
signum    = lambda x: (x > 0) - (x < 0)                                  # -1, 0, or 1
    ↪ by the sign of x
sub       = lambda a, b: a - b                                           # (-) as a
    ↪ value: zipWith(sub, a, b)
succ      = lambda x: chr(ord(x) + 1) if isinstance(x, str) else x + 1  # next:
    ↪ increments, advances chars

```

At the REPL:

```

>>> signum(-7), signum(0), signum(3)
(-1, 0, 1)
>>> div(-7, 2), quot(-7, 2)          # floor vs truncate
(-4, -3)
>>> isqrt(10**18)
1000000000

```

Chapter 5

Lists I: The Basics

The list vocabulary. `head/tail/init/last` name the four ways to take a list apart at its ends; `null` asks if there is anything left – together they are the grammar of structural recursion. Three definitions deserve special study. `nub` deduplicates *keeping first occurrences*, exploiting the guarantee that dicts remember insertion order. `transpose` flips rows and columns, ragged-safe like Haskell’s – short rows simply drop out of later columns. `pairwise` zips a list against its own tail, giving every element its right neighbour – the tool for any rule about adjacency. The prefix family (`isPrefixOf`, `isSuffixOf`, `stripPrefix`) works on strings and lists alike, and `isSuffixOf`’s comment preserves a scar: `xs[-0:]` is the *whole* list, so never slice a suffix by `-len(p)`.

```
# ===== 4. list basics =====

drop      = lambda n, xs: list(xs)[max(n, 0):]          # finite
    ↪ lists; n<0 drops none
elem      = lambda x, xs: x in xs                      # true if x
    ↪ is in xs
zip_      = lambda a, b: list(zip(a, b))               # shortest
    ↪ wins, any iterable
enum      = lambda xs, start=0: zip_(list(range(start, start + len(xs))), xs) #
    ↪ pairs each x with its index
head      = lambda xs: xs[0]                          # first
    ↪ element
init      = lambda xs: xs[:-1]                        # all but
    ↪ the last element
isPrefixOf = lambda p, xs: list(xs[:len(p)]) == list(p) #
    ↪ Data.List; strings or lists
isSuffixOf = lambda p, xs: list(xs[len(xs) - len(p):]) == list(p) # not
    ↪ [-len(p):]: [-0:] is ALL
last      = lambda xs: xs[-1]                        # last
    ↪ element
lookup     = lambda k, pairs: next((v for kk, v in pairs if kk == k), None) #
    ↪ Nothing -> None
notElem    = lambda x, xs: x not in xs                # true if x
    ↪ is not in xs
nub        = compose(list, dict.fromkeys)              # unique;
    ↪ first seen wins
null       = lambda xs: len(xs) == 0                  # true for
    ↪ an empty list
```

```

pairwise    = lambda xs: zip_(xs, xs[1:])           # zip xs
               ↪ (tail xs)
replicate   = lambda n, x: [x] * n                 # x
               ↪ repeated n times
splitAt     = lambda n, xs: (xs[:n], xs[n:]) if n >= 0 else (xs[:0], xs) # n<0
               ↪ splits at the front
stripPrefix = lambda p, xs: xs[len(p):] if isPrefixOf(p, xs) else None   # Just the
               ↪ rest, or Nothing
tail        = lambda xs: xs[1:]                    # all but
               ↪ the first element
def transpose(xss):                                # Data.List: rows <-> cols, RAGGED-SAFE like
               ↪ Haskell
    xss = [list(xs) for xs in xss]                 # any iterables in (rows may be one-shot);
               ↪ materialise once
    n = max(map(len, xss), default=0)               # short rows just drop out of later columns (no
               ↪ truncation)
    return [[xs[i] for xs in xss if i < len(xs)] for i in range(n)]
unzip       = lambda ps: tuple(map(list, zip(*ps))) if ps else ([], [])  # inverse
               ↪ of zip: pairs -> two lists
zip3        = lambda a, b, c: list(zip(a, b, c))    # zip three
               ↪ lists together

```

At the REPL:

```

>>> nub([3, 1, 3, 2, 1])
[3, 1, 2]
>>> transpose([[1, 2, 3], [4, 5, 6]])
[[1, 4], [2, 5], [3, 6]]
>>> pairwise([1, 4, 9])
[(1, 4), (4, 9)]
>>> stripPrefix("re", "rebuild")
'build'

```

Chapter 6

Lists II: Higher-Order

Where loops go to be named. `map_` transforms, `filter_` selects, `concat` flattens one level, and `concatMap` does both at once – it is the backbone of enumeration (“for every choice, produce all results, pooled”). `find` is the early exit folds cannot perform: it is lazy, works on infinite streams, and returns `None` on failure, feeding `fromMaybe`. `mapMaybe` is the messy-input workhorse – map a parser that returns value-or-`None`, drop the `Nothings`, one pass. When input is dirty, reach for it before you reach for a loop with an `if` and a flag.

```
# ===== 5. higher-order lists =====

filter_ = lambda cond, xs: [x for x in xs if cond(x)]           # keep elements
    ↪ where cond holds
catMaybes = partial(filter_, isJust)                            # mapMaybe id
concat   = lambda xss: [x for xs in xss for x in xs]           # flatten one
    ↪ level of nesting
concatMap = lambda f, xs: [y for x in xs for y in f(x)]         # map f over xs,
    ↪ then flatten
cross     = lambda a, b: [(x, y) for x in a for y in b]         # cartesian;
    ↪ `product` is the fold
# Data.List find: lazy first-match -- folds can't stop early, find can
find      = lambda cond, xs: next((x for x in xs if cond(x)), None) # first match,
    ↪ else None
map_      = lambda f, xs: [f(x) for x in xs]                   # apply f to
    ↪ every element
# Data.Maybe mapMaybe: map and drop the Nothings in ONE pass -- the parse-and-filter
    ↪ shape
mapMaybe = lambda f, xs: [y for y in (f(x) for x in xs) if y is not None]
def partition(cond, xs):                                       # (keepers, rest) in ONE pass; cond called
    ↪ once per element
    yes, no = [], []
    for x in xs:
        (yes if cond(x) else no).append(x)
    return (yes, no)
starmap   = lambda f, pairs: [f(*p) for p in pairs]           # map . uncurry
zipWith   = lambda f, a, b: [f(x, y) for x, y in zip(a, b)]    # combine two
    ↪ lists elementwise
zipWith3  = lambda f, a, b, c: [f(x, y, z) for x, y, z in zip(a, b, c)] # combine
    ↪ three lists elementwise
```

At the REPL:

```
>>> concatMap(lambda w: [w, w.upper()], ["a", "b"])
['a', 'A', 'b', 'B']
>>> find(lambda x: x % 7 == 0, count(1))      # lazy: an infinite stream
7
>>> mapMaybe(lambda s: int(s) if s.isdigit() else None, ["3", "x", "7"])
[3, 7]
```

Chapter 7

Folds

The fold is the master pattern: an accumulator, a combining function, one pass. `foldl` is the definition – modern Python has no builtin fold (its `reduce` was banished to `functools`), so the prelude keeps only the Haskell name – `functools` can wait for days when imports return. Study `foldl`'s handling of the missing initial value via `NOTHING`. `foldr` is derived from `foldl` by flipping and reversing – an identity that is only valid because Python is strict, and stated in the file header so you never forget the caveat. `all/any` are the Boolean folds, used bare as builtins over Booleans; `compose` folds a tuple of functions into one.

The section ends with the *enumeration folds* – brute-force licenses. `subsequences` builds the powerset by doubling the accumulator per element (2^n : fine to $n \approx 20$, and you should say that bound out loud); `replicateM` builds every length- n word over an alphabet ($|xs|^n$). When n is small, honest enumeration beats clever and fragile.

```
# ===== 6. folds ===== (collapse a list to one value)
# foldl, foldr and compose are hoisted up to section 1 (combinators): compose needs
#   ↪ foldr, foldr needs
# foldl, and swap (section 2) needs compose -- so all three must exist before section
#   ↪ 2 even begins.

foldl1 = foldl                                     # foldl seeded
#   ↪ by the first element
foldr1 = lambda f, xs: foldl(flip(f), reversed(list(xs))) # foldr seeded
#   ↪ by the last element
product = lambda xs: foldl(lambda a, x: a * x, xs, 1)      # the numeric
#   ↪ fold

# enumeration folds -- brute-force licenses for small n (say the bound out loud):
# Control.Monad replicateM: every length-n word over xs -- cross, n times; |xs|^n
replicateM = lambda n, xs: foldl(lambda acc, _: [s + [x] for s in acc for x in xs],
#   ↪ range(n), [[]])
# Data.List subsequences: the powerset; each element DOUBLES acc -- 2^n, fine to n ~ 20
subsequences = lambda xs: foldl(lambda acc, x: acc + [s + [x] for s in acc], list(xs),
#   ↪ [[]])
```

At the REPL:

```
>>> foldl(lambda a, x: a * 10 + x, [1, 2, 3], 0)
123
>>> compose(str.strip, str.upper)(" hi ")
```

```
'HI'  
>>> subsequences([1, 2])  
[[], [1], [2], [1, 2]]  
>>> replicateM(2, "ab")  
[['a', 'a'], ['a', 'b'], ['b', 'a'], ['b', 'b']]
```

Chapter 8

Scans, Streams, and Finite Views

A scan is a fold that keeps its history: `scanl` hands back every intermediate accumulator. Prefix sums are `scanl (+) 0`; running minima are `scanl1 min`; the depth of nested parentheses is a scan over ± 1 . When you need “the state at every step”, do not run n folds – run one scan.

Section 8 turns to streams: generators that may never end. `count`, `repeat`, `cycle`, `iterate` produce; nothing is computed until something consumes. `unfoldr` is `iterate`’s finite twin – `f(seed)` returns `(value, seed’)` or `None` to stop – and it ends the habit of threading `(state, acc)` tuples through loops. Section 9 supplies the consumers and finite views: `take`, `takeWhile/dropWhile`, `span/break_` for splitting at a condition (one pass, safe on generators), and the view factories `inits`, `tails` (every prefix and suffix – substring problems start there) and `chunksOf`. Note the deliberate naming: these carry Python’s own `itertools` names with Haskell semantics, so both vocabularies stay warm.

```
# ===== 7. scans ===== (a fold that keeps its history)

def accumulate(xs, f=None, initial=NOTHING):    # scanl / scanl1
    if f is None:
        f = add
    it = iter(xs)
    if initial is NOTHING:
        try:
            acc = next(it)
        except StopIteration:
            return
    else:
        acc = initial
    yield acc
    for x in it:
        acc = f(acc, x)
        yield acc

scanl = lambda f, z, xs: list(accumulate(xs, f, initial=z))    # foldl,
    ↪ keeping every accumulator
scanl1 = lambda f, xs: list(accumulate(xs, f))                # scanl with
    ↪ no explicit base
scanr = lambda f, z, xs: list(reversed(scanl(flip(f), z, list(reversed(list(xs)))))
    ↪ # foldr, keeping every acc
scanr1 = lambda f, xs: list(reversed(scanl1(flip(f), list(reversed(list(xs)))))
```

```

    ↪ # scanr with no base

# ===== 8. streams ===== (lazy, possibly infinite)

chain      = lambda *iterables: (x for it in iterables for x in it)    # concat, lazily

def count(start=0, step=1):                                           # [start, start+step ..]
    n = start
    while True:
        yield n
        n += step

def cycle(xs):                                                         # repeat xs forever
    xs = list(xs)
    if not xs:
        raise ValueError("cycle: empty list")    # Haskell errors too; the alternative
        ↪ is a silent hang
    while True:
        yield from xs

def iterate(f, x):                                                    # iterate f x = [x, f x, f (f x), ..]
    while True:
        yield x
        x = f(x)

def repeat(x, n=None):                                                # yield x forever, or n times if given
    if n is None:
        while True:
            yield x
    else:
        for _ in range(n):
            yield x

def unfoldr(f, seed):                                                 # Data.List unfoldr: f(seed) -> None |
    ↪ (value, seed')
    out = []                                                         # iterate's finite twin: grows a list
    ↪ until f says stop --
    step = f(seed)                                                    # no more threading (state, acc) tuples
    ↪ through until
    while step is not None:
        val, seed = step
        out.append(val)
        step = f(seed)
    return out

# ===== 9. slicing & spans ===== (finite views of streams)

def span(cond, xs):                                                  # split at first failure, ONE pass; safe
    ↪ on one-shot iters
    xs = list(xs)
    i = next((i for i, x in enumerate(xs) if not cond(x)), len(xs))
    return xs[:i], xs[i:]
break_      = lambda cond, xs: span(lambda x: not cond(x), xs)      # split at

```

```

    ↪ first hit (opposite of span)
chunksOf = lambda n, xs: [xs[i:i + n] for i in range(0, len(xs), n)]    #
    ↪ Data.List.Split; short last ok

def dropwhile(cond, xs):                                              # generator: drop the leading run where
    ↪ cond holds
    it = iter(xs)
    for x in it:
        if not cond(x):
            yield x
            break
    yield from it

dropWhile = lambda cond, xs: list(dropwhile(cond, xs))    # drop the leading run where
    ↪ cond holds, as a list
inits      = lambda xs: [xs[:i] for i in range(len(xs) + 1)]    # Data.List: every
    ↪ prefix, [] first

def islice(iterable, stop):                                          # take
    it = iter(iterable)
    for _ in range(stop):
        try:
            yield next(it)
        except StopIteration:
            return

tails      = lambda xs: [xs[i:] for i in range(len(xs) + 1)]    # every suffix;
    ↪ substrings start here
take       = lambda n, xs: list(islice(iter(xs), n))              # works on infinite
    ↪ streams

def takeuntil(cond, xs):                                            # like takewhile(not . cond), but INCLUDES
    ↪ the first hit
    for x in xs:
        yield x
        if cond(x):
            return

def takewhile(cond, xs):                                            # generator: yield the leading run where
    ↪ cond holds
    for x in xs:
        if not cond(x):
            return
    yield x

takeWhile = lambda cond, xs: list(takewhile(cond, xs))    # take the leading run where
    ↪ cond holds, as a list

```

At the REPL:

```

>>> scanl(lambda a, b: a + b, 0, [3, 1, 4])
[0, 3, 4, 8]
>>> take(5, iterate(lambda x: 2 * x, 1))
[1, 2, 4, 8, 16]

```

```
>>> unfoldr(lambda m: None if m == 0 else (m % 10, m // 10), 407)
[7, 0, 4]
>>> tails("abc")
['abc', 'bc', 'c', '']
```

Chapter 9

Order and Text

Sorting with keys is the single most-used line in any timed test. `sortOn` evaluates the key once per element; tuple keys give multi-level orderings, and the `(-count, name)` idiom gets “count descending, ties alphabetical” in one shot – something `reverse=True` cannot do. `minOn`/`maxOn` answer “best element by key” in $O(n)$; sorting first to take the head is paying $O(n \log n)$ for the same information. `merge` joins two sorted lists stably – the heart of merge sort – and the `bisect` pair performs binary search over sorted data: `bisect_left` finds the first slot $\geq x$, `bisect_right` the first $> x$; their difference counts occurrences. Section 11 is four honest aliases (`words`, `unwords`, `lines`, `unlines`) that make string pipelines read like Haskell.

```
# ===== 10. sorting & searching =====

def bisect_left(a, x):                                # first index where a[i] >= x
    lo, hi = 0, len(a)
    while lo < hi:
        mid = halve(lo + hi)
        if a[mid] < x:
            lo = mid + 1
        else:
            hi = mid
    return lo

def bisect_right(a, x):                               # first index where a[i] > x
    lo, hi = 0, len(a)
    while lo < hi:
        mid = halve(lo + hi)
        if a[mid] <= x:
            lo = mid + 1
        else:
            hi = mid
    return lo

def cmp_to_key(cmp):                                 # resurrect Python 2's cmp for sortBy
    class K:
        def __init__(self, x): self.x = x
        def __lt__(self, other): return cmp(self.x, other.x) < 0
    return K
```

```

maxOn = lambda f, xs: max(xs, key=f)      # maximumBy (comparing f) in O(n); first
    ↪ wins ties

def merge(a, b):                          # merge two sorted lists, stable; the
    ↪ heart of merge sort
    out, i, j = [], 0, 0                  # (section 13's merge_lists is this same
    ↪ loop on ListNodes)
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            out.append(a[i]); i += 1
        else:
            out.append(b[j]); j += 1
    return out + a[i:] + b[j:]

minOn = lambda f, xs: min(xs, key=f)      # minimumBy (comparing f): sortOn + head
    ↪ without the sort

sortBy = lambda cmp, xs: sorted(xs, key=cmp_to_key(cmp))  # comparator sort, Python 2
    ↪ style

sortOn = lambda f, xs: sorted(xs, key=f)  # sortOn (schwartzian, f called once per
    ↪ element)

# ===== 11. strings =====

lines = str.splitlines                    # split on newlines
unlines = lambda ls: "".join(l + "\n" for l in ls)  # join lines back with trailing
    ↪ newlines
unwords = " ".join                        # join words with single spaces
words = str.split                          # split on whitespace

```

At the REPL:

```

>>> sortOn(lambda w: (len(w), w), ["bb", "a", "ccc", "aa"])
['a', 'aa', 'bb', 'ccc']
>>> minOn(abs, [-3, 2, 5])
2
>>> bisect_right([1, 3, 3, 5], 3) - bisect_left([1, 3, 3, 5], 3)
2
>>> unwords(words(" such spacing "))
'such spacing'

```

Chapter 10

Containers: Dicts, Bags, Trees, Heaps

The longest section, and the richest. `defaultdict` rebuilds autovivification from one dunder: `touch` a missing key and the factory fills it. `Counter` is its `int` instance. Then the dict-building folds: `insertWith` combines on collision, `fromListWith` folds a pair list into a dict, combining collisions with `f(new, old)` – Haskell’s order, so groups keep arrival order when you combine `old + new` – and `unionWith` merges two dicts under any combining function. `insertWith` is persistent, returning a fresh dict like the immutable original. The bag operations show the same shape specialised to multisets – and comparing `bag_union` (`max`) with a summing merge (`+`) is a first lesson in monoids.

`Tree` and `ITree` nest `defaultdicts` into tries; `paths` folds a whole tree to `(keypath, leaf)` pairs; `setpath` writes (autovivifying), `getpath` reads *without* autovivifying – reading with `[]` would plant ghost branches. The section closes with three machines built from scratch: a two-stack `deque` with all four ends amortised $O(1)$, union-find with path halving, and a binary heap as two functions on a plain list. Owning these means never being helpless when a “batteries included” import is off the table.

```
# ===== 12. containers =====

# multiset ops on Counters (use .get, not [] -- indexing a Counter autovivifies zeros):
bag_diff = lambda a, b: {k: a[k] - b[k] for k in a if a[k] > b.get(k, 0)}    #
    ↪ clipped at 0
bag_inter = lambda a, b: {k: v for k in a.keys() & b.keys() if (v := min(a.get(k, 0),
    ↪ b.get(k, 0)))} # min count per shared key
bag_sub = lambda a, b: all(b.get(k, 0) >= n for k, n in a.items())          # a <= b
    ↪ (inclusion)
bag_union = lambda a, b: {k: max(a.get(k, 0), b.get(k, 0)) for k in a.keys() |
    ↪ b.keys()}

class defaultdict(dict):                                # dict that manufactures a value via
    ↪ factory() on first miss
    def __init__(self, factory=None, *args, **kw):
        super().__init__(*args, **kw)
        self.factory = factory
    def __missing__(self, key):
        if self.factory is None:
            raise KeyError(key)
        self[key] = self.factory()
        return self[key]
```

```

def Counter(xs):
    d = defaultdict(int)
    for x in xs:
        d[x] += 1
    return d
# count-by-value; a defaultdict(int) fold

class deque:
    def __init__(self, xs=()):
        self._in, self._out = [], list(reversed(list(xs)))
    def append(self, x): self._in.append(x)
    def appendleft(self, x): self._out.append(x)
    def pop(self):
        if not self._in:
            self._in, self._out = list(reversed(self._out)), []
        return self._in.pop()
    def popleft(self):
        if not self._out:
            self._out, self._in = list(reversed(self._in)), []
        return self._out.pop()
    def __len__(self): return len(self._in) + len(self._out)
    def __iter__(self): return iter(self._out[::-1] + self._in)
# two-stack; all four ends amortized O(1)

def dsu(n):
    ↪ if already joined
    parent = list(range(n))
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x
    def union(a, b):
        ra, rb = find(a), find(b)
        if ra == rb:
            return False
        parent[ra] = rb
        return True
    return find, union
# union-find, path halving; union -> False

def fromListWith(f, pairs):
    ↪ fold; f(new, old)
    d = {}
    ↪ concat REVERSES each
    for k, v in pairs:
        ↪ add for counts
        d[k] = f(v, d[k]) if k in d else v
    return d
# Data.Map fromListWith: THE dict-building
# like insertWith -- so grouping with
# group (the classic Haskell gotcha); use
# Counter == fromListWith(add) over (x, 1);
# grouping == fromListWith(++) over (k,
    ↪ [v])

def getpath(t, ks, default=None):
    for k in ks:
        if not isinstance(t, dict) or k not in t:
            return default
        t = t[k]
# read along ks WITHOUT autovivifying

```

```

    return t

def groupBy(eq, xs):
    ↪ elements; each new
    out = []
    ↪ run's FIRST element,
    for x in xs:
        ↪ its neighbor
        if out and eq(out[-1][0], x):
            out[-1].append(x)
        else:
            out.append([x])
    return out
# Data.List groupBy: runs of CONSECUTIVE
# element is compared via eq against the
# exactly like Haskell (span-based) -- not

group = partial(groupBy, lambda a, b: a == b) # Data.List group: runs of equals,
↪ [[a]] -- no keys

def heappop(h):
    # min-heap on a plain list: sift-down
    h[0], h[-1] = h[-1], h[0]
    top = h.pop()
    i, n = 0, len(h)
    while True:
        s, l, r = i, 2*i + 1, 2*i + 2
        if l < n and h[l] < h[s]: s = l
        if r < n and h[r] < h[s]: s = r
        if s == i:
            return top
        h[i], h[s] = h[s], h[i]
        i = s

def heappush(h, x):
    # min-heap on a plain list: sift-up
    h.append(x)
    i = len(h) - 1
    while i and h[halve(i - 1)] > h[i]:
        h[halve(i - 1)], h[i] = h[i], h[halve(i - 1)]
        i = halve(i - 1)

insertWith = lambda f, k, v, d: {**d, k: f(v, d[k]) if k in d else v} # Data.Map
↪ insertWith: PERSISTENT
# (fresh
↪ dict); f(new, old)

ITree = lambda: defaultdict(ITree) # infinitely-nestable defaultdict: a tree
↪ that grows on access

def paths(t):
    # cata for Tree/ITree: [(keypath, leaf)]
    # recursion depth = tree height (word length / len(nums)) -- well under the ~1000
    ↪ limit
    if not isinstance(t, dict) or not t: # leaf = non-dict value, or empty node
        return [[], t]
    return [[k] + p, leaf) for k, sub in t.items() for p, leaf in paths(sub)]

leaves = compose(partial(map_, snd), paths) # every leaf value in a Tree/ITree, in
↪ path order

```

```

def setpath(t, ks, v):                                # write leaf v at key path ks
    ↪ (autovivifies interior)
    for k in ks[:-1]:
        t = t[k]
    t[ks[-1]] = v                                     # NB: clobbers any subtree already at ks

Tree = lambda depth, leaf: (defaultdict(leaf) if depth == 1          # fixed-depth
    ↪ nested defaultdict;
    else defaultdict(lambda: Tree(depth - 1, leaf))) #
    ↪ leaves default to leaf()

unionWith = lambda f, a, b: {**a, **{k: f(a[k], v) if k in a else v for k, v in
    ↪ b.items()}}
    ↪ on shared keys;
    ↪ unionWith(add)                                     # Data.Map unionWith: f(a's value, b's value)
    ↪ bag_union == unionWith(max), merge-sum ==

```

At the REPL:

```

>>> fromListWith(lambda a, b: a + b, [("a", [1]), ("b", [2]), ("a", [3])])
{'a': [1, 3], 'b': [2]}
>>> unionWith(max, Counter("aab"), Counter("abb")) == {'a': 2, 'b': 2}
True
>>> t = ITree(); setpath(t, list("hi") + ["$"], "hi"); paths(t)
[(['h', 'i', '$'], 'hi')]
>>> h = []; heappush(h, 3); heappush(h, 1); heappop(h)
1

```

Chapter 11

Nodes, Grids, Windows, Control

Section 13 covers the LeetCode givens. `TreeNode` with `tfold` – a catamorphism – makes every traversal a one-liner: in-, pre-, post-order, depth and size are just different combining functions. `levelorder` is BFS, the deque earning its keep. The linked-list block collects the pointer idioms worth knowing cold: build by `foldr`, reverse with three pointers, fast/slow for the middle, Floyd for cycles, dummy-head for merges.

Section 14 gives grids their neighbour generators (bounds-checked, so BFS bodies stay clean) and `longest_window`, the sliding-window skeleton: you supply `add`, `shed` and `valid` as closures and it does the two-pointer bookkeeping. Section 15 is control: `memo`, an unbounded memo in eight lines (with the cache exposed, so you can watch a DP table fill), and `until`, the fixed-point loop – iteration without recursion depth limits.

```
# ===== 13. nodes ===== (the LeetCode givens: binary trees & linked
    ↪ lists)

class ListNode:                                # the LeetCode linked list
    def __init__(self, val=0, nxt=None):
        self.val, self.next = val, nxt

class TreeNode:                                # the LeetCode binary tree
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

from_list = lambda xs: foldr(ListNode, xs, None)    # build; foldr of ListNode

def has_cycle(node):                            # Floyd: fast laps slow iff a cycle exists
    slow = fast = node
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
    if slow is fast:
        return True
    return False

# tfold: the tree cata -- f(val, l_acc, r_acc), z for the empty tree; recursion depth
    ↪ = tree height
tfold    = lambda f, z, t: z if t is None else f(t.val, tfold(f, z, t.left), tfold(f,
    ↪ z, t.right))
inorder  = partial(tfold, lambda v, l, r: l + [v] + r, [])    # left, root, right
```

```

def levelorder(t):
    out, q = [], deque([t] if t else [])
    while len(q):
        n = q.popleft()
        out.append(n.val)
        if n.left: q.append(n.left)
        if n.right: q.append(n.right)
    return out
# BFS: the deque earning its keep

def merge_lists(a, b):
    dummy = tail_ = ListNode()
    while a and b:
        if a.val <= b.val:
            tail_.next, a = a, a.next
        else:
            tail_.next, b = b, b.next
        tail_ = tail_.next
    tail_.next = a or b
    return dummy.next
# merge two sorted lists; dummy-head idiom

def middle(node):
    slow = fast = node
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
    return slow
# fast/slow: second middle for even lengths

postorder = partial(tfold, lambda v, l, r: l + r + [v], [])
preorder = partial(tfold, lambda v, l, r: [v] + l + r, [])
# left, right, root
# root, left, right

def reverse_list(node):
    prev = None
    while node:
        node.next, prev, node = prev, node, node.next
    return prev
# three-pointer; prev is a fold accumulator

tdepth = partial(tfold, lambda _, l, r: 1 + max(l, r), 0)
to_list = compose(list, partial(unfoldr, lambda n: None if n is None else (n.val,
    ↪ n.next)))
# ListNode chain -> list

tsize = partial(tfold, lambda _, l, r: 1 + l + r, 0)
    ↪ the tree
# number of nodes in

# ===== 14. grids & windows =====

def longest_window(xs, valid, add, shed):
    ↪ the closures
    lo = best = 0
    for hi in range(len(xs)):
        add(xs[hi])
        while not valid():
            shed(xs[lo])
            lo += 1
        best = max(best, hi - lo + 1)
# shrink until legal again

```

```

    return best

def neighbors4(r, c, R, C):                # in-bounds orthogonal neighbors
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        if 0 <= r + dr < R and 0 <= c + dc < C:
            yield r + dr, c + dc

def neighbors8(r, c, R, C):                # ... plus diagonals
    for dr in (-1, 0, 1):
        for dc in (-1, 0, 1):
            if (dr or dc) and 0 <= r + dr < R and 0 <= c + dc < C:
                yield r + dr, c + dc

# ===== 15. control =====

def memo(f):                               # unbounded memoizer; enough for DP (no
    ↪ eviction by design)
    cache = {}
    def wrapped(*args, **kw):
        key = (args, frozenset(kw.items())) if kw else args
        if key not in cache:
            cache[key] = f(*args, **kw)
        return cache[key]
    wrapped.cache = cache                  # peek at the DP table if curious
    return wrapped

def until(cond, f, x):                      # until cond f x -- loop, not recursion
    ↪ (unbounded depth)
    while not cond(x):
        x = f(x)
    return x

```

At the REPL:

```

>>> t = TreeNode(2, TreeNode(1), TreeNode(3))
>>> inorder(t), tdepth(t), tsize(t)
([1, 2, 3], 2, 3)
>>> to_list(reverse_list(from_list([1, 2, 3])))
[3, 2, 1]
>>> sorted(neighbors4(0, 0, 2, 2))
[(0, 1), (1, 0)]
>>> until(lambda x: x > 100, lambda x: x * 2, 1)
128

```


Chapter 12

Deep Dives

The chapters above teach the prelude a section at a time; this one slows down for the twenty-five names where the concept is the hard part. Each dive keeps one shape: how the definition actually works (often with the evaluation traced), why it earns its place, the Haskell connection where there is one, and the sibling to contrast it with – ending in one sentence built to be remembered. Every example is a doctest from `prelude-doctests.py`, so everything here runs.

12.1 flip

[combinators] — swaps the two arguments of a function. An adapter: the function you have takes (a, b), the slot you are filling supplies (b, a). This is how the prelude derives foldr from foldl.

```
>>> sub(10, 1), flip(sub)(10, 1)
(9, -9)
>>> flip(pow)(3, 2)                # pow(2, 3)
8
```

How it works: `flip(f)` returns a new function that hands its two arguments to `f` in the opposite order. Nothing is computed at flip time – it is pure plumbing, applied when the data arrives in the "wrong" order for the function you already have.

```
>>> pair = lambda a, b: (a, b)
>>> flip(pair)(1, 2)
(2, 1)
```

Why it earns its place: the prelude's own foldr is the showcase. foldl's combiner takes (accumulator, element); a right fold's combiner takes (element, accumulator). Rather than build a second fold machine, `foldr = foldl` of the FLIPPED combiner over the reversed list – one adapter turned an existing engine into its mirror image.

```
>>> f = lambda x, acc: '(' + x + acc + ')'
>>> foldr(f, "abc", "*") == foldl(flip(f), list(reversed("abc")), "*")
True
```

Remember it as: when argument order is the only thing wrong, adapt – don't rewrite.

Roll your own — the definition:

```
flip = lambda f: (lambda x, y: f(y, x))  # flip f x y = f y x
```

12.2 curry

[*combinators*] — splits a two-argument function into two one-argument stages, so you can supply the first argument now and the second later.

```
>>> curry(add)(1)(2)
3
>>> add_ten = curry(add)(10)
>>> add_ten(5)
15
```

How it works: read the three lambdas inside-out, then watch a call peel one layer at a time. Each call binds ONE argument by closure – after `curry(add)(10)`, the 10 lives in the returned function's captured environment, and that half-applied function is a value you can name, store, and pass.

```
>>> curry(add)(10)(5)
15
>>> add_ten = curry(add)(10)
>>> add_ten(5), add_ten(90)
(15, 100)
```

Why it earns its place: the prelude's pipelines – `map_`, `filter_`, `find`, `compose` – all want ONE-argument functions in their function slot, while real logic usually takes two ("multiply by k", "is x greater than n?"). Currying is the adapter: close over the configuration argument now, hand the pipeline the unary stage it wants.

```
>>> gt = curry(lambda n, x: x > n)
>>> filter_(gt(10), [4, 11, 8, 40])
[11, 40]
>>> scale = curry(lambda k, x: k * x)
>>> map_(scale(3), [1, 2, 3])
[3, 6, 9]
```

The Haskell connection: in Haskell EVERY function is curried automatically – `add :: Int -> Int -> Int` really means `Int -> (Int -> Int)`, so `add 10` is ordinary code and `map (add 10) xs` falls out for free. Every arrow in a Haskell type is another layer of this doll.

Versus `partial`: both bind early. `partial` binds any number of arguments, all at once, when you have them; `curry` restructures the function so binding happens one call at a time, decided later by whoever holds it. In Python, `partial` is the cheaper everyday tool; `curry` is the pipeline-shaper and the reading key for Haskell signatures.

Remember it as: `curry` turns one two-argument function into two one-argument functions – and one-argument functions are the only thing pipelines eat.

Roll your own: three lambdas, each returning the next – every layer is a closure capturing one more argument until the innermost body finally has both. No state, no storage: the captured environment IS the data structure.

```
| curry = lambda f: lambda x: lambda y: f(x, y)
```

12.3 on

[combinators] — builds a two-argument function that first sends BOTH arguments through a key function: `on(f, key)(x, y) == f(key(x), key(y))`. Its natural habitat is comparators: compare two records by one field.

```
>>> on(add, len)("ab", "c")           # len("ab") + len("c")
3
>>> same_length = on(lambda a, b: a == b, len)
>>> same_length("abc", "xyz"), same_length("ab", "abc")
(True, False)
>>> sortBy(on(sub, snd), [(1, 9), (2, 3)])
[(2, 3), (1, 9)]
```

How it works: `on(f, key)` builds a two-argument function that routes BOTH inputs through key before combining them with f: `on(f, key)(x, y) == f(key(x), key(y))`. Combination logic and projection logic stay separate, so each is reusable on its own.

```
>>> shorter = on(lambda a, b: a < b, len)
>>> shorter("ab", "abc"), shorter("abc", "ab")
(True, False)
```

Why it earns its place: comparator factories. Any "compare records by one field" collapses to `on(compare, field)` – Haskell's `sortBy` (compare 'on' snd) is exactly `sortBy(on(cmp, snd))` here, and reads as its own documentation.

```
>>> sortBy(on(sub, snd), [('a', 9), ('b', 1), ('c', 5)])
[('b', 1), ('c', 5), ('a', 9)]
```

Versus `sortOn`: when a per-element key exists, `sortOn(key, xs)` is simpler and faster (the key runs once per element). `on` shines when an API demands a genuine two-argument comparator, or when the combining function is something other than comparison.

Remember it as: compare the keys, not the things.

Roll your own — the definition:

```
on = lambda f, g: lambda x, y: f(g(x), g(y))  # Data.Function: combine via a key --
      ↪ cmp `on` fst
```

12.4 NOTHING

[combinators] — a private sentinel object meaning "no argument was supplied". Why not `None`? Because `None` is a legitimate VALUE a caller might pass; `NOTHING` can only mean "omitted", since nobody else has a reference to it. Always test with 'is'. This is how `foldl` and `accumulate` tell an absent initial value from a real one.

```
>>> x = NOTHING
>>> x is NOTHING
True
>>> def greet(name=NOTHING):
...     return "hello, stranger" if name is NOTHING else "hello, " + name
>>> greet()
'hello, stranger'
>>> greet("ada")
```

```
| 'hello, ada'
```

How it works: `object()` creates a fresh, featureless object whose only property is its IDENTITY – nothing else in the program can ever be it. Testing `x` is `NOTHING` therefore asks exactly one question: "was I handed this precise sentinel?", which makes it a safe flag for "no argument was supplied".

Why not `None`, demonstrated: `None` is data (fromMaybe's whole job is handling it), so using it to also mean "argument omitted" would blur two different situations. The sentinel keeps them apart: `None` is Nothing in the DATA; `NOTHING` is Nothing in the ARGUMENT LIST.

```
>>> def describe(x=NOTHING):
...     if x is NOTHING:
...         return "no argument"
...     return "got " + repr(x)
>>> describe()
'no argument'
>>> describe(None)
'got None'
```

Where the prelude leans on it: `foldl` and `accumulate` take an optional seed. With `NOTHING` as the default they can tell "seed omitted – use the first element" from "the seed IS `None`", which a `None` default could not.

Remember it as: `None` is a value; `NOTHING` is the absence of one.

Roll your own — the definition:

```
NOTHING = object() # Maybe's Nothing: "no arg given"; test with `is`
# hoisted ahead of foldl, which needs it as a sentinel
```

12.5 fromMaybe

[combinators] — lands a default: `fromMaybe(d, x)` is `x` unless `x` is `None`, in which case it is `d`. The standard way to finish a pipeline built on the `None-is-Nothing` convention (`find`, `lookup`, `getpath`, `stripPrefix` all return `None` on failure).

```
>>> fromMaybe(0, None), fromMaybe(0, 5)
(0, 5)
>>> fromMaybe('_', find(lambda c: c == 'q', "abc"))
'_'
>>> fromMaybe(99, lookup('b', [(('a', 1), ('b', 2))]))
2
```

Start from a problem you already have: `find` looks for something that might not be there, and when it is not, you are left holding `None` – which you cannot print to a user, add, or put in a report. So you write a backup plan:

```
>>> result = find(even, [3, 5, 7])
>>> if result is None:
...     result = 0
>>> result
0
```

`fromMaybe` is those three lines as one word. Argument order: the backup plan first, then the

risky thing – said aloud it parses: "use 0 if this comes back empty".

```
>>> fromMaybe(0, find(even, [3, 5, 7]))
0
>>> fromMaybe(0, find(even, [3, 5, 8]))
8
```

Versus x or d: Python's or falls back on ANYTHING falsy – 0, "", [], False – not just None, so it silently destroys legitimate zero-ish answers. fromMaybe asks the only correct question: is this the absence marker?

```
>>> fromMaybe(99, 0), (0 or 99)
(0, 99)
```

Remember it as: ifNoneUse – the three-line backup plan as one word; or-with-a-conscience.

Roll your own — the definition:

```
fromMaybe = lambda d, x: d if x is None else x # Data.Maybe: default for every
        ↪ None-returning tool
```

12.6 isNothing

[combinators] — the mirror: True only for None. Counting failures reads as one word.

```
>>> isNothing(None), isNothing(0)
(True, False)
>>> len(filter_(isNothing, [None, 3, None]))
2
```

Together they finish the Maybe family: fromMaybe LANDS an optional value, mapMaybe/catMaybes FILTER optional values, isJust/isNothing TEST one. Haskell's Data.Maybe has all five under the same names.

```
>>> readings = [(1, None), (2, 7.5), (3, None)]
>>> fromMaybe(-1, find(isJust, map(snd, readings)))
7.5
```

Roll your own — the definition:

```
isNothing = lambda x: x is None # Data.Maybe: true when x is Nothing (None)
```

12.7 isqrt

[arithmetic & logic] — floor square root using Newton's method on INTEGERS – no floats, so no precision loss on huge numbers; a negative input raises rather than lying.

```
>>> isqrt(16), isqrt(17)
(4, 4)
>>> isqrt(10**18)
1000000000
```

Roll your own: Newton's iteration on integers – repeatedly replace x with the average of x and n // x. It converges from above, so loop while the new estimate still shrinks; all arithmetic is //, so

no float (and no float rounding lie) ever appears.

```
def isqrt(n): # floor sqrt without floats (Newton)
    if n < 0:
        raise ValueError("isqrt of negative")
    x = n
    y = halve(x + 1)
    while y < x:
        x, y = y, halve(y + n // y)
    return x if n else 0
```

12.8 transpose

[list basics] — rows become columns, RAGGED-SAFE exactly like Haskell’s: short rows simply drop out of later columns; nothing is silently truncated.

```
>>> transpose([[1, 2, 3], [4, 5, 6]])
[[1, 4], [2, 5], [3, 6]]
>>> transpose([[1, 2, 3], [4, 5]])
[[1, 4], [2, 5], [3]]
>>> transpose([])
[]
```

Rows may be any iterables, even one-shot ones – the rows are materialised first, so a generator of iterators transposes like a list of lists (without that step the generator is spent by the length scan):

```
>>> transpose(iter([1, 2]) for _ in range(2))
[[1, 1], [2, 2]]
>>> transpose(["abc", "de"])
[['a', 'd'], ['b', 'e'], ['c']]
```

Roll your own: materialise the rows, find the longest, then for each column index keep `xs[i]` only from rows that reach it – the `'if i < len(xs)'` filter is precisely where short rows drop out of later columns instead of truncating everyone.

```
def transpose(xss): # Data.List: rows <-> cols, RAGGED-SAFE like Haskell
    xss = [list(xs) for xs in xss] # any iterables in (rows may be one-shot);
    ↪ materialise once
    n = max(map(len, xss), default=0) # short rows just drop out of later columns (no
    ↪ truncation)
    return [[xs[i] for xs in xss if i < len(xs)] for i in range(n)]
```

12.9 find

[higher-order lists] — the first element satisfying the predicate, else None. Lazy: it stops at the first hit, which folds cannot do, and therefore works even on infinite streams. Finish with `fromMaybe` if you need a default.

```
>>> find(even, [1, 3, 4, 6])
4
>>> find(even, [1, 3]) is None
True
```

```
>>> find(lambda x: x % 7 == 0, count(1))
7
```

How it works: `find` wraps a generator expression in `next()` with a `None` default. The generator is the point – elements are tested one at a time, on demand, and the search STOPS at the first hit. Nothing after the match is ever examined.

Why it earns its place: `fold`s and `filter__` consume the whole list; they cannot stop early. `find` can – which makes it the right tool the moment the word "first" appears in a statement, and the only tool that survives an infinite stream. `find` defaults with `fromMaybe`.

```
>>> find(lambda x: x * x > 50, count(1))
8
>>> fromMaybe('?', find(str.isdigit, "abc7de"))
'7'
```

Versus `filter__`: `filter__` answers "which ones?"; `find` answers "who was first, if anyone?". Taking `filter__(p, xs)[0]` instead does wasted work and explodes when nothing matches; `find` does neither.

Remember it as: `find` is the early exit that `fold`s cannot perform.

Roll your own: a generator expression handed to `next()` with a default. The generator is what makes it lazy – elements are produced one at a time, so the first hit stops everything – and the default is what makes it total.

```
find = lambda cond, xs: next((x for x in xs if cond(x)), None) # first match, else
                        ↪ None
```

12.10 partition

[higher-order lists] — split into (satisfy, don't) in ONE pass, both sides in original order; the predicate runs exactly once per element, so it may be expensive or even stateful.

```
>>> partition(even, [1, 2, 3, 4])
([2, 4], [1, 3])
>>> partition(str.isalpha, "a1b2")
(['a', 'b'], ['1', '2'])
```

Roll your own: two buckets and one conditional append – (yes if `cond(x)` else no).`append(x)`. The conditional expression picks the LIST, then the append mutates it; one pass, exactly one predicate call per element.

```
def partition(cond, xs): # (keepers, rest) in ONE pass; cond called once per element
    yes, no = [], []
    for x in xs:
        (yes if cond(x) else no).append(x)
    return (yes, no)
```

12.11 mapMaybe

[higher-order lists] — map a function that returns a value or `None`, and keep only the values: parse-and-filter in ONE pass. The shape for messy input – write a parser that answers `None` for garbage, then `mapMaybe` it over everything. `dict.get` is already such a function.

```
>>> mapMaybe(lambda s: int(s) if s.isdigit() else None, ["3", "x", "7"])
[3, 7]
>>> mapMaybe({'a': 1, 'b': 2}.get, ['a', 'x', 'b'])
[1, 2]
```

How it works: run `f` over every element; `f` answers a `VALUE` when it can make sense of the input and `None` when it cannot; keep only the values. Map and filter fused into one pass, with the parser itself deciding what survives.

```
>>> parse_int = lambda s: int(s) if s.lstrip('-').isdigit() else None
>>> mapMaybe(parse_int, ["12", "x", "-3", ""])
[12, -3]
```

Why it earns its place: messy input. The alternative is a loop with an if, a flag and an append – three chances for a bug. Here the contract is crisp: write `parse(item) -> value-or-None` once, and `mapMaybe` handles every item. `dict.get` is already such a function, so lookups that may miss compose directly.

```
>>> legend = {'a': 1, 'b': 2}
>>> mapMaybe(legend.get, "cabbage")
[1, 2, 2, 1]
```

The Haskell connection: `Data.Maybe`'s `mapMaybe`, with `None` standing in for `Nothing` – the "parse, don't validate" discipline: turn questionable data into clean data at the boundary, and everything downstream stops worrying.

Remember it as: map with a parser, drop the Nothings, one pass.

Roll your own — the definition:

```
mapMaybe = lambda f, xs: [y for y in (f(x) for x in xs) if y is not None]
```

12.12 foldl

[folds] — THE left fold, and the prelude's most important definition. It is exactly this loop: `acc = base`; for `x` in `xs`: `acc = f(acc, x)`; return `acc`. The combiner takes (`ACCUMULATOR`, `element`) – accumulator first. Argument order of `foldl` itself: (`f`, `xs`, `base`), with `base` optional – omitted, the first element seeds the accumulator (and an empty list is then an error).

```
>>> foldl(lambda acc, x: acc * 10 + x, [1, 2, 3], 0)
123
>>> foldl(max, [3, 9, 2])           # no base: 3 seeds it
9
>>> foldl(add, [], 0)               # empty + base: the base comes back
0
>>> foldl(lambda acc, w: acc + len(w), ["ab", "c"], 0)
3
```

How it works: `foldl` is precisely this loop, given a name and a contract:

```
>>> def foldl_spelled_out(f, xs, base):
...     acc = base
...     for x in xs:
...         acc = f(acc, x)
...     return acc
```

```
>>> foldl_spelled_out(add, [1, 2, 3], 0) == foldl(add, [1, 2, 3], 0)
True
```

Two disciplines hide in the signature. The combiner takes (ACCUMULATOR, element) – accumulator first, always; reversing them is the classic fold bug. And the optional base is guarded by NOTHING, so the fold can tell "no seed – use the first element" from "the seed is None".

Why it earns its place: the fold is the universal list consumer. sum, max, reverse, "build a dict", "run a state machine over events" – each is one choice of combiner and seed. When you can say "carry THIS state, update it THIS way per element", the loop is already written.

```
>>> foldl(lambda acc, x: [x] + acc, [1, 2, 3], [])          # reverse is a fold
[3, 2, 1]
>>> foldl(lambda acc, w: acc + len(w), ["ab", "cde"], 0)
5
```

Remember it as: a fold is a for-loop whose state has been given a contract.

Roll your own: seed the accumulator – base if one was given, else the first element (the NOTHING guard is what lets None be a legitimate seed) – then a single for-loop folding $acc = f(acc, x)$. The entire contract is which side the accumulator sits on.

```
def foldl(f, xs, base=NOTHING): # THE left fold; Python buried its own in functools
    ↪ as reduce
    it = iter(xs)
    if base is NOTHING:
        try:
            acc = next(it)
        except StopIteration:
            raise TypeError("fold of empty sequence with no initial value")
    else:
        acc = base
    for x in it:
        acc = f(acc, x)
    return acc
```

12.13 foldr

[folds] — the RIGHT fold: nests from the right, f takes (element, ACCUMULATOR) – element first, mirror of foldl. Implemented as foldl of the flipped function over the reversed list: iterative and stack-safe, an identity that only holds because Python is strict. The parenthesisation example makes the direction visible.

```
>>> foldr(lambda x, acc: '(' + x + '+' + acc + ')', "abc", "z")
'(a+(b+(c+z)))'
>>> foldl(lambda acc, x: '(' + acc + '+' + x + ')', "abc", "z")
'(((z+a)+b)+c)'
>>> foldr(lambda x, acc: [x] + acc, [1, 2, 3], [])
[1, 2, 3]
```

How it works: the mirror image – the combiner takes (element, ACCUMULATOR) and the nesting grows from the RIGHT: $f(a, f(b, f(c, z)))$. The prelude does not write a second recursion: `foldr = foldl(flip(f), reversed(xs), base)`. Flip fixes the argument order, reversing fixes the traversal, and the existing engine runs backwards.

The honesty clause: that identity only holds because Python is STRICT. Haskell's foldr can process infinite lists – laziness lets f decide whether the rest is ever needed; ours materialises and reverses the list first, so it cannot. Same name, same finite behaviour, different superpower.

Why it earns its place: building right-to-left. Consing onto a front is natural from the right – section 13's from_list IS a foldr of ListNode, and here it is, derived by hand:

```
>>> foldr(lambda x, acc: [x] + acc, [1, 2, 3], [])      # rebuild, order kept
[1, 2, 3]
>>> to_list(foldr(ListNode, [1, 2, 3], None))          # from_list, spelled out
[1, 2, 3]
```

Remember it as: foldl eats left-to-right; foldr thinks right-to-left – and in strict Python it is foldl wearing flip and a reversal.

Roll your own — the definition:

```
foldr = lambda f, xs, base: foldl(flip(f), reversed(list(xs)), base) # THE right
      ↪ fold, via foldl
```

12.14 compose

[folds] — fold any number of functions into one; the RIGHTMOST runs first, like mathematical composition $f(g(x))$.

```
>>> compose(str.strip, str.upper)(" hi ")
'HI'
>>> f = compose(succ, succ, succ)
>>> f(0)
3
```

How it works: a fold over the functions themselves – neighbours merge into $\lambda x: f(g(x))$, seeded with the identity, so `compose()` with no arguments is `id`. The RIGHTMOST function runs first, matching mathematical $(f \circ g)(x) = f(g(x))$.

```
>>> compose()(42) == id(42)
True
>>> compose(len, str.strip)(" hi ")      # strip first, then len
2
```

Why it earns its place: it turns a pipeline into a VALUE. A composed function can be named, mapped, stored in a rules table – processing steps become data you can shuffle. The constraint is the one curry answers: every stage must be unary, which is why `compose`-heavy code leans on `curry` and `partial` to shape its stages.

```
>>> normalize = compose(str.lower, str.strip)
>>> map_(normalize, [" Hi ", " YO "])
['hi', 'yo']
```

Remember it as: `foldr (.) id` – a pipeline you can hold in your hand.

Roll your own: it IS `foldr (.) id` – `foldr` over the functions with `x` as the seed, each stage applied to the accumulated result: `foldr(lambda f, acc: f(acc), fns, x)`. Because the prelude's `foldr` is an iterative loop underneath, stack depth stays constant no matter how many stages you compose.

```
compose = lambda *fns: lambda x: foldr(lambda f, acc: f(acc), fns, x) # foldr (.) id:
```

| $\hookrightarrow$ *RIGHTMOST first*

12.15 subsequences

[*folds*] — every subset of the elements (the powerset), built by a fold in which each element **DOUBLES** the accumulator: keep every existing subset, and every existing subset extended by the newcomer. 2^n results — a brute-force license for n up to about 20, and you should say that bound out loud before using it.

```
>>> subsequences([1, 2])
[[], [1], [2], [1, 2]]
>>> subsequences("ab")
[[], ['a'], ['b'], ['a', 'b']]
>>> len(subsequences("abcd"))
16
```

How it works: watch the accumulator double. Start from `[]` — one subset, the empty one. Each element x rewrites `acc` as `acc + [s + [x] for s in acc]`: every existing subset survives (x left out), and every existing subset reappears extended by x (x taken). n elements, n doublings, 2^n subsets, in a stable order.

```
>>> acc = []
>>> acc = acc + [s + [1] for s in acc]; acc
[[], [1]]
>>> acc = acc + [s + [2] for s in acc]; acc
[[], [1], [2], [1, 2]]
```

Why it earns its place: it is a brute-force **LICENSE**. At $n \leq 20$, checking every subset is a few million cheap operations — honest, exhaustive, and unbeatable to debug. Say the bound out loud (" 2^{15} is 32768 — fine") and enumerate; cleverness can come later if the bound grows.

```
>>> best = maxOn(sum, [s for s in subsequences([3, -1, 4, -2]) if sum(s) <= 6])
>>> sum(best)
6
```

Remember it as: the powerset fold — each element doubles the world.

Roll your own — the definition:

```
subsequences = lambda xs: foldl(lambda acc, x: acc + [s + [x] for s in acc], list(xs),
     $\hookrightarrow$  [[]])
```

12.16 accumulate

[*scans*] — the generator behind the scans: yields the running combination of everything seen so far. With no function it sums; 'initial' seeds it. This is Python's `itertools` name with Haskell's scan semantics.

```
>>> list(accumulate([1, 2, 3]))
[1, 3, 6]
>>> list(accumulate([1, 2], initial=10))
[10, 11, 13]
```

```
>>> list(accumulate([3, 1, 2], min))
[3, 1, 1]
```

Roll your own: the same skeleton as `foldl`, but a GENERATOR – yield the accumulator once before the loop, then yield again after every combine. The early bare return on an empty, seedless input is what makes an empty scan come back empty instead of raising.

```
def accumulate(xs, f=None, initial=NOTHING): # scanl / scanl1
    if f is None:
        f = add
    it = iter(xs)
    if initial is NOTHING:
        try:
            acc = next(it)
        except StopIteration:
            return
    else:
        acc = initial
    yield acc
    for x in it:
        acc = f(acc, x)
        yield acc
```

12.17 scanl

[scans] — a fold that KEEPS ITS HISTORY: returns every intermediate accumulator, starting with the seed, so the result has `len(xs)+1` elements and its last element equals the `foldl`. Argument order (f, z, xs) – seed in the middle, Haskell’s order, unlike `foldl`’s trailing base. Prefix sums are the canonical scan.

```
>>> scanl(add, 0, [3, 1, 4])
[0, 3, 4, 8]
>>> last(scanl(add, 0, [1, 2, 3])) == foldl(add, [1, 2, 3], 0)
True
```

The DP secret: many celebrated one-pass algorithms are scans in a trench coat, because a scan IS a dynamic-programming table – entry i holds the answer for the prefix ending at i . Kadane’s famous maximum-subarray “trick” is one `scanl1` and a `max`:

```
>>> xs = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
>>> max(scanl1(lambda best, x: max(x, best + x), xs))
6
```

Prefix sums, the other superpower: scan once with $(+)$ and every range sum becomes two reads, `pref[j] - pref[i]` – $O(n)$ window questions collapse to $O(1)$.

```
>>> pref = scanl(add, 0, [3, 1, 4, 1, 5])
>>> pref[4] - pref[1] # sum of elements 1..3
6
```

Remember it as: when you need the state at EVERY step, one scan replaces n folds.

Roll your own — the definition:

```
scanl = lambda f, z, xs: list(accumulate(xs, f, initial=z)) # foldl, keeping every
```

| $\hookrightarrow$ *accumulator*

12.18 unfoldr

[*streams*] — iterate's FINITE twin, and the cure for threading (state, acc) tuples through loops. You supply $f(\text{seed})$ returning either (value, seed') to emit a value and continue, or None to stop; unfoldr collects the values. Anything that "peels" a structure – digits, chunks, parent chains – is an unfoldr.

```
>>> unfoldr(lambda m: None if m == 0 else (m % 10, m // 10), 407)
[7, 0, 4]
>>> unfoldr(lambda n: None if n == 0 else (n, n - 1), 5)
[5, 4, 3, 2, 1]
>>> unfoldr(const(None), "seed")          # stop immediately: empty
[]
```

How it works: the step function carries a two-outcome contract – $f(\text{seed})$ returns (value, next_seed) to emit and continue, or None to stop – and unfoldr collects what was emitted. It is the fold REVERSED: a fold consumes a list into a summary; an unfold grows a list out of a seed. (Category theory says anamorphism; interviewers say "generate the sequence".)

Why it earns its place: it retires the (state, accumulator) tuple you would otherwise thread through a while loop. Anything that PEELS – digits off a number, chunks off a list, parents up a tree, a Collatz trajectory – states its entire logic in the step function.

```
>>> unfoldr(lambda n: None if n == 1 else (n, n // 2 if even(n) else 3 * n + 1), 6)
[6, 3, 10, 5, 16, 8, 4, 2]
```

Versus iterate: iterate is the infinite cousin – an unfold whose step never says None, consumed with take. Reach for iterate when the stream is conceptually endless, unfoldr when the seed itself knows where the end is.

Remember it as: fold tears down; unfold builds up; the seed carries the future.

More unfolds, same two-outcome contract – peeling chunks off a list, bits off a number, walking a parent chain to its root:

```
>>> unfoldr(lambda xs: None if xs == [] else (xs[:2], xs[2:]), [1, 2, 3, 4, 5])
[[1, 2], [3, 4], [5]]
>>> unfoldr(lambda m: None if m == 0 else (m % 2, m // 2), 13)          # bits, LSB first
[1, 0, 1, 1]
>>> parents = {'c': 'b', 'b': 'a', 'a': None}
>>> unfoldr(lambda n: None if n is None else (n, parents[n]), 'c')      # ancestry
['c', 'b', 'a']
```

And the seed can carry several facts at once – here (current, next, remaining) generates Fibonacci with its own countdown built in:

```
>>> unfoldr(lambda s: None if s[2] == 0 else (s[0], (s[1], s[0] + s[1], s[2] - 1)),
     $\hookrightarrow$  (0, 1, 8))
[0, 1, 1, 2, 3, 5, 8, 13]
```

Roll your own: prime the pump with $\text{step} = f(\text{seed})$, then loop while step is not None – append the value, feed the new seed back to f . The two-outcome contract lives entirely in f ; unfoldr is just the loop that honours it, plus the list that collects what was emitted.

```
def unfoldr(f, seed): # Data.List unfoldr: f(seed) -> None | (value, seed')
    out = [] # iterate's finite twin: grows a list until f says stop --
    step = f(seed) # no more threading (state, acc) tuples through until
    while step is not None:
        val, seed = step
        out.append(val)
        step = f(seed)
    return out
```

12.19 span

[slicing & spans] — split at the first failure: (longest satisfying prefix, the rest), computed in ONE pass so it is safe on one-shot iterators (the old two-scan version silently broke on generators – this is the fix). A lexer in one call.

```
>>> span(lambda x: x < 3, [1, 2, 3, 4, 1])
([1, 2], [3, 4, 1])
>>> span(str.isdigit, "123abc")
(['1', '2', '3'], ['a', 'b', 'c'])
```

Roll your own: materialise xs once (that is the one-pass guarantee), locate the first failing index with next() over enumerate – defaulting to len(xs) when the predicate never fails – and slice twice. The previous two-scan version ran takewhile and dropwhile separately and silently broke on one-shot generators; this is the repair.

```
def span(cond, xs): # split at first failure, ONE pass; safe on one-shot iters
    xs = list(xs)
    i = next((i for i, x in enumerate(xs) if not cond(x)), len(xs))
    return xs[:i], xs[i:]
```

12.20 cmp_to_key

[sorting & searching] — adapts an old-style comparator (returning negative, zero or positive) into the key= protocol. Reach for it only when a rule genuinely compares two elements against each other and no per- element key exists.

```
>>> sorted(["bb", "a"], key=cmp_to_key(lambda a, b: len(a) - len(b)))
['a', 'bb']
```

Roll your own: a tiny wrapper class implementing ONE dunder – __lt__ defers to cmp(a, b) < 0. Python's sort only ever asks "is a less than b?", so that single method is enough to smuggle any comparator through the key= interface.

```
def cmp_to_key(cmp): # resurrect Python 2's cmp for sortBy
    class K:
        def __init__(self, x): self.x = x
        def __lt__(self, other): return cmp(self.x, other.x) < 0
    return K
```

12.21 merge

[sorting & searching] — join two ALREADY-SORTED lists into one sorted list, stably, in $O(n+m)$: the two-pointer loop at the heart of merge sort.

```
>>> merge([1, 4, 6], [2, 3, 7])
[1, 2, 3, 4, 6, 7]
>>> merge([], [1, 2])
[1, 2]
```

Roll your own: two cursors, always copying the smaller head forward; the $\leq$ (rather than $<$) is what makes the merge STABLE, preferring the left list on ties. When either side runs dry, glue both tails on – one of them is empty, so the $+$ is harmless.

```
def merge(a, b): # merge two sorted lists, stable; the heart of merge sort
    out, i, j = [], 0, 0 # (section 13's merge_lists is this same loop on ListNodes)
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            out.append(a[i]); i += 1
        else:
            out.append(b[j]); j += 1
    return out + a[i:] + b[j:]
```

12.22 bisect_left

[sorting & searching] — binary search over a SORTED list: the first index whose element is $\geq x$; equivalently, where x would insert to the left of its equals. If $a[i] \neq x$ there, x is absent – that check is binary search.

```
>>> bisect_left([10, 20, 20, 30], 20)
1
>>> bisect_left([10, 20, 30], 25)          # insertion point for a missing value
2
```

How it works: on a sorted list the predicate "element $< x$ " reads True for a prefix and False for the rest – TTTTFFFF. Binary search does not hunt for x ; it finds that BOUNDARY, halving the interval per probe. `bisect_left` returns the first index $\geq x$, `bisect_right` the first $> x$, and from those two facts three idioms fall out.

```
>>> xs = [10, 20, 20, 30]
>>> bisect_left(xs, 20), bisect_right(xs, 20)    # the run of 20s occupies [1, 3)
(1, 3)
```

Idiom one, membership: x is present iff the slot `bisect_left` names actually holds it. Idiom two, insertion point: that same index is where x belongs to keep the list sorted. Idiom three, counting: right minus left counts occurrences – and with two different probes, the values inside any range.

```
>>> i = bisect_left(xs, 25)
>>> i, (i < len(xs) and xs[i] == 25)            # 25 would insert at 3; absent
(3, False)
```

Why it earns its place: every threshold ladder is secretly a sorted list. Grading bands, tax brackets, rate tiers – one `bisect` replaces the `if/elif` staircase:

```
>>> grade = lambda score: "FDCBA"[bisect_right([60, 70, 80, 90], score)]
>>> grade(59), grade(60), grade(95)
('F', 'D', 'A')
```

Remember it as: bisect finds the boundary in a sea of Trues and Falses; everything else is reading that boundary three ways.

Roll your own: the lo/hi halving loop over an invariant – everything left of lo is $< x$, everything from hi rightwards is $\geq x$. One comparison, $a[mid] < x$, decides which half survives; when lo meets hi, that meeting point IS the boundary. `bisect_right` is the identical loop with $\leq$ in place of $<$ – one character moves the boundary to the far side of the equals.

```
def bisect_left(a, x): # first index where a[i] >= x
    lo, hi = 0, len(a)
    while lo < hi:
        mid = halve(lo + hi)
        if a[mid] < x:
            lo = mid + 1
        else:
            hi = mid
    return lo
```

12.23 defaultdict

[*containers*] — a dict whose missing keys create themselves via a factory: touch `d[k]` and `factory()` is installed there. This "autovivification" makes grouping and counting one-liners. Gotcha: READING a missing key also plants it – use `'in'` or `.get` when you only want to look.

```
>>> d = defaultdict(list)
>>> d['x'].append(1)
>>> d
{'x': [1]}
>>> d2 = defaultdict(int)
>>> d2['ghost'] # a mere read...
0
>>> 'ghost' in d2 # ...planted the key
True
```

How it works: one dunder does everything. When `d[k]` misses, Python calls `d.__missing__(k)`; this subclass responds by calling the stored factory, installing the result, and returning it. The factory runs only on a miss – and it takes NO arguments, which is why the leaf recipes in *Tree/ITree* are zero-argument lambdas.

Why it earns its place: it deletes the check-then-create dance from every grouping and counting loop. The write path becomes a bare `append` or `+=`, and the structure assembles itself on touch – autovivification, which section 12's trie tools then apply recursively.

The discipline it demands: a mere READ also plants the key, so split your paths – write with `d[k]`, but ask questions with `'in'` or `.get`. The prelude's bag operations read Counters with `.get` for exactly this reason.

```
>>> d = defaultdict(int)
>>> d['hits'] += 1 # write path: indexing is the point
>>> d.get('misses', 0), 'misses' in d # read path: no ghost key planted
```


Roll your own: one dict display – `**d, k: ...`: the splat copies `d` (that IS the persistence), and the new key’s value is `f(v, d[k])` on collision (new value first, Haskell’s order) or plain `v`. Compare `fromListWith`, which applies the same rule but mutates, because there the dict is its own private accumulator.

```
insertWith = lambda f, k, v, d: {**d, k: f(v, d[k]) if k in d else v} # Data.Map
    ↪ insertWith: PERSISTENT
    # (fresh dict); f(new, old)
```

12.26 fromListWith

[*containers*] — THE dict-building fold: pour (key, value) pairs into a dict, combining collisions with `f(new, old)` – Haskell’s order, like `insertWith`. Keys keep first-seen order.

```
>>> fromListWith(add, [(c, 1) for c in "aab"])
{'a': 2, 'b': 1}
>>> fromListWith(lambda new, old: old + new, [('a', [1]), ('b', [2]), ('a', [3])])
{'a': [1, 3], 'b': [2]}
```

The order gotcha, faithfully Haskell’s: because `f` receives (new, old), grouping with a bare `concat` REVERSES each group – the classic `Data.Map` surprise. Combine as `old + new` to keep arrival order, or use plain addition where order cannot matter (counts).

```
>>> fromListWith(add, [(len(w), [w]) for w in ["hi", "ox", "sun"]])
{2: ['ox', 'hi'], 3: ['sun']}
>>> fromListWith(lambda new, old: old + new, [(len(w), [w]) for w in ["hi", "ox",
    ↪ "sun"]])
{2: ['hi', 'ox'], 3: ['sun']}
```

Why it earns its place – one function, three monoids: choosing `f` is choosing what the dict MEANS. Addition over `(k, 1)` pairs counts; `old + new` over `(k, [v])` pairs groups; `max` merges bags. That is the whole family: `insertWith` is one collision, `fromListWith` a list of them, `unionWith` two dicts’ worth (where `f` receives (a’s value, b’s value)).

The recognition cue: whenever a loop builds a dict with an if-else on key existence, you are hand-rolling this fold. Name the pairs, name the monoid, and the loop disappears.

Remember it as: grouping, counting, and merging are one fold – whose combiner hears the NEW value first.

Roll your own: an empty dict and one loop applying the collision rule per pair – `d[k] = f(v, d[k])` if the key exists, else `v`. It is `insertWith` inlined and IN-PLACE: mutation is fine here because the dict being built is the fold’s own accumulator, invisible until returned.

```
def fromListWith(f, pairs): # Data.Map fromListWith: THE dict-building fold; f(new,
    ↪ old)
    d = {} # like insertWith -- so grouping with concat REVERSES each
    for k, v in pairs: # group (the classic Haskell gotcha); use add for counts
        d[k] = f(v, d[k]) if k in d else v
    return d # Counter == fromListWith(add) over (x, 1);
    # grouping == fromListWith(++) over (k, [v])
```

12.27 unionWith

[*containers*] — merge two dicts, combining SHARED keys with *f*; *a*’s keys keep their order, *b*’s new keys follow. Choosing *f* is choosing a monoid: *max* gives bag union, *+* gives a summing merge.

```
>>> unionWith(max, {'a': 1, 'b': 5}, {'b': 2, 'c': 3})
{'a': 1, 'b': 5, 'c': 3}
>>> unionWith(add, {'a': 1, 'b': 2}, {'b': 3, 'c': 4})
{'a': 1, 'b': 5, 'c': 4}
```

Roll your own: two splats — ***a, **...*: *a* copied whole, then overlaid with *b*’s items after each has been through the collision rule, *f(a[k], v)* when *k* is shared, plain *v* when it is new. The asymmetry is deliberate: *a*’s keys keep their positions, and only keys new to *b* append at the end.

```
unionWith = lambda f, a, b: {**a, **{k: f(a[k], v) if k in a else v for k, v in
    ↪ b.items()}}
# Data.Map unionWith: f(a's value, b's value) on shared keys;
# bag_union == unionWith(max), merge-sum == unionWith(add)
```

12.28 groupBy

[*containers*] — runs by an equivalence predicate; each newcomer is compared with *eq* against the run’s FIRST element (span-based, exactly Haskell’s *groupBy*), not against its neighbour — a subtle and deliberate fidelity.

```
>>> groupBy(lambda a, b: a == b, [1, 1, 2])
[[1, 1], [2]]
>>> groupBy(lambda a, b: b - a <= 1, [1, 2, 3, 10, 11])
[[1, 2], [3], [10, 11]]
```

The word that matters is CONSECUTIVE: a new run starts whenever *eq* against the run’s first element fails, so ‘abab’ yields four runs. Not a weakness — a different question: runs, streaks and compression care about position, and these are the only tools asking about it.

```
>>> [(run[0], len(run)) for run in group("aaabbbbcc")]      # run lengths
[('a', 3), ('b', 4), ('c', 2)]
```

For CATEGORIES — position irrelevant — either sort first, so equals become adjacent, or use *fromListWith*, which never cared about position at all. Choosing between them is stating what your key means.

```
>>> [(run[0], len(run)) for run in group(sorted("abab"))]
[('a', 2), ('b', 2)]
```

Remember it as: *group* answers “how long are the runs?”; *fromListWith* answers “what are the piles?”.

Roll your own: one pass in which each element either joins the current run or starts a fresh one. The test is *eq(out[-1][0], x)* — against the run’s FIRST element, not its last — which is exactly how Haskell’s span-based *groupBy* behaves, and the detail that makes non-transitive predicates (like “within 1 of”) agree with it.

```
def groupBy(eq, xs): # Data.List groupBy: runs of CONSECUTIVE elements; each new
    out = [] # element is compared via eq against the run's FIRST element,
```

```

for x in xs: # exactly like Haskell (span-based) -- not its neighbor
    if out and eq(out[-1][0], x):
        out[-1].append(x)
    else:
        out.append([x])
return out

```

12.29 ITree

[*containers*] — the INFINITE tree: every branch is another ITree, so any depth autovivifies on touch. This is the trie: index letter by letter and the branch builds itself.

```

>>> t = ITree()
>>> t['x']['y']['z'] = 9
>>> t
{'x': {'y': {'z': 9}}}

```

How it works: the definition is a knot – `ITree = lambda: defaultdict(ITree)`. Every missing key manufactures another ITree, which will do the same, without end. Indexing therefore GROWS the tree: touching `t['c']['a']['t']` conjures the whole branch. A trie in one line, built from nothing but `missing`.

```

>>> t = ITree()
>>> for w in ["cat", "car"]: setpath(t, list(w) + ['$'], w)
>>> sorted([leaf for p, leaf in paths(t) if p[-1] == '$'])
['car', 'cat']

```

Why it earns its place: hierarchies – tries, directory trees, nested groupings – stop needing setup code. Writing is indexing (or `setpath`, which also plants a leaf); harvesting is `paths`. The one edged rule: reads autovivify too, so QUESTIONS go through `getpath`, never bare indexing – `getpath`'s entry demonstrates the ghost branch this avoids.

Versus `Tree(depth, leaf)`: `Tree` is the finite cousin – fixed depth, with a real leaf factory (`list, int`) at the bottom instead of more tree. `Tree(1, list)` is the grouping dict; `ITree` is for depth you cannot know in advance, like words.

Remember it as: a dict that dreams up more of itself on demand.

Roll your own: `ITree = lambda: defaultdict(ITree)` mentions itself – legal, because the lambda body only runs on a MISS, by which time the name is bound. `Tree` bottoms the same recursion out at a fixed depth by handing the last level a real leaf factory instead of more tree.

```

ITree = lambda: defaultdict(ITree) # infinitely-nestable defaultdict: a tree that
    ↪ grows on access

```

12.30 paths

[*containers*] — fold a whole `Tree/ITree` into a flat list of (keypath, leaf) pairs, in insertion order. The read- everything companion to `setpath`'s write-one-path.

```

>>> t = ITree()
>>> setpath(t, ['a', 'b'], 1); setpath(t, ['a', 'c'], 2)
>>> paths(t)

```

```
| [[('a', 'b'), 1], (('a', 'c'), 2)]
```

How it works: structural recursion with two arms, exactly the case-arms-as-fold story. A non-dict (or empty node) is a LEAF: report the single row ([], leaf). A dict is a BRANCH: recurse into every subtree and prepend the branching key to each keypath that comes back. The whole tree flattens into (keypath, leaf) rows, insertion-ordered, losslessly.

Why it earns its place: building a trie is the easy half; the value is HARVESTING it. autocomplete filters rows under a prefix, deepest_directory takes maxOn of keypath length, leaves reads the second column. One cata, many harvests – you never write tree-walking code again.

```
>>> t = ITree()
>>> setpath(t, ['a', 'b'], 1); setpath(t, ['c'], 2)
>>> paths(t)
[[('a', 'b'), 1], (('c'), 2)]
>>> maxOn(lambda row: len(row[0]), paths(t))
(('a', 'b'), 1)
```

Remember it as: paths turns a tree back into rows; setpath turns rows into a tree.

Roll your own: recursion on one question – is this still a dict? A leaf reports a single row with an empty path; a branch recurses into every child and conses its key onto every path that comes back. Depth equals tree height, comfortably inside Python’s limit for sane tries.

```
def paths(t): # cata for Tree/ITree: [(keypath, leaf)]
    # recursion depth = tree height (word length / len(nums)) -- well under the ~1000
    ↪ limit
    if not isinstance(t, dict) or not t: # leaf = non-dict value, or empty node
        return [([], t)]
    return [(k + p, leaf) for k, sub in t.items() for p, leaf in paths(sub)]
```

12.31 setpath

[containers] — write a leaf value at the end of a key path, autovivifying the branch on the way. Caution: writing at an interior key CLOBBERS the subtree below it.

```
>>> t = ITree()
>>> setpath(t, ['a', 'b'], 7)
>>> t
{'a': {'b': 7}}
>>> setpath(t, ['a'], 0) # clobbers {'b': 7}
>>> t
{'a': 0}
```

Roll your own: walk all but the last key by PLAIN INDEXING – on an ITree that autovivifies the branch as you go – then assign at the final key, the single write. getpath is the mirror image with the safety catch on: isinstance and 'in' checks instead of indexing, so reading plants nothing.

```
def setpath(t, ks, v): # write leaf v at key path ks (autovivifies interior)
    for k in ks[:-1]:
        t = t[k]
    t[ks[-1]] = v # NB: clobbers any subtree already at ks
```

12.32 deque

[*containers*] — a double-ended queue built from two stacks, all four end operations amortised $O(1)$ (elements migrate between stacks only when one runs dry). This is the BFS frontier; a plain list's `pop(0)` is $O(n)$.

```
>>> d = deque([1, 2, 3])
>>> d.appendleft(0); d.append(4)
>>> d.popleft(), d.pop()
(0, 4)
>>> len(d), list(d)
(3, [1, 2, 3])
```

How it works: two plain lists, nose to nose – `_out` holds the front (reversed), `_in` holds the back. `append` pushes onto `_in`, `appendleft` onto `_out`, both $O(1)$. The trick is popping an EMPTY side: the other list is reversed across in one $O(n)$ migration, and popping is $O(1)$ again.

Why that still counts as $O(1)$: amortization. Each element migrates at most once between being pushed and popped, so n operations cost $O(n)$ in total – constant per operation on average, which is what the workload's big-O cares about. A bare Python list cannot say this: `pop(0)` shifts every element, every single time.

```
>>> q = deque()
>>> for x in [1, 2, 3]: q.append(x)
>>> q.popleft(), q.popleft()          # first pop pays the migration; the next is
    ↪ free
(1, 2)
```

Why it earns its place: BFS. The frontier must yield its OLDEST member (`popleft`) while newcomers join at the back (`append`) – `levelorder` and `grid_path` are this class doing its one job.

Remember it as: two stacks facing each other make a queue; the reversal bill is paid once per element.

Roll your own: two plain lists nose to nose, each end appending and popping its own list at $O(1)$. All the cleverness is in the pop guard: when your side is empty, reverse the OTHER list across in one go. Each element makes that crossing at most once between entering and leaving, so the occasional $O(n)$ reversal amortises to $O(1)$ per operation.

```
class deque: # two-stack; all four ends amortized O(1)
    def __init__(self, xs=()):
        self._in, self._out = [], list(reversed(list(xs)))
    def append(self, x): self._in.append(x)
    def appendleft(self, x): self._out.append(x)
    def pop(self):
        if not self._in:
            self._in, self._out = list(reversed(self._out)), []
        return self._in.pop()
    def popleft(self):
        if not self._out:
            self._out, self._in = list(reversed(self._in)), []
        return self._out.pop()
    def __len__(self): return len(self._in) + len(self._out)
    def __iter__(self): return iter(self._out[::-1] + self._in)
```

12.33 dsu

[containers] — disjoint-set union (union-find) over n elements, with path halving. find gives a set's representative; union merges and returns False when the two were ALREADY connected – which is exactly a cycle detector (Kruskal's test).

```
>>> find, union = dsu(4)
>>> union(0, 1), union(1, 2)
(True, True)
>>> union(0, 2)                                # already connected: a cycle
False
>>> find(0) == find(2), find(0) == find(3)
(True, False)
```

How it works: `parent[]` links every element toward its set's ROOT, and `find` follows the chain up. The subtle line is `parent[x] = parent[parent[x]]` – path HALVING: every `find` re-points each visited node at its grandparent, so chains shrink as a side effect of being walked. That one line makes operations effectively constant time (inverse Ackermann, for the curious).

Why it earns its place: dynamic connectivity. "Are these two in the same group yet?" under a stream of merges is awkward for every other structure and trivial here. And union returning False – already connected – IS cycle detection: Kruskal's spanning tree is built by skipping every False.

```
>>> find, union = dsu(5)
>>> [union(a, b) for a, b in [(0, 1), (1, 2), (3, 4), (0, 2)]]
[True, True, True, False]
>>> len({find(i) for i in range(5)})    # two components remain
2
```

Remember it as: `find` walks to the root, halving as it goes; a False union is a cycle caught.

Roll your own: `parent[]` as the forest and two closures instead of a class. `find`'s loop carries the entire trick in one line – `parent[x] = parent[parent[x]]` re-points every visited node at its grandparent (path halving), flattening chains as a side effect of walking them. `union` is `find` twice plus one root re-point, answering False when the roots already agree.

```
def dsu(n):  # union-find, path halving; union -> False if already joined
    parent = list(range(n))
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x
    def union(a, b):
        ra, rb = find(a), find(b)
        if ra == rb:
            return False
        parent[ra] = rb
        return True
    return find, union
```

12.34 heappush

[*containers*] — push onto a min-heap kept in a plain list (sift-up). The smallest element is always `h[0]`. For a MAX-heap, push negated values.

```
>>> h = []
>>> for x in [5, 1, 8]: heappush(h, x)
>>> h[0]
1
```

How it works: the flat list IS a binary tree – `h[i]`'s children live at $2i+1$ and $2i+2$ – kept under one invariant: every parent $\leq$ its children. `heappush` appends at the first free leaf and sifts UP, swapping with its parent while smaller; `heappop` moves the last leaf to the root and sifts DOWN toward the smaller child. Each walks one root-to-leaf path: $O(\log n)$.

Why the invariant is enough: nothing is fully sorted – siblings sit in any order – yet `h[0]` is ALWAYS the minimum, because the root beats its children, who beat theirs. You pay $\log n$ per operation for a permanently readable minimum; draining the heap is `heapsort`; pushing negatives turns min into max.

```
>>> h = []
>>> for x in [7, 2, 9, 4]: heappush(h, x)
>>> h[0]
2
>>> [heappop(h) for _ in range(4)]
[2, 4, 7, 9]
```

Why it earns its place: "repeatedly take the best next thing" – Dijkstra's frontier, running `_median`'s two halves, any streaming top-k. When next-best is needed many times while the data keeps changing, the heap beats re-sorting every time.

Remember it as: a flat list, a parent-beats-children promise, and $\log n$ repairs.

Roll your own: append at the first free leaf, then sift UP – while the parent at $(i - 1) // 2$ is larger, swap and climb. The index arithmetic IS the tree; watch the array after four pushes:

```
>>> h = []
>>> for x in [7, 2, 9, 4]: heappush(h, x)
>>> h
[2, 4, 9, 7]
# root 2; children 4 and 9; 7 pushed down under 4
```

```
def heappush(h, x): # min-heap on a plain list: sift-up
    h.append(x)
    i = len(h) - 1
    while i and h[halve(i - 1)] > h[i]:
        h[halve(i - 1)], h[i] = h[i], h[halve(i - 1)]
        i = halve(i - 1)
```

12.35 heappop

[*containers*] — remove and return the minimum (sift-down). Popping repeatedly drains in sorted order – that is `heapsort`.

```
>>> h = []
>>> for x in [5, 1, 8]: heappush(h, x)
```

```
>>> [heappop(h) for _ in range(3)]
[1, 5, 8]
>>> g = []
>>> for x in [3, 9, 5]: heappush(g, -x)
>>> -heappop(g)                # max-heap via negation
9
```

Roll your own: swap the root with the last leaf, pop it off the end, then sift DOWN – repeatedly swap with the SMALLER of the two children (choosing the smaller is the classic bug site) until neither child is smaller. One root-to-leaf path each way: $O(\log n)$.

```
def heappop(h): # min-heap on a plain list: sift-down
    h[0], h[-1] = h[-1], h[0]
    top = h.pop()
    i, n = 0, len(h)
    while True:
        s, l, r = i, 2*i + 1, 2*i + 2
        if l < n and h[l] < h[s]: s = l
        if r < n and h[r] < h[s]: s = r
        if s == i:
            return top
        h[i], h[s] = h[s], h[i]
        i = s
```

12.36 tfold

[nodes] — the tree catamorphism: recursively combines each node’s value with the already-folded left and right results, *z* standing in for empty subtrees. EVERY structural tree computation is one choice of *f* and *z* – the five traversal/measure functions below are all tfold one-liners.

```
>>> t = TreeNode(2, TreeNode(1), TreeNode(3))
>>> tfold(lambda v, l, r: v + l + r, 0, t)    # sum
6
>>> tfold(lambda v, l, r: max(v, l, r), 0, t) # max
3
```

How it works: the tree catamorphism – structural recursion with a contract. Two arms, exactly like a case expression: the empty tree answers *z*; a node answers *f*(value, left_result, right_result), both results computed by the same recursion. Hand it (*f*, *z*) and any question about a tree becomes one line – the five traversals and both measures in section 13 are all tfold with different arms.

Why it earns its place: it separates WALKING from DECIDING. The recursion is written once, correct forever; you only ever author the arms. Even tree TRANSFORMATION fits – mirror a tree by swapping the child results while rebuilding:

```
>>> t = TreeNode(2, TreeNode(1), TreeNode(3))
>>> mirror = tfold(lambda v, l, r: TreeNode(v, r, l), None, t)
>>> inorder(mirror)
[3, 2, 1]
```

The one caveat: recursion depth equals tree height – fine for balanced and interview-sized trees; say a sentence out loud about adversarially skewed ones.

Remember it as: *z* is the empty-tree arm, *f* is the node arm – the fold walks, you decide.

Roll your own: one line of structural recursion – `z` when the tree is `None`, else `f` over the node's value and the two recursive results.

Roll your own: one line of structural recursion – `z` when the tree is `None`, else `f` over the node's value and the two recursive results. Every traversal and measure in this section is that line with a different `f` plugged in.

```
tfold = lambda f, z, t: z if t is None else f(t.val, tfold(f, z, t.left), tfold(f, z,
    ↪ t.right))
```

12.37 levelorder

[nodes] — breadth-first values, level by level: the deque earning its keep. Compare preorder on the same tree to see DFS vs BFS.

```
>>> t = TreeNode(1, TreeNode(2, TreeNode(4)), TreeNode(3))
>>> levelorder(t)
[1, 2, 3, 4]
>>> preorder(t)
[1, 2, 4, 3]
```

Roll your own: seed a deque with the root, then loop – `popleft`, record the value, append the children that exist. FIFO order IS breadth-first; swap the deque for a list used as a stack and the identical loop turns into depth-first.

```
def levelorder(t): # BFS: the deque earning its keep
    out, q = [], deque([t] if t else [])
    while len(q):
        n = q.popleft()
        out.append(n.val)
        if n.left: q.append(n.left)
        if n.right: q.append(n.right)
    return out
```

12.38 reverse_list

[nodes] — reverse in place with the three-pointer walk; `prev` is a fold accumulator wearing pointers.

```
>>> to_list(reverse_list(from_list([1, 2, 3])))
[3, 2, 1]
```

Roll your own: the three-pointer walk compressed into one simultaneous assignment – `node.next`, `prev`, `node = prev`, `node, node.next`. The right-hand side is evaluated in full before any assignment lands, which is what saves the pointer you are about to overwrite.

```
def reverse_list(node): # three-pointer; prev is a fold accumulator
    prev = None
    while node:
        node.next, prev, node = prev, node, node.next
    return prev
```

12.39 middle

[nodes] — the middle node by fast/slow pointers (fast moves twice per step); on even lengths, the SECOND middle.

```
>>> middle(from_list([1, 2, 3])).val
2
>>> middle(from_list([1, 2, 3, 4])).val
3
```

Roll your own: fast advances two hops for slow's one, so when fast falls off the end, slow stands at the middle. The loop condition (fast and fast.next) is precisely what selects the SECOND middle on even lengths.

```
def middle(node): # fast/slow: second middle for even lengths
    slow = fast = node
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
    return slow
```

12.40 has_cycle

[nodes] — Floyd's tortoise and hare: the fast pointer laps the slow one if and only if a cycle exists; O(1) space.

```
>>> has_cycle(from_list([1, 2, 3]))
False
>>> n = ListNode(1); n.next = n          # a self-loop
>>> has_cycle(n)
True
```

Roll your own: the same fast/slow walk, but watching for the pointers to MEET – on a cycle the fast runner laps the slow one; on a straight list it simply falls off. O(1) space, no visited set.

```
def has_cycle(node): # Floyd: fast laps slow iff a cycle exists
    slow = fast = node
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
        if slow is fast:
            return True
    return False
```

12.41 merge_lists

[nodes] — merge two SORTED linked lists, using the dummy-head idiom to avoid special-casing the first node.

```
>>> to_list(merge_lists(from_list([1, 3]), from_list([2, 4])))
[1, 2, 3, 4]
```

Roll your own: the dummy-head idiom – a throwaway node to hang the result from, so appending the first real node needs no special case; walk both lists copying the smaller head, glue the survivor,

return dummy.next.

```
def merge_lists(a, b): # merge two sorted lists; dummy-head idiom
    dummy = tail_ = ListNode()
    while a and b:
        if a.val <= b.val:
            tail_.next, a = a, a.next
        else:
            tail_.next, b = b, b.next
        tail_ = tail_.next
    tail_.next = a or b
    return dummy.next
```

12.42 neighbors4

[grids & windows] — the in-bounds orthogonal neighbours of cell (r, c) in an R-by-C grid. Bounds checking lives HERE, so BFS bodies stay clean.

```
>>> sorted(neighbors4(0, 0, 2, 2)) # corner: two neighbours
[(0, 1), (1, 0)]
>>> sorted(neighbors4(1, 1, 3, 3)) # centre: four
[(0, 1), (1, 0), (1, 2), (2, 1)]
```

Roll your own: yield each in-bounds delta of (r, c) — the bounds check lives HERE so every caller's BFS body stays clean. neighbors8 is the same generator over a 3x3 delta grid, with (dr or dc) excluding the centre cell.

```
def neighbors4(r, c, R, C): # in-bounds orthogonal neighbors
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        if 0 <= r + dr < R and 0 <= c + dc < C:
            yield r + dr, c + dc
```

12.43 longest_window

[grids & windows] — the sliding-window skeleton: it grows the right edge one element at a time (calling add), shrinks the left edge while your valid() says the window is illegal (calling shed), and tracks the best length. YOU supply the state as closures; the skeleton does the two-pointer bookkeeping. Longest- substring-without-repeats and longest-run-under-a- budget are both three closures away.

```
>>> seen = []
>>> longest_window("abcabb", lambda: len(seen) == len(set(seen)),
...               seen.append, lambda c: seen.pop(0))
3
>>> w = []
>>> longest_window([2, 1, 3, 4], lambda: sum(w) <= 6,
...               w.append, lambda x: w.pop(0))
3
```

How it works: inversion of control. The skeleton owns the two pointers — it grows hi one element per step (calling add), then shrinks lo while your valid() reports the window broken (calling shed

per evicted element) – and it tracks the best length ever seen legal. You own only the STATE, held in closures over variables sitting next to the call.

Designing the closures is answering one question: "what makes a window ILLEGAL, and what must I track to know?" Distinct characters -> a duplicate count. A budget -> a running sum. At most k of something -> a counter. add and shed are that state's increment and decrement; valid is the legality test.

```
>>> w = []
>>> longest_window("aabcb", lambda: len(w) == len(set(w)), w.append, lambda c:
    ↪ w.pop(0))
3
```

Why closures and not a class: the state lives exactly as long as the call, sits in scope where you can read it, and needs no ceremony. Functions carrying state – the same muscle as curry's captured argument, here working a two-pointer job.

Remember it as: the skeleton slides the window; your three closures say what legal means.

Roll your own: hi sweeps forward exactly once, add() greeting each newcomer; an inner while shrinks lo, shed() by shed(), until valid() approves again. Every element is added once and removed at most once – the entire two-pointer $O(n)$ argument, visible in two loops.

```
def longest_window(xs, valid, add, shed): # sliding-window skeleton; state lives in
    ↪ the closures
    lo = best = 0
    for hi in range(len(xs)):
        add(xs[hi])
        while not valid(): # shrink until legal again
            shed(xs[lo])
            lo += 1
        best = max(best, hi - lo + 1)
    return best
```

12.44 memo

[control] — an unbounded memoizer as a plain decorator (no parentheses): results are cached by the argument tuple, so each distinct call computes once. Keyword arguments fold into the key; the cache is exposed as .cache.

```
>>> @memo
... def fib(n): return n if n < 2 else fib(n - 1) + fib(n - 2)
>>> fib(30)
832040
>>> fib.cache[(10,)]
55
>>> len(fib.cache) # one entry per distinct argument
31
```

How it works: the decorator wraps f with a dict keyed on the arguments. First call computes and stores; every repeat is a lookup. That is the entire mechanism – and why arguments must be HASHABLE: lists become tuples before they can be keys.

Why it earns its place: memoized recursion IS dynamic programming. Write the honest recurrence – the answer for state (i, j) in terms of smaller states – decorate it, and the cache is your

DP table, filled in dependency order with no index bookkeeping. The exposed `.cache` lets you SEE the table: its size is the state count, exactly the number complexity analysis wants said out loud.

```
>>> @memo
... def ways(r, c):                # lattice paths to (r, c), moving right/down
...     return 1 if r == 0 or c == 0 else ways(r - 1, c) + ways(r, c - 1)
>>> ways(3, 3)
20
>>> len(ways.cache)                # the 4x4 grid minus (0,0), never needed
15
```

Versus `functools.lru_cache`: the standard library's version adds eviction (`maxsize`); `memo` is deliberately unbounded, because DP wants every state kept – and it is nine lines you can retype in any interview that bans imports.

Remember it as: recursion states the truth; the cache makes it affordable; `.cache` is the table.

Roll your own: a dict captured in a closure, keyed by the argument tuple (keyword arguments folded in as a `frozenset` when present – tuples and `frozensets` because keys must be hashable). Compute on miss, look up on hit, and hang the dict on the wrapper as `.cache` so the DP table stays inspectable.

```
def memo(f): # unbounded memoizer; enough for DP (no eviction by design)
    cache = {}
    def wrapped(*args, **kw):
        key = (args, frozenset(kw.items())) if kw else args
        if key not in cache:
            cache[key] = f(*args, **kw)
        return cache[key]
    wrapped.cache = cache # peek at the DP table if curious
    return wrapped
```

12.45 until

[control] — iterate `f` from `x` until the predicate holds, returning the first satisfying value: a fixed-point loop with no recursion-depth limit. The state can be a tuple carrying whatever the loop must remember.

```
>>> until(lambda x: x > 100, lambda x: x * 2, 1)
128
>>> collatz = lambda s: (s[0] // 2 if even(s[0]) else 3 * s[0] + 1, s[1] + 1)
>>> until(lambda s: s[0] == 1, collatz, (6, 0))    # 8 steps to reach 1
(1, 8)
```

How it works: keep applying `f` until `p` approves – iteration to a goal or a FIXED POINT, as a loop rather than recursion, so depth is unbounded. The state can be anything; a tuple lets one loop carry several facts at once, like the `collatz` example's (value, steps).

Why it earns its place: some processes have no list to fold over – they run "until stable". Newton's `isqrt` converges; `bellman_ford` relaxes every edge until the distance map stops changing (the textbook fixed point; `g7`'s hint calls it by name). `until` states such loops declaratively: here is the step, here is done.

```
>>> until(lambda n: n * n >= 30, succ, 0)          # smallest n with n squared >= 30
6
```

```
>>> until(lambda xs: xs == sorted(xs), sorted, [3, 1, 2])  
[1, 2, 3]
```

Remember it as: while-not-done, promoted to an expression that returns its answer.

Roll your own: while not p(x): x = f(x); return x. Iteration promoted to a value-returning expression – and a loop rather than recursion on purpose, because a fixed point may take unboundedly many steps.

```
def until(cond, f, x): # until cond f x -- loop, not recursion (unbounded depth)  
    while not cond(x):  
        x = f(x)  
    return x
```


Part III

The Nine Problem Classes

Chapter 13

Class I — Arithmetic and Unfolds

The warm-up class, and the home of the *unfold*: producing a sequence from a seed instead of consuming one. Digits, base renderings and numeral systems all peel a value apart step by step – `unfoldr` when you want the pieces as a list, `until` when you only want the final state, `foldl` when a value table drives the steps. Watch how often a produced list is immediately folded back into a value: unfold-then-fold is a pipeline, not a coincidence. *Practice files live in `g1_arith_and_unfolds/`;*

each carries the prelude snippets it needs inline. Attempt every problem there before reading on – the doctests are the referee.

13.1 `sign`: Sign of a number

Return -1 for negative, 0 for zero, 1 for positive.

Contract:

`sign(x: int | float) -> int`

Hint:

The prelude ships this exact function: `signum`, one comparison subtraction, no branches. Recognize it, don't reinvent it.

Doctests:

```
>>> sign(-7)
-1
>>> sign(0)
0
>>> sign(3)
1
>>> sign(-0.5)
-1
```

Solution.

```
sign = signum
```

13.2 rect_area: Rectangle area

Contract:

`rect_area(w, h) -> w * h` (result type follows the inputs)

Hint:

Deliberate overkill: area is the numeric fold, `product([w, h])`.
See the smallest possible fold once; recognize every fold after.

Doctests:

```
>>> rect_area(3, 4)
12
>>> rect_area(2.5, 4)
10.0
>>> rect_area(0, 99)
0
```

Solution.

```
rect_area = lambda w, h: product([w, h])
```

13.3 reverse_digits: Reverse an integer's digits

Negatives keep their sign; trailing zeros vanish.

Contract:

`reverse_digits(n: int) -> int`

Hint:

`unfoldr` peels the digits with `(m % 10, m // 10)` -- least significant first, which IS reversed order. Fold them back into a number with `acc * 10 + d`; `signum` reapplies the sign.

Doctests:

```
>>> reverse_digits(123)
321
>>> reverse_digits(-45)
-54
>>> reverse_digits(120)
21
>>> reverse_digits(7)
7
```

Solution.

```
def reverse_digits(n):
    digits = unfoldr(lambda m: None if m == 0 else (m % 10, m // 10), abs(n))
    return signum(n) * foldl(lambda a, d: a * 10 + d, digits, 0)
```

Remark. The state never needed to be a tuple threaded through `until` – `unfoldr` peels the digits into a list (already reversed, because `m % 10` yields the low digit first), and a fold reassembles them. Unfold, then fold: when you see that pair, you are looking at a pipeline.

13.4 to_base: Render n in base b

Non-negative n, base 2..16, uppercase digits.

Contract:

```
to_base(n: int, b: int) -> str
```

Hint:

```
unfoldr emits digits low-to-high -- "0123456789ABCDEF"[m % b],
seed m // b, stop at 0. Reverse and join; `or "0"` covers zero
(its unfold is empty).
```

Doctests:

```
>>> to_base(255, 16)
'FF'
>>> to_base(5, 2)
'101'
>>> to_base(0, 8)
'0'
>>> to_base(100, 10)
'100'
>>> to_base(31, 16)
'1F'
```

Solution.

```
DIGITS = "0123456789ABCDEF"
```

```
def to_base(n, b):
    ds = unfoldr(lambda m: None if m == 0 else (DIGITS[m % b], m // b), n)
    return "".join(reversed(ds)) or "0"
```

13.5 to_roman: Integer to Roman numeral, 1 ≤ n ≤ 3999

Contract:

```
to_roman(n: int) -> str
```

Hint:

```
A left fold over the value table (subtractive pairs included:
900 'CM', 40 'XL', ...) carrying (remaining, out): each step
appends sym * (remaining // val) and keeps remaining % val.
```

Doctests:

```
>>> to_roman(1994)
'MCMXCIV'
>>> to_roman(9)
'IX'
>>> to_roman(58)
'LVIII'
>>> to_roman(3000)
'MMM'
```

```
>>> to_roman(1)
'I'
```

Solution.

```
TABLE = [(1000, "M"), (900, "CM"), (500, "D"), (400, "CD"), (100, "C"),
          (90, "XC"), (50, "L"), (40, "XL"), (10, "X"), (9, "IX"),
          (5, "V"), (4, "IV"), (1, "I")]

def to_roman(n):
    step = lambda acc, vs: (acc[0] % vs[0], acc[1] + vs[1] * (acc[0] // vs[0]))
    return foldl(step, TABLE, (n, ""))[1]
```

13.6 from_roman: Parse a valid Roman numeral

```
Contract:
    from_roman(s: str) -> int

Hint:
    pairwise gives each symbol its right neighbor; a value counts
    NEGATIVE when its neighbor is bigger ('IV' = -1 + 5). Pad the
    value list with a trailing 0 so the last symbol gets a neighbor
    too, then sum.
```

Doctests:

```
>>> from_roman('MCMXCIV')
1994
>>> from_roman('IX')
9
>>> from_roman('LVIII')
58
>>> from_roman('III')
3
>>> from_roman('MMM')
3000
```

Solution.

```
VAL = {"I": 1, "V": 5, "X": 10, "L": 50, "C": 100, "D": 500, "M": 1000}

def from_roman(s):
    vs = map_(lambda c: VAL[c], s) + [0]
    return sum(map_(lambda p: -p[0] if p[0] < p[1] else p[0], pairwise(vs)))
```

Remark. Every subtractive rule in Roman numerals is a statement about a symbol and its *right neighbour*. `pairwise` materialises exactly that relationship, and the padding zero gives the final symbol a neighbour too. When a rule talks about adjacent elements, reach for `pairwise` before you reach for indices.

Chapter 14

Class II — Lists and Strings

Bread and butter: `map_`, `filter_`, `concat`, `concatMap`, and the string pair `words/unwords`. Two problems here are deliberate one-liners (`flatten is concat`, `transpose_matrix is transpose`): recognising an existing tool is a skill worth isolating and drilling. The harder members (`chunks`, `permutations`) preview unfolds and backtracking enumeration. *Practice files live in `g2_lists_and_strings/`; each carries the prelude snippets it needs inline. Attempt every problem there before reading on – the doctests are the referee.*

14.1 greet: Greeting string

Contract:

```
greet(name: str) -> 'Hello, <name>!'
```

Hint:

```
unwords joins words with single spaces; the bang concatenates.  
Trivial on purpose -- the warmup proves the pipeline.
```

Doctests:

```
>>> greet("Ada")  
'Hello, Ada!'  
>>> greet("world")  
'Hello, world!'
```

Solution.

```
greet = lambda name: unwords(["Hello,", name]) + "!"
```

14.2 count_vowels: Count vowels, case-insensitive

Contract:

```
count_vowels(s: str) -> int
```

Hint:

```
len . filter_ (in "aeiou") . lower -- a filter consumed by len is  
the counting idiom.
```

Doctests:

```
>>> count_vowels("Programming")
3
>>> count_vowels("AEIOU")
5
>>> count_vowels("xyz")
0
>>> count_vowels("")
0
```

Solution.

```
count_vowels = lambda s: len(filter_(lambda c: c in "aeiou", s.lower()))
```

14.3 fizzbuzz: FizzBuzz as data

For 1..n: multiples of 3 -> "Fizz", of 5 -> "Buzz", of both -> "FizzBuzz", else the number as a string.

Contract:

```
fizzbuzz(n: int) -> list[str]
```

Hint:

Solve it BOTH ways and compare -- the contrast is the lesson:

1. COMPOSITION OF ARMS: map_ one per-number rule over range(1, n+1), building the word by concatenating guards -- "Fizz" * (i % 3 == 0) + "Buzz" * (i % 5 == 0) -- with `or str(i)` catching the empty case. Overlaps compose: 15 earns "FizzBuzz" from two rules for free.
2. FIRST-MATCH SELECTION: a RULES table of (predicate, result) pairs, find picking the first hit; const(otherwise) is the catch-all arm. Overlaps need their own row (15 goes first), but the ladder is now DATA -- appendable, testable rule by rule. Which generalizes better depends on whether cases overlap.

Doctests:

```
>>> fizzbuzz(5)
['1', '2', 'Fizz', '4', 'Buzz']
>>> fizzbuzz(15)[-1]
'FizzBuzz'
>>> fizzbuzz(0)
[]
```

Solution.

```
# way 1 -- composition of arms: overlapping guards concatenate
fizzbuzz = lambda n: map_(
    lambda i: ("Fizz" * (i % 3 == 0) + "Buzz" * (i % 5 == 0)) or str(i),
    range(1, n + 1))

# way 2 -- first-match selection: the ladder as a RULES table (this one wins)
```

```

RULES = [(lambda i: i % 15 == 0, const("FizzBuzz")),
         (lambda i: i % 3 == 0,  const("Fizz")),
         (lambda i: i % 5 == 0,  const("Buzz")),
         (const(otherwise),      str)]
speak = lambda i: snd(find(lambda rule: fst(rule)(i), RULES))(i)
fizzbuzz = lambda n: map_(speak, range(1, n + 1))

```

14.4 word_lengths: Map each word to its length

```

Contract:
  word_lengths(sentence: str) -> dict[str, int]
  First-appearance key order; repeats collapse harmlessly.

Hint:
  words splits; zip_ the words against map_ len of them; dict() the
  pairs. Duplicate keys overwrite with the same value -- free dedup.

```

Doctests:

```

>>> word_lengths("the quick fox")
{'the': 3, 'quick': 5, 'fox': 3}
>>> word_lengths("go go go")
{'go': 2}
>>> word_lengths("")
{}

```

Solution.

```

word_lengths = lambda s: dict(zip_(words(s), map_(len, words(s))))

```

14.5 flatten: Flatten one level of nesting

```

Contract:
  flatten(lists: list[list]) -> list

Hint:
  This IS the prelude's concat. One name, zero work.

```

Doctests:

```

>>> flatten([[1, 2], [3], [4, 5]])
[1, 2, 3, 4, 5]
>>> flatten([], [1], [])
[1]
>>> flatten([])
[]

```

Solution.

```

flatten = concat

```

14.6 caesar: Caesar cipher, letters only, case preserved

Contract:
 caesar(s: str, k: int) -> str

Hint:
 map_ a per-char shift over the string; the wrap is mod 26 on
 ord() offsets; join the pieces. One helper, one map.

Doctests:

```
>>> caesar("abc", 2)
'cde'
>>> caesar("XYZ", 3)
'ABC'
>>> caesar("a-b", 1)
'b-c'
>>> caesar("Hello, World!", 13)
'Uryyb, Jbeyq!'
>>> caesar("abc", 0)
'abc'
```

Solution.

```
def caesar(s, k):
    def shift(c):
        if not c.isalpha():
            return c
        base = ord("A") if c.isupper() else ord("a")
        return chr(base + (ord(c) - base + k) % 26)
    return "".join(map_(shift, s))
```

14.7 contains_duplicate: NeetCode: Contains Duplicate

Contract:
 contains_duplicate(nums: list[int]) -> bool
 True iff any value appears at least twice.

Hint:
 nub keeps first occurrences; duplicates exist iff nub shrinks the
 list.

Doctests:

```
>>> contains_duplicate([1, 2, 3, 1])
True
>>> contains_duplicate([1, 2, 3, 4])
False
>>> contains_duplicate([])
False
```

Solution.

```
contains_duplicate = lambda nums: len(nub(nums)) != len(nums)
```

14.8 chunks: Split a list into pieces of length k

Last piece may be shorter.

Contract:

```
chunks(xs: list, k: int) -> list[list]
```

Hint:

```
splitAt already returns (front, rest) == (value, next_seed), so
chunks IS unfoldr of splitAt; null(rest) is the stop. One line.
(You are deriving the prelude's chunksOf -- deriving a tool once
is why it sticks.)
```

Doctests:

```
>>> chunks([1, 2, 3, 4, 5], 2)
[[1, 2], [3, 4], [5]]
>>> chunks([1, 2, 3, 4], 2)
[[1, 2], [3, 4]]
>>> chunks([], 3)
[]
>>> chunks([1], 5)
[[1]]
```

Solution.

```
chunks = lambda xs, k: unfoldr(
    lambda rest: None if null(rest) else splitAt(k, rest), list(xs))
```

Remark. `splitAt` already returns `(front, rest)` – which is precisely the `(value, next_seed)` contract of `unfoldr`. The whole solution is recognising that two signatures line up. You are deriving the prelude's own `chunksOf`; deriving a tool once is why it sticks.

14.9 permutations: All distinct orderings, sorted

Contract:

```
permutations(s: str) -> list[str]
```

Hint:

```
concatMap over the choice of first character: pick s[i], prepend
it to every permutation of the rest. nub kills duplicates from
repeated letters; sort at the end.
```

Doctests:

```
>>> permutations("abc")
['abc', 'acb', 'bac', 'bca', 'cab', 'cba']
>>> permutations("aab")
['aab', 'aba', 'baa']
>>> permutations("x")
['x']
>>> permutations("")
['']
```

Solution.

```
def permutations(s):
    if len(s) <= 1:
        return [s]
    opts = concatMap(
        lambda i: [s[i] + rest for rest in permutations(s[:i] + s[i + 1:])],
        range(len(s)))
    return sorted(nub(opts))
```

Remark. `concatMap` is the shape of “for every choice, recurse, and pool the results.” Any backtracking enumeration can be written this way; `nub` then absorbs duplicate letters without a special case.

14.10 reverse_words: Reverse the word order

Runs of whitespace collapse (that's what a bare `split` does).

Contract:

```
reverse_words(s: str) -> str
```

Hint:

```
An unwords . reverse . words sandwich -- three names, no loop.
```

Doctests:

```
>>> reverse_words("the quick brown fox")
'fox brown quick the'
>>> reverse_words(" hello world ")
'world hello'
>>> reverse_words("solo")
'solo'
```

Solution.

```
reverse_words = lambda s: unwords(list(reversed(words(s))))
```

14.11 transpose_matrix: Flip rows and columns

Contract:

```
transpose_matrix(rows: list[list]) -> list[list]
```

Hint:

```
This IS the prelude's transpose -- like flatten was concat.
One name, zero work; know WHY zip(*rows) does it.
```

Doctests:

```
>>> transpose_matrix([[1, 2, 3], [4, 5, 6]])
[[1, 4], [2, 5], [3, 6]]
>>> transpose_matrix([[5]])
[[5]]
>>> transpose_matrix([[7], [8], [9]])
[[7, 8, 9]]
```

Solution.

```
transpose_matrix = transpose
```

14.12 rotate_right: Rotate a list right by k

k may be 0 or larger than the list.

Contract:

```
rotate_right(xs: list, k: int) -> list
```

Hint:

One splitAt at len - k%len, then glue the halves swapped.

The k=0 trap: xs[-0:] thinking sneaks in if you slice by -k.

Doctests:

```
>>> rotate_right([1, 2, 3, 4, 5], 2)
[4, 5, 1, 2, 3]
>>> rotate_right([1, 2], 0)
[1, 2]
>>> rotate_right([1, 2, 3], 5)
[2, 3, 1]
>>> rotate_right([], 3)
[]
```

Solution.

```
def rotate_right(xs, k):
    if not xs:
        return []
    a, b = splitAt(len(xs) - k % len(xs), list(xs))
    return b + a
```


Chapter 15

Class III — Folds, Scans and Streams

The heart of functional problem solving. A fold collapses a list to one value; a scan is a fold that keeps its history; a stream generates lazily until `take` says stop. The famous surprises live here: Kadane’s algorithm is a `scanl1`, Fibonacci is `iterate` on a pair, prefix sums turn every window-sum question into two slices and a `zipWith`. When a statement says “running”, “so far” or “at each step”, you are being told the answer is a scan. *Practice files live in g3_folds_scans_streams/; each carries the prelude snippets it needs inline. Attempt every problem there before reading on – the doctests are the referee.*

15.1 two_sum: NeetCode: Two Sum

Contract:

```
two_sum(nums: list[int], target: int) -> tuple[int, int]
Return indices (i, j), i < j, with nums[i] + nums[j] == target.
Exactly one solution exists. len(nums) >= 2.
```

Hint:

```
A fold over enum(nums) carrying a {value: index} dict.
The future needs to know: which values have I seen, and where?
```

Doctests:

```
>>> two_sum([2, 7, 11, 15], 9)
(0, 1)
>>> two_sum([3, 2, 4], 6)
(1, 2)
>>> two_sum([3, 3], 6)
(0, 1)
```

Solution.

```
def two_sum(nums, target):
    seen = {}
    for i, x in enum(nums):
        if target - x in seen:
            return (seen[target - x], i)
        seen[x] = i
```

15.2 best_time_stock: NeetCode: Best Time to Buy and Sell Stock

Contract:

```
max_profit(prices: list[int]) -> int
One buy, one later sell. Best achievable profit, 0 if none.
len(prices) >= 1.
```

Hint:

```
scanl1 min gives the running minimum -- the best buy so far.
zipWith (-) prices against it, then maximum. A scan, not a loop.
```

Doctests:

```
>>> max_profit([7, 1, 5, 3, 6, 4])
5
>>> max_profit([7, 6, 4, 3, 1])
0
>>> max_profit([5])
0
```

Solution.

```
def max_profit(prices):
    mins = scanl1(min, prices)
    return max(zipWith(lambda p, m: p - m, prices, mins))
```

15.3 maximum_subarray: NeetCode: Maximum Subarray (Kadane)

Contract:

```
max_subarray(nums: list[int]) -> int
Largest sum of a non-empty contiguous subarray. len(nums) >= 1.
```

Hint:

```
Kadane IS scanl1: best-ending-here = max(x, best_prev + x).
The scan is the DP table; maximum reads the answer off it.
```

Doctests:

```
>>> max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4])
6
>>> max_subarray([1])
1
>>> max_subarray([5, 4, -1, 7, 8])
23
>>> max_subarray([-3, -1, -2])
-1
```

Solution.

```
def max_subarray(nums):
    return max(scanl1(lambda best, x: max(x, best + x), nums))
```

Remark. Kadane’s algorithm is famous as a clever trick; seen functionally it is one `scanl1`. The scan is the DP table (best sum ending at each index), and `max` reads the answer off it. Many celebrated one-pass algorithms are scans wearing a trench coat.

15.4 `product_except_self`: NeetCode: Product of Array Except Self

Contract:

```
product_except_self(nums: list[int]) -> list[int]
out[i] == product of all nums except nums[i]. No division. O(n).
len(nums) >= 2.
```

Hint:

```
Prefix products = scanl (*) 1, suffix products = scanr (*) 1.
out = zipWith (*) prefixes suffixes -- offset by one on each side.
```

Doctests:

```
>>> product_except_self([1, 2, 3, 4])
[24, 12, 8, 6]
>>> product_except_self([-1, 1, 0, -3, 3])
[0, 0, 9, 0, 0]
```

Solution.

```
def product_except_self(nums):
    mul = lambda a, b: a * b
    pre = scanl(mul, 1, nums)
    suf = scanr(mul, 1, nums)
    return zipWith(mul, pre[:-1], suf[1:])
```

Remark. One scan from the left, one from the right, and a `zipWith` to marry them. The offset slicing (`pre[:-1]`, `suf[1:]`) is where the “except self” lives – each position sees the product of everything strictly before and strictly after it.

15.5 `moving_average`: Average of every k-window

Contract:

```
moving_average(xs: list, k: int) -> list[float]
len(xs) >= k >= 1.
```

Hint:

```
Prefix sums = scanl (+) 0 -- one scan, then every window sum is
pref[i+k] - pref[i]: zipWith (-) the two offset views of the same
scan, divide by k.
```

Doctests:

```
>>> moving_average([1, 2, 3, 4], 2)
[1.5, 2.5, 3.5]
>>> moving_average([1, 1, 1], 3)
[1.0]
```

```
>>> moving_average([5], 1)
[5.0]
```

Solution.

```
def moving_average(xs, k):
    pref = scanl(add, 0, xs)
    return zipWith(lambda hi, lo: (hi - lo) / k, pref[k:], pref[:-k])
```

15.6 pascal: First n rows of Pascal's triangle

```
Contract:
    pascal(n: int) -> list[list[int]]

Hint:
    The triangle is a stream: iterate the row step from [1], where
    next = zipWith (+) (0:row) (row:0) -- the row against itself,
    shifted. take n consumes the stream.
```

Doctests:

```
>>> pascal(4)
[[1], [1, 1], [1, 2, 1], [1, 3, 3, 1]]
>>> pascal(1)
[[1]]
>>> pascal(0)
[]
>>> pascal(6)[-1]
[1, 5, 10, 10, 5, 1]
```

Solution.

```
def pascal(n):
    step = lambda row: zipWith(add, [0] + row, row + [0])
    return take(n, iterate(step, [1]))
```

15.7 climbing_stairs: NeetCode: Climbing Stairs

```
Contract:
    climb_stairs(n: int) -> int
    Ways to reach step n taking 1 or 2 steps. n >= 1.

Hint:
    Fibonacci in disguise. iterate the pair step, take the nth --
    an unfold consumed by indexing, no table needed.
```

Doctests:

```
>>> climb_stairs(1)
1
>>> climb_stairs(2)
2
```

```
>>> climb_stairs(5)
8
>>> climb_stairs(45)
1836311903
```

Solution.

```
def climb_stairs(n):
    step = lambda p: (snd(p), fst(p) + snd(p))
    return fst(last(take(n + 1, iterate(step, (1, 1)))))
```

15.8 primes_up_to: All primes $\leq n$, ascending

```
Contract:
    primes_up_to(n: int) -> list[int]

Hint:
    Streams, Haskell style: candidates = takeWhile (<= n) count(2);
    keep p when no divisor in takeWhile (<= isqrt p) count(2) divides
    it -- all over the remainders. (The array sieve is faster; this
    version is about reading streams.)
```

Doctests:

```
>>> primes_up_to(20)
[2, 3, 5, 7, 11, 13, 17, 19]
>>> primes_up_to(2)
[2]
>>> primes_up_to(1)
[]
>>> len(primes_up_to(100))
25
```

Solution.

```
def primes_up_to(n):
    cans = takeWhile(lambda x: x <= n, count(2))
    is_prime = lambda p: all(
        p % d for d in takeWhile(lambda d: d <= isqrt(p), count(2)))
    return filter_(is_prime, cans)
```

15.9 totals_and_deltas: A scan and its inverse

```
Contract:
    running_totals(xs: list) -> list    # cumulative sums
    deltas(xs: list) -> list           # each element minus the previous
```

```
Hint:
    running_totals IS scanl1 (+). deltas is the pairwise differences.
    They undo each other: prepend the first element to
    deltas(running_totals(xs)) and xs comes back -- say why.
```

Doctests:

```
>>> running_totals([3, 1, 4])
[3, 4, 8]
>>> running_totals([])
[]
>>> deltas([5, 3, 8])
[-2, 5]
>>> deltas([7])
[]
>>> ts = running_totals([2, 5, 1]); [2] + deltas(ts) == [2, 5, 1]
True
```

Solution.

```
running_totals = lambda xs: scanl1(add, xs)
deltas = lambda xs: map_(lambda p: p[1] - p[0], pairwise(xs))
```

Remark. `scanl1 (+)` and `pairwise` differences are inverses – discrete integration and differentiation. Recognising an operation’s inverse is a testing superpower: the round-trip doctest here is a property test in miniature.

15.10 paren_depth: Deepest paren nesting

Input is balanced, only '(' and ')'.
 Contract:

```
max_depth(s: str) -> int
```

Hint:

Depth is a running total: map each paren to +1/-1, `scanl (+) 0` is the entire depth trace, maximum reads off the answer. No stack needed -- the scan IS the stack height's history.

Doctests:

```
>>> max_depth("((()()))")
3
>>> max_depth("()()")
1
>>> max_depth("")
0
```

Solution.

```
def max_depth(s):
    steps = map_(lambda c: 1 if c == "(" else -1, s)
    return max(scanl(add, 0, steps))
```

Chapter 16

Class IV — Sorting and Searching

“Does order unlock it?” is the first question of the recognition ladder for a reason: sorting linearises a problem so that greedy passes, binary searches and pairwise arguments become available. The exchange argument (`photo_lineup`), the tie-break trick (`russian_doll`) and patience sorting (`lis`) are all consequences of choosing the *right* sort key – which is where the thinking happens. `sensor_pairing` closes the class with the interview’s own domain problem. *Practice files live in `g4_sorting_and_searching/`; each carries the prelude snippets it needs inline. Attempt every problem there before reading on – the doctests are the referee.*

16.1 `is_anagram_sorted`: Anagrams via canonical form

```
True when a and b use exactly the same letters, ignoring case and
non-letters. (g5 solves the same problem with bags -- do both, argue
which you'd ship.)
```

Contract:

```
is_anagram(a: str, b: str) -> bool
```

Hint:

```
A canonical form makes equality trivial: sort of the cleaned,
lowered characters. filter_str.isalpha does the cleaning.
```

Doctests:

```
>>> is_anagram("Listen", "Silent")
True
>>> is_anagram("hello", "world")
False
>>> is_anagram("a gentleman", "elegant man")
True
>>> is_anagram("ab", "abb")
False
```

Solution.

```
canon = lambda s: sorted(filter_(str.isalpha, s.lower()))
is_anagram = lambda a, b: canon(a) == canon(b)
```

16.2 second_largest: Second largest distinct value

At least two distinct values guaranteed.

Contract:

second_largest(xs: list) -> value

Hint:

nub then sort; the answer is one from the end.

Doctests:

```
>>> second_largest([1, 5, 3])
3
>>> second_largest([5, 1, 5, 3])
3
>>> second_largest([2, 1])
1
>>> second_largest([-1, -5, -3])
-3
```

Solution.

```
second_largest = lambda xs: sorted(nub(xs))[-2]
```

16.3 binary_search: Index of target in a sorted list, else -1

Contract:

binary_search(xs: list, target) -> int

Hint:

bisect_left finds the leftmost slot where the target COULD be;
one bounds-check plus one equality test finishes the job. Know
why bisect_left(xs, x) == len(xs) is possible.

Doctests:

```
>>> binary_search([1, 3, 5, 7], 5)
2
>>> binary_search([1, 3, 5, 7], 1)
0
>>> binary_search([1, 3, 5, 7], 7)
3
>>> binary_search([1, 3], 2)
-1
>>> binary_search([], 1)
-1
```

Solution.

```
def binary_search(xs, x):
    i = bisect_left(xs, x)
    return i if i < len(xs) and xs[i] == x else -1
```

16.4 merge_intervals: Merge overlapping (and touching) intervals

Intervals are inclusive [s, e]: [1,2] and [2,3] merge to [1,3].

Contract:

```
merge_intervals(intervals: list[list[int]]) -> list[list[int]]
Result sorted by start.
```

Hint:

```
sortOn fst, then one fold: each interval either extends the
accumulator's last entry or starts a new one.
```

Doctests:

```
>>> merge_intervals([[1, 3], [2, 6], [8, 10], [15, 18]])
[[1, 6], [8, 10], [15, 18]]
>>> merge_intervals([[1, 2], [2, 3]])
[[1, 3]]
>>> merge_intervals([[5, 7], [1, 2], [3, 6]])
[[1, 2], [3, 7]]
>>> merge_intervals([[4, 4]])
[[4, 4]]
```

Solution.

```
def merge_intervals(intervals):
    def step(acc, iv):
        s, e = iv
        if acc and s <= acc[-1][1]:
            acc[-1][1] = max(acc[-1][1], e)
        else:
            acc.append([s, e])
        return acc
    return foldl(step, sortOn(fst, [list(iv) for iv in intervals]), [])
```

16.5 photo_lineup: The reported Anduril prompt, prelude-sized

Back row and front row, same length; can the columns be arranged so every back person is STRICTLY taller than the front person?

Contract:

```
can_lineup(back: list[int], front: list[int]) -> bool
```

Hint:

```
Sort both rows; all over zipWith (>). Be ready to argue why
sorted-vs-sorted decides it (the exchange argument).
```

Doctests:

```
>>> can_lineup([5, 1, 3], [2, 4, 0])
True
>>> can_lineup([1, 2, 3], [1, 2, 3])
False
```

```
>>> can_lineup([10, 9], [1, 2])
True
>>> can_lineup([2], [3])
False
```

Solution.

```
can_lineup = lambda back, front: all(
    zipWith(lambda b, f: b > f, sorted(back), sorted(front)))
```

16.6 merge_sort: A real $O(n \log n)$ sort, by hand

No `sorted()`, no `.sorted()`.

Contract:

```
merge_sorted(xs: list) -> list (new list; stable)
```

Hint:

`splitAt` at the midpoint, recurse on both halves, and the prelude's `merge` does the two-pointer join. Also retype `merge` itself from memory once -- it's the loop interviews ask for.

Doctests:

```
>>> merge_sorted([5, 2, 8, 1])
[1, 2, 5, 8]
>>> merge_sorted([])
[]
>>> merge_sorted([3, 3, 1])
[1, 3, 3]
>>> merge_sorted([1])
[1]
```

Solution.

```
def merge_sorted(xs):
    if len(xs) <= 1:
        return list(xs)
    a, b = splitAt(len(xs) // 2, list(xs))
    return merge(merge_sorted(a), merge_sorted(b))
```

16.7 lis: Longest strictly increasing subsequence (length)

Contract:

```
lis(xs: list[int]) -> int
```

Hint:

Patience sorting: fold the sequence into ``tails``, where `tails[i]` is the smallest possible tail of an increasing run of length `i+1`. `bisect_left` says which pile each element lands on; a landing past the end grows the answer. $O(n \log n)$ where the obvious DP is

```
|    O(n^2) -- say both.
```

Doctests:

```
>>> lis([10, 9, 2, 5, 3, 7, 101, 18])
4
>>> lis([5, 4, 3])
1
>>> lis([1, 2, 3])
3
>>> lis([])
0
>>> lis([7, 7, 7])
1
```

Solution.

```
def lis(xs):
    def place(tails, x):
        i = bisect_left(tails, x)
        return tails + [x] if i == len(tails) else tails[:i] + [x] + tails[i + 1:]
    return len(foldl(place, xs, []))
```

Remark. Patience sorting: `tails[i]` is the smallest possible tail of an increasing run of length $i+1$; `bisect_left` says which pile each element lands on. Landing past the end grows the answer. Know the $O(n^2)$ DP too, and say both complexities out loud.

16.8 russian_doll: LC 354: Russian Doll Envelopes

```
| Envelope (w, h) fits inside another only when BOTH are strictly
| smaller. No rotation. Longest nesting chain.
```

Contract:

```
| max_dolls(envelopes: list[tuple[int, int]]) -> int
```

Hint:

```
| Sort by (width asc, height DESC), then plain LIS on the heights
| -- g4_07 verbatim. The descending tie-break is the whole trick:
| equal-width envelopes arrive tallest-first, so no two of them can
| ever chain in a strictly-increasing height run. Sorting
| linearized one dimension; LIS handles the other. O(n log n).
```

Doctests:

```
>>> max_dolls([(5, 4), (6, 4), (6, 7), (2, 3)])
3
>>> max_dolls([(1, 1), (1, 1), (1, 1)])
1
>>> max_dolls([(3, 4), (3, 5), (3, 6)])
1
>>> max_dolls([])
0
>>> max_dolls([(2, 3)])
1
```

Solution.

```
def max_dolls(envelopes):
    hs = [h for _, h in sortOn(lambda e: (e[0], -e[1]), envelopes)]
    def place(tails, x):
        i = bisect_left(tails, x)
        return tails + [x] if i == len(tails) else tails[:i] + [x] + tails[i + 1:]
    return len(foldl(place, hs, []))
```

Remark. Sorting by (width asc, height desc) is the whole problem. The descending tie-break guarantees equal-width envelopes can never chain, which collapses a 2-D dominance question into a 1-D LIS on heights. Sorting linearised one dimension; the tails trick handles the other.

16.9 pairs_within_budget: Count pairs whose sum fits a budget

Pairs are (i, j) with i < j; values may be negative; the count does not depend on input order -- that sentence licenses sorting.

Contract:

```
pairs_within_budget(t: int, xs: list[int]) -> int
```

Hint:

Sort first. Then each element asks: how many LATER elements keep the sum <= t? bisect_right(ys, t - x) answers in O(log n) -- subtract the i+1 elements at or before x itself, clip at 0.

Doctests:

```
>>> pairs_within_budget(5, [3, 1, 4, 2])
4
>>> pairs_within_budget(-1, [-2, -3, 5])
1
>>> pairs_within_budget(100, [1, 2, 3])
3
>>> pairs_within_budget(0, [1, 2])
0
```

Solution.

```
def pairs_within_budget(t, xs):
    ys = sorted(xs)
    return sum(max(0, bisect_right(ys, t - x) - (i + 1)) for i, x in enum(ys))
```

16.10 sensor_pairing: The screen's domain problem, function form

Two sensor feeds watch the same airspace. A detection is (id, t, bearing_degrees); ids are sortable strings. Detections a, b are COMPATIBLE when |a.t - b.t| <= t_tol and their bearing difference is <= b_tol -- and bearings wrap: 359.5 and 0.3 are 0.8 apart.

Contract:

```
pair_detections(a, b, t_tol, b_tol)
```

```

-> (pairs, unpaired_a_ids, unpaired_b_ids)
Greedy: repeatedly take the lowest-cost compatible pair of still-
unused detections, cost = |dt| + wrapped bearing diff (exact ties
by ids). pairs as (a_id, b_id); all three outputs sorted.

```

Hint:

```

Build the compatible candidates as one comprehension over both
feeds (cost, a_id, b_id, i, j), sort -- the fold that follows
takes each candidate whose indices are still unused. The wrap is
min(d, 360 - d) in ONE helper. This is mock_phased.py distilled.

```

Doctests:

```

>>> a = [('A0', 10.0, 90.0), ('A1', 20.0, 180.0)]
>>> b = [('B0', 10.2, 91.0), ('B1', 20.1, 179.5)]
>>> pair_detections(a, b, 0.5, 2.0)
([('A0', 'B0'), ('A1', 'B1')], [], [])
>>> pair_detections([('A0', 10.0, 359.5)], [('B0', 10.1, 0.3)], 0.5, 2.0)
([('A0', 'B0')], [], [])
>>> pair_detections([('A0', 0.0, 10.0)], [('B0', 900.0, 200.0)], 0.5, 2.0)
([], ['A0'], ['B0'])
>>> a = [('A0', 10.0, 45.0), ('A1', 30.0, 90.0)]
>>> b = [('B0', 10.5, 44.5), ('B1', 10.6, 46.0), ('B2', 50.0, 200.0)]
>>> pair_detections(a, b, 1.0, 2.0)
([('A0', 'B0')], ['A1'], ['B1', 'B2'])

```

Solution.

```

def pair_detections(a, b, t_tol, b_tol):
    wrapd = lambda x, y: min(abs(x - y) % 360, 360 - abs(x - y) % 360)
    cand = sorted([(abs(ta - tb) + wrapd(ba, bb), ai, bi, i, j)
                    for i, (ai, ta, ba) in enum(a)
                    for j, (bi, tb, bb) in enum(b)
                    if abs(ta - tb) <= t_tol and wrapd(ba, bb) <= b_tol])
    used_a, used_b, pairs = set(), set(), []
    for _cost, ai, bi, i, j in cand:
        if i not in used_a and j not in used_b:
            used_a.add(i)
            used_b.add(j)
            pairs.append((ai, bi))
    ua = [d[0] for i, d in enum(a) if i not in used_a]
    ub = [d[0] for j, d in enum(b) if j not in used_b]
    return sorted(pairs), sorted(ua), sorted(ub)

```

Remark. The interview capstone, distilled: build every compatible candidate, sort by cost, greedily take what is still unused. The wrap-around rule lives in *one* helper, which is why phase mutations cost one line here. Name the better algorithm (optimal assignment, Hungarian, $O(n^3)$) and deliberately choose the simpler one – that sentence is a senior signal.

Chapter 17

Class V — Bags, Dicts and Grouping

Aggregating by key. `Counter` counts, `fromListWith` is the general dict-building fold, `unionWith` merges two dicts under any combining function – and choosing that function is choosing a monoid (`max` gives bag union, `+` gives a summing merge). The class also carries the Maybe pipeline (`first_unique`, `log_report`): parse to value-or-None, then let the Maybe tools do the bookkeeping. *Practice files live in `g5_bags_and_grouping/`; each carries the prelude snippets it needs inline.*

Attempt every problem there before reading on – the doctests are the referee.

17.1 `valid_anagram`: NeetCode: Valid Anagram

Contract:

```
is_anagram(s: str, t: str) -> bool
True iff t is a permutation of s. Lowercase ASCII.
```

Hint:

```
Two Counter folds; anagram == equal multisets. (Compare with
g4's sorted-canonical version: same problem, different algebra.)
```

Doctests:

```
>>> is_anagram("anagram", "nagaram")
True
>>> is_anagram("rat", "car")
False
>>> is_anagram("", "")
True
```

Solution.

```
is_anagram = lambda s, t: Counter(s) == Counter(t)
```

17.2 `ransom_note`: HackerRank: Ransom Note

Contract:

```
can_make(note: list[str], magazine: list[str]) -> bool
True iff every word in note is available in magazine with at
```

least the required multiplicity. Case-sensitive whole words.

Hint:

Multiset inclusion: `Counter(note) <= Counter(magazine)`, spelled as an ``all`` over the note's counts.

Doctests:

```
>>> can_make("give me one grand today night".split(), "give me one grand
↪ today".split())
False
>>> can_make("two times three is not four".split(), "two times three is not
↪ four".split())
True
>>> can_make(["give"], ["Give"])
False
```

Solution.

```
def can_make(note, magazine):
    have = Counter(magazine)
    need = Counter(note)
    return all(have[w] >= n for w, n in need.items())
```

17.3 `most_frequent`: Most frequent value; ties -> smallest

Contract:

`most_frequent(xs: list) -> value`

Hint:

`Counter`, then `minOn` the tuple key `(-count, value)`. The two-level tie-break lives entirely in the key, and `minOn` is $O(n)$ where `sortOn` + `head` pays $O(n \log n)$ for the same answer.

Doctests:

```
>>> most_frequent([1, 2, 2, 3, 3])
2
>>> most_frequent([5, 5, 1])
5
>>> most_frequent([3, 1, 2])
1
>>> most_frequent(["b", "a", "b", "a"])
'a'
```

Solution.

```
most_frequent = lambda xs: minOn(
    lambda kv: (-kv[1], kv[0]), list(Counter(xs).items()))[0]
```

17.4 `top_k_words`: The k most frequent words

Ordered by count descending, ties by word ascending. At least k distinct words guaranteed.

Contract:

```
top_k_words(k: int, text: str) -> list[tuple[str, int]]
```

Hint:

Counter over the words, sortOn (-count, word), take k. The whole pipeline is three prelude names deep.

Doctests:

```
>>> top_k_words(2, "the cat the dog the cat bird")
[('the', 3), ('cat', 2)]
>>> top_k_words(3, "b a a b c c")
[('a', 2), ('b', 2), ('c', 2)]
```

Solution.

```
top_k_words = lambda k, text: take(
    k, sortOn(lambda kv: (-kv[1], kv[0]), list(Counter(text.split()).items())))
```

17.5 group_words: Group words by first letter

First-appearance order for both keys and members.

Contract:

```
group_words(words: list[str]) -> dict[str, list[str]]
```

Hint:

THE fromListWith showcase: pairs (first letter, [word]). The combiner receives f(new, old) -- Haskell's order -- so KEEP arrival order by combining old + new; a bare (+) would reverse each group (the classic Haskell gotcha). Keys keep first-seen order.

Doctests:

```
>>> group_words(["ant", "bee", "ape"])
{'a': ['ant', 'ape'], 'b': ['bee']}
>>> group_words(["cat"])
{'c': ['cat']}
>>> group_words([])
{}
```

Solution.

```
group_words = lambda ws: fromListWith(
    lambda new, old: old + new, [(w[0], [w]) for w in ws])
```

17.6 group_anagrams: NeetCode: Group Anagrams

Contract:

```
group_anagrams(words: list[str]) -> list[list[str]]
Partition words into anagram classes; classes in first-seen order,
members in input order.
```

Hint:

Key = canonical form (sorted letters). Two prelude routes, do both: `Tree(1, list)` with a bare `append` (autovivification aids the write-heavy access), or one `fromListWith` over `(key, [w])` pairs -- combining old + new, since `f` receives `(new, old)`; a bare `(+)` would reverse each group. Compare.

Doctests:

```
>>> group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"])
[['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
>>> group_anagrams("")
[['']]
>>> group_anagrams("a")
[['a']]
```

Solution.

```
def group_anagrams(words):
    return list(fromListWith(lambda new, old: old + new,
                             ["".join(sorted(w)), [w]] for w in words)).values()
```

Remark. Two respectable routes: `Tree(1, list)` with a bare `append` (autovivification suits write-heavy access) or one `fromListWith` over `(key, [w])` pairs, combining old + new (the combiner hears the new value first). Solve it both ways once; afterwards you will see every grouping problem as a one-liner.

17.7 merge_sum: Merge two dicts of numbers, summing shared keys

Don't mutate the inputs. Key order: a's keys, then b's new ones.

Contract:

```
merge_sum(a: dict, b: dict) -> dict
```

Hint:

This IS `unionWith` with `(+)`: merge, combining shared keys by addition. The prelude's `bag_union` is the SAME merge with `max` -- one combinator, different monoid. Spot the difference and say it. (`fromListWith` over the two `items()` lists concatenated gets there too; decide which reads better.)

Doctests:

```
>>> merge_sum({'a': 1, 'b': 2}, {'b': 3, 'c': 4})
{'a': 1, 'b': 5, 'c': 4}
>>> merge_sum({}, {'x': 1})
{'x': 1}
```

```
>>> merge_sum({'x': 1}, {})
{'x': 1}
```

Solution.

```
merge_sum = lambda a, b: unionWith(lambda x, y: x + y, a, b)
```

17.8 rle_encode: Run-length encode

```
"aaabbc" -> "a3b2c1"; every run gets a count, even runs of one.
```

Contract:

```
rle_encode(s: str) -> str
```

Hint:

```
The prelude's group gathers runs of CONSECUTIVE equals as bare
lists. map_ each run to run[0] + str(len(run)), join.
```

Doctests:

```
>>> rle_encode("aaabbc")
'a3b2c1'
>>> rle_encode("abc")
'a1b1c1'
>>> rle_encode("aaaa")
'a4'
>>> rle_encode("")
''
```

Solution.

```
rle_encode = lambda s: "".join(
    map_(lambda run: run[0] + str(len(run)), group(s)))
```

17.9 lru_class: An LRU cache class, dict-only

Capacity at construction; put(key, value); get(key) -> value or None.
Both get and put count as use; puts beyond capacity evict the least
recently used entry.

Contract:

```
class LRU(capacity)
    .put(key, value) -> None
    .get(key) -> value | None
```

Hint:

```
The container lesson with no snippet to lean on: dicts remember
insertion order, so pop + re-insert IS move-to-most-recent, and
next(iter(d)) is the least recent. That's the whole machine.
```

Doctests:

```

>>> c = LRU(2)
>>> c.put('a', 1); c.put('b', 2)
>>> c.get('a')
1
>>> c.put('c', 3)
>>> c.get('b') is None
True
>>> c.get('c')
3
>>> c.get('a')
1
>>> c.put('a', 99); c.get('a')
99

```

Solution.

```

class LRU:
    def __init__(self, capacity):
        self.capacity = capacity
        self.data = {}
    def get(self, key):
        if key not in self.data:
            return None
        val = self.data.pop(key)
        self.data[key] = val
        return val
    def put(self, key, value):
        if key in self.data:
            self.data.pop(key)
        elif len(self.data) >= self.capacity:
            self.data.pop(next(iter(self.data)))
        self.data[key] = value

```

17.10 first_unique: First non-repeating character

Contract:

```
first_unique(s: str) -> str # the char, or '_' if none
```

Hint:

Three prelude names, one line: Counter the string, find the first char whose count is 1, fromMaybe the '_' default. Two passes, O(n) -- and find stops at the first hit.

Doctests:

```

>>> first_unique("swiss")
'w'
>>> first_unique("aabb")
'_'
>>> first_unique("")
'_'

```

Solution.

```
def first_unique(s):
    counts = Counter(s)
    return fromMaybe('_', find(lambda c: counts[c] == 1, s))
```

Remark. Three prelude names, one line: `Counter` builds the evidence, `find` stops at the first witness, `fromMaybe` supplies the default. This is what vocabulary buys: the solution is its own specification.

17.11 log_report: Error report from messy logs

A well-formed line is "<ts> <LEVEL> <service>: <message...>": LEVEL in {INFO, WARN, ERROR}, service nonempty and glued to its colon, message may be empty. Blank lines are ignored; any other non-blank line is malformed.

Contract:

```
error_report(loglines: list[str]) -> (report, malformed_count)
report = [(service, error_count)] for services with >= 1 ERROR,
sorted by count desc, then name asc.
```

Hint:

```
THE mapMaybe shape: parse(line) -> (level, service) or None.
The Nones ARE the malformed count; catMaybes + a filter feed
Counter; sortOn (-count, name) ranks. Messy input, no raw loops.
```

Doctests:

```
>>> logs = ["t1 ERROR api: down", "t2 INFO web: ok", "garbage line",
...         "t3 ERROR api: db", "t4 ERROR web: 500", "t5 WARN api: slow",
...         "t6 DEBUG api: no", "t7 ERROR db missing"]
>>> error_report(logs)
([('api', 2), ('web', 1)], 3)
>>> error_report(["t1 INFO web: ok", "half a line"])
([], 1)
>>> error_report([])
([], 0)
```

Solution.

```
LEVELS = {"INFO", "WARN", "ERROR"}

def parse(line):
    parts = line.split(maxsplit=3)
    if (len(parts) < 3 or parts[1] not in LEVELS
        or not parts[2].endswith(":") or parts[2] == ":"):
        return None
    return (parts[1], parts[2][:-1])

def error_report(loglines):
    parsed = map_(parse, filter_(lambda l: l.strip(), loglines))
    malformed = len(filter_(lambda p: p is None, parsed))
    errs = [svc for lv, svc in catMaybes(parsed) if lv == "ERROR"]
    report = sortOn(lambda kv: (-kv[1], kv[0]), list(Counter(errs).items()))
    return report, malformed
```

Remark. The messy-input shape: `parse` returns a value or `None`, and everything downstream is bookkeeping over Maybes – `catMaybes` for the survivors, a filter for the casualty count, `Counter` and `sortOn` for the report. No raw loops, no flags.

17.12 `invert__dict`: Swap a dict's keys and values

Values are unique (so the inversion is lossless).

Contract:

`invert(d: dict) -> dict`

Hint:

A dict is a list of pairs wearing a hat: `items()`, swap each pair, `dict()` the result. The swap showcase.

Doctests:

```
>>> invert({'a': 1, 'b': 2})
{1: 'a', 2: 'b'}
>>> invert({})
{}
```

Solution.

```
invert = lambda d: dict(map_(swap, list(d.items())))
```

Chapter 18

Class VI — Tries and Paths

Hierarchy as nested dicts. `ITree` autovivifies branches on touch; `setpath/getpath` write and read key paths; `paths` folds a whole tree back into a flat list of (keypath, leaf). The subset-sum pair is the deep exhibit: the same search recorded first as a decision tree (a multiset of solutions) and then as a trie (a set) – the data structure changed the semantics. *Practice files live in `g6_tries_and_paths/`; each carries the prelude snippets it needs inline. Attempt every problem there before reading on – the doctests are the referee.*

18.1 `deepest_directory`: Challenge: deepest path in a directory forest

Contract:

```
deepest(dirs: list[str]) -> str
Each string is a '/'-separated path ("a/b/c"). Return the deepest
full path (unique deepest guaranteed). Paths may share prefixes.
```

Hint:

```
Building IS walking: foldl indexing over the parts autovivifies the
whole branch -- no setpath needed since there is no leaf value.
Then one paths() + maxOn keypath length. Note why setpath would
be WRONG here: writing "a/b" after "a/b/c" would clobber the
subtree.
```

Doctests:

```
>>> deepest(["a/b/c", "a/d", "e"])
'a/b/c'
>>> deepest(["x"])
'x'
>>> deepest(["a/b", "a/b/c/d", "a/b/c"])
'a/b/c/d'
```

Solution.

```
def deepest(dirs):
    t = ITree()
    for d in dirs:
        foldl(lambda node, k: node[k], d.split('/'), t)
```

```
|     return '/'.join(maxOn(lambda pl: len(pl[0]), paths(t))[0])
```

Remark. Walking *is* building: folding indexing over the path parts autovivifies the whole branch. Note why `setpath` would be wrong – writing `a/b` after `a/b/c` would clobber the subtree. Choosing the writing primitive is part of the design.

18.2 autocomplete: Challenge: prefix trie with completion

Contract:

```
make_trie(words: list[str]) -> ITree
    Letter-by-letter trie; each word terminated by '$' whose leaf
    is the word itself.
complete(trie, prefix: str) -> list[str]
    All stored words starting with prefix, sorted. A word counts
    as its own completion.
```

Hint:

```
setpath writes each word; getpath descends to the prefix's subtree
WITHOUT autovivifying (reading an ITree with [] would plant ghost
branches); paths() harvests every '$' leaf below.
(No-trie baseline: filter_ + isPrefixOf over the word list, O(n*m)
per query -- the trie is what repeated queries buy.)
```

Doctests:

```
>>> make_trie(["at"]) == {'a': {'t': {'$': 'at'}}}
True
>>> t = make_trie(["cat", "car", "card", "dog", "do"])
>>> complete(t, "ca")
['car', 'card', 'cat']
>>> complete(t, "do")
['do', 'dog']
>>> complete(t, "z")
[]
>>> 'z' in t
False
```

Solution.

```
def make_trie(words):
    t = ITree()
    for w in words:
        setpath(t, list(w) + ['$'], w)
    return t

def complete(trie, prefix):
    sub = getpath(trie, list(prefix), default={})
    return sorted([leaf for p, leaf in paths(sub) if p and p[-1] == '$'])
```

18.3 subset_sum_paths: Challenge: all paths in the decision tree summing to a total

Contract:

```
build_tree(nums: list[int], total: int) -> tree
    The take/skip decision tree over nums. Node = dict with keys
    ('take', x) and ('skip', x); leaf = bool (True iff the takes
    along that path sum to total).
tree_paths(tree) -> list[list[int]]
    The taken values along every path ending in True, take-first
    DFS order.
sum_paths(nums: list[int], total: int) -> list[list[int]]
    Every subsequence of nums summing to total. Negatives allowed,
    so a path only ends at full depth -- rem == 0 mid-path proves
    nothing.
```

Hint:

An unfold builds the tree from seed (nums, remaining); a fold over the tree collects paths ending in True. unfold + fold = hylomorphism: fuse them and the tree never exists -- but here it is the exhibit. 2^n leaves: every subset gets judged, exactly once. (The prelude now also ships subsequences -- filtering the powerset by sum is the same 2^n with the tree left implicit.)

Doctests:

```
>>> build_tree([], 0)
True
>>> build_tree([5], 5) == {'take', 5): True, ('skip', 5): False}
True
>>> tree_paths(True)
[[]]
>>> tree_paths(False)
[]
>>> tree_paths(build_tree([1, 2], 3))
[[1, 2]]
>>> sum_paths([1, 2, 3], 3)
[[1, 2], [3]]
>>> sum_paths([2, 4, 6, 10], 16)
[[2, 4, 10], [6, 10]]
>>> sum_paths([1, -1, 2], 2)
[[1, -1, 2], [2]]
>>> sum_paths([], 0)
[[]]
>>> sum_paths([1, 1], 2)
[[1, 1]]
```

Solution.

```
def build_tree(nums, total):
    if null(nums):
        return total == 0
    x = head(nums)
    return {'take', x): build_tree(tail(nums), total - x),
```

```

        ('skip', x): build_tree(tail(nums), total)}

def tree_paths(tree):
    if tree is True:
        return [[]]
    if tree is False:
        return []
    return concat([[x] + p if tag == 'take' else p for p in tree_paths(sub)]
                  for (tag, x), sub in tree.items())

def sum_paths(nums, total):
    return tree_paths(build_tree(nums, total))

```

18.4 subset_sum_trie: Challenge: subset-sum, winners recorded in a solution trie

Contract:

```

solution_trie(nums: list[int], total: int) -> ITree
    Search the take/skip space; write only the WINNING paths (the
    taken values, terminated by '$') into an ITree.
sum_paths(nums: list[int], total: int) -> list[list[int]]
    The winning paths read back off the trie via paths().
NB: a trie stores a SET of solutions -- duplicates arising from
different index choices (e.g. [1, 1] with total 1) collapse to one.
That is a semantic change from 03's multiset.

```

Hint:

```

The full trio: unfold searches, ITree records, paths() reads.
Why the '$' terminator? One solution can be a prefix of another
([2] inside [2, 4, -4]) -- without an end marker the shorter one
vanishes into the longer one's interior.

```

Doctests:

```

>>> solution_trie([1, 2], 3) == {1: {2: {'$': True}}}
True
>>> sum_paths([1, 2, 3], 3)
[[1, 2], [3]]
>>> sum_paths([2, 4, -4], 2)
[[2, 4, -4], [2]]
>>> sum_paths([1, 1], 1)
[[1]]
>>> sum_paths([], 0)
[[]]

```

Solution.

```

def solution_trie(nums, total):
    t = ITree()
    def go(i, rem, picked):
        if i == len(nums):
            if rem == 0:

```

18.4. SUBSET_SUM_TRIE: CHALLENGE: SUBSET-SUM, WINNERS RECORDED IN A SOLUTION TRIE10

```

        setpath(t, picked + ['$'], True)
    return
    go(i + 1, rem - nums[i], picked + [nums[i]])
    go(i + 1, rem, picked)
go(0, total, [])
return t

def sum_paths(nums, total):
    return [p[:-1] for p, _ in paths(solution_trie(nums, total)) if p[-1] == '$']

```

Remark. The \$ terminator earns its place: one solution can be a prefix of another, and without an end marker the shorter one vanishes into the longer one's interior. Also note the semantic shift from the decision-tree version: a trie stores a *set* of solutions, so duplicate index-choices collapse.

Chapter 19

Class VII — Graphs, Grids, Windows and Heaps

Things pointing at other things. Adjacency is a `Tree(1, list)` fold over the edges; BFS is the `deque` earning its keep; Dijkstra adds a heap and lazy deletion; Kahn's topological sort turns cycle detection into a counting argument; Bellman-Ford is `until` iterating to a fixed point. The window problems ride along because their state discipline (add, remove, valid) is the same closure-shaped thinking on a 1-D track. *Practice files live in `g7_graphs_grids_heaps/`; each carries the prelude snippets it needs inline. Attempt every problem there before reading on – the doctests are the referee.*

19.1 `longest_unique`: Longest substring without repeats

Contract:
`longest_unique(s: str) -> int`

Hint:
The `longest_window` skeleton does the two-pointer bookkeeping; you supply the state as closures: per-char counts (`Tree(1, int)`) plus a duplicate tally. `valid() == "no duplicates in the window"`.

Doctests:

```
>>> longest_unique("abcabcbb")
3
>>> longest_unique("bbbbbb")
1
>>> longest_unique("")
0
>>> longest_unique("abba")
2
```

Solution.

```
def longest_unique(s):
    counts, dups = Tree(1, int), [0]
    def add(c):
        counts[c] += 1
        dups[0] += counts[c] == 2
```

```
def rem(c):
    counts[c] -= 1
    dups[0] -= counts[c] == 1
    return longest_window(s, lambda: dups[0] == 0, add, rem)
```

Remark. The window skeleton does the two-pointer bookkeeping; you supply state as closures. Writing `valid/add/shed` separately is exactly the decomposition that survives a mutating interview prompt.

19.2 `grid_path`: Shortest path through a grid

Steps from top-left to bottom-right through '.' cells, around '#' walls, moving orthogonally. -1 if unreachable (or an endpoint is a wall); a 1x1 open grid is 0.

Contract:

```
grid_path(grid: list[str]) -> int
```

Hint:

BFS: the prelude deque is the frontier, `neighbors4` the edges. Mark seen when you ENQUEUE, not when you pop.

Doctests:

```
>>> grid_path(["..", ".."])
2
>>> grid_path([".#", "#."])
-1
>>> grid_path(["."])
0
>>> grid_path(["..#", "#.#", "#.."])
4
```

Solution.

```
def grid_path(grid):
    R, C = len(grid), len(grid[0])
    if grid[0][0] == "#" or grid[R - 1][C - 1] == "#":
        return -1
    q, seen = deque([(0, 0, 0)]), {(0, 0)}
    while len(q):
        r, c, d = q.popleft()
        if (r, c) == (R - 1, C - 1):
            return d
        for nr, nc in neighbors4(r, c, R, C):
            if grid[nr][nc] == "." and (nr, nc) not in seen:
                seen.add((nr, nc))
                q.append((nr, nc, d + 1))
    return -1
```

19.3 `rate_limiter`: Sliding-window rate limiter

A request at time `ts` is denied when its client already has ``limit`` ALLOWED requests in `(ts - window, ts]`. Denied requests consume no budget. Timestamps arrive non-decreasing.

Contract:

```
rate_limiter(window: int, limit: int,
              events: list[tuple[int, str]]) -> list[tuple[int, str]]
The denied (ts, client) pairs, in stream order.
```

Hint:

`Tree(1, deque)`: each client's allowed timestamps in a deque. The prelude deque has no peek -- evict by `popleft`, and `appendleft` the survivor back. Each stamp enters and leaves once: $O(1)$ amortized.

Doctests:

```
>>> rate_limiter(10, 2, [(1, 'alice'), (2, 'alice'), (3, 'alice'),
...                      (5, 'bob'), (12, 'alice'), (13, 'alice')])
[(3, 'alice')]
>>> rate_limiter(5, 3, [(1, 'a'), (2, 'a'), (3, 'a')])
[]
>>> rate_limiter(5, 1, [(1, 'a'), (2, 'b'), (3, 'a'), (7, 'a'), (8, 'b')])
[(3, 'a')]
```

Solution.

```
def rate_limiter(window, limit, events):
    allowed = Tree(1, deque)
    denied = []
    for ts, client in events:
        q = allowed[client]
        while len(q):
            t = q.popleft()
            if t > ts - window:
                q.appendleft(t)
                break
        if len(q) >= limit:
            denied.append((ts, client))
        else:
            q.append(ts)
    return denied
```

19.4 running_median: Median after each arrival

Even counts average the two middle values; one decimal is the caller's problem -- return floats.

Contract:

```
running_median(xs: list[int]) -> list[float]
```

Hint:

Two heaps: `lo` holds the lower half NEGATED (the prelude heap is min-only), `hi` the upper half. Push through `lo` into `hi`, rebalance

so lo keeps the extra; the median reads off the tops.

Doctests:

```
>>> running_median([12, 4, 5, 3, 8, 7])
[12.0, 8.0, 5.0, 4.5, 5.0, 6.0]
>>> running_median([10])
[10.0]
>>> running_median([1, 2, 3, 4])
[1.0, 1.5, 2.0, 2.5]
```

Solution.

```
def running_median(xs):
    lo, hi, out = [], [], []
    for x in xs:
        heappush(lo, -x)
        heappush(hi, -heappop(lo))
        if len(hi) > len(lo):
            heappush(lo, -heappop(hi))
        m = -lo[0] if len(lo) > len(hi) else (-lo[0] + hi[0]) / 2
        out.append(float(m))
    return out
```

19.5 dijkstra: Challenge: shortest paths from weighted (from, to, dist) edges

Contract:

```
dijkstra(edges: list[tuple], src) -> (dist: dict, parent: dict)
    Directed edges (u, v, w), w >= 0. dist[n] = shortest distance
    src -> n; unreachable nodes absent. parent[n] = predecessor on
    one shortest path (parent[src] = None).
shortest_path(edges, src, dst) -> list | None
    The node sequence of a shortest src -> dst path, None if
    unreachable.
```

Hint:

Adjacency map = Tree(1, list) fold over edges. Frontier = the heap.
 Lazy deletion: skip a popped entry unless it matches the current
 best -- the heap may hold stale distances. Path reconstruction =
 an unfold: iterate the parent chain, takeWhile not None, reverse.
 Graphs are not trees -- cycles make tree unfolds diverge, so fold
 over STATES to a fixed point instead.

Doctests:

```
>>> dist, _ = dijkstra([('a','b',1), ('b','c',2), ('a','c',4)], 'a')
>>> dist == {'a': 0, 'b': 1, 'c': 3}
True
>>> dist, _ = dijkstra([('a','b',5), ('a','c',1), ('c','b',1)], 'a')
>>> dist['b']
2
>>> dijkstra([('x','y',1)], 'y')[0]
```

```
{'y': 0}
>>> shortest_path([('a','b',5), ('a','c',1), ('c','b',1)], 'a', 'b')
['a', 'c', 'b']
>>> shortest_path([('a','b',1)], 'a', 'a')
['a']
>>> shortest_path([('a','b',1)], 'b', 'a') is None
True
```

Solution.

```
INF = float("inf")

def dijkstra(edges, src):
    adj = Tree(1, list)
    for u, v, w in edges:
        adj[u].append((w, v))
    dist, parent, h = {src: 0}, {src: None}, [(0, src)]
    while h:
        d, u = heappop(h)
        if d == dist.get(u, INF):
            for w, v in adj[u]:
                if d + w < dist.get(v, INF):
                    dist[v] = d + w
                    parent[v] = u
                    heappush(h, (d + w, v))
    return dist, parent

def shortest_path(edges, src, dst):
    dist, parent = dijkstra(edges, src)
    if dst not in dist:
        return None
    back = takeWhile(lambda n: n is not None, iterate(lambda n: parent[n], dst))
    return list(reversed(back))
```

Remark. Lazy deletion is the grown-up move: the heap may hold stale distances, so skip a popped entry unless it matches the current best. Path reconstruction is an unfold along the parent chain. Graphs are not trees – cycles make tree unfolds diverge, so we fold over *states* to a fixed point.

19.6 dag_longest_path: Challenge: longest path in a weighted DAG

Contract:

```
dag_longest(edges: list[tuple]) -> (total: int, path: list)
Directed acyclic edges (u, v, w). The maximum-weight path anywhere
in the DAG and its node sequence. Ties broken toward the
lexically-first end node. Input guaranteed acyclic.
```

Hint:

```
Longest path is NP-hard on general graphs; acyclicity is what
makes it linear. Kahn's topo sort (indegree fold + deque), then a
fold over topo order relaxing best[v] = max(best[v], best[u] + w).
Same skeleton as Dijkstra's relax with max for min and topo order
```

replacing the heap -- the LCS-vs-edit-distance move on graphs.
 maxOn best picks the end node; feed it the SORTED nodes so
 ties break lexically.

Doctests:

```
>>> dag_longest([('a','b',3), ('b','c',4), ('a','c',5)])
(7, ['a', 'b', 'c'])
>>> dag_longest([('s','a',1), ('s','b',5), ('a','t',6), ('b','t',1)])
(7, ['s', 'a', 't'])
>>> dag_longest([('u','v',2)])
(2, ['u', 'v'])
```

Solution.

```
def dag_longest(edges):
    adj, indeg, nodes = Tree(1, list), Tree(1, int), set()
    for u, v, w in edges:
        adj[u].append((w, v))
        indeg[v] += 1
        nodes |= {u, v}
    q = deque([n for n in sorted(nodes) if indeg[n] == 0])
    topo = []
    while len(q):
        u = q.popleft()
        topo.append(u)
        for w, v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    best = {n: 0 for n in nodes}
    parent = {n: None for n in nodes}
    for u in topo:
        for w, v in adj[u]:
            if best[u] + w > best[v]:
                best[v] = best[u] + w
                parent[v] = u
    end = maxOn(lambda n: best[n], sorted(nodes))
    back = takeWhile(lambda n: n is not None, iterate(lambda n: parent[n], end))
    return best[end], list(reversed(back))
```

19.7 bellman_ford: Challenge: shortest paths as a fixed point (negative weights ok)

Contract:

bellman_ford(edges: list[tuple], src) -> dict | None
 Directed edges (u, v, w); w may be negative. dist[n] = shortest
 distance src -> n, unreachable nodes absent. Returns None if a
 negative cycle is reachable from src (distances then have no
 fixed point -- they diverge to -inf).

Hint:

Shortest paths ARE a fixed point: `dist` is the least map satisfying `dist[v] <= dist[u] + w` for every edge. So: relax ALL edges as one step function, `until` the map stops changing. Convergence within $|V| - 1$ steps is guaranteed (a shortest path has at most $|V| - 1$ edges); a change on step $|V|$ proves a negative cycle. Dijkstra is the same relax with a clever ORDER; Bellman-Ford orders nothing and pays for it in passes.

Doctests:

```
>>> bellman_ford([(('a','b',1), ('b','c',2), ('a','c',4)], 'a') == {'a': 0, 'b': 1,
    ↪ 'c': 3}
True
>>> bellman_ford([(('a','b',5), ('b','c',-4), ('a','c',2)], 'a')['c']
1
>>> bellman_ford([(('a','b',1), ('b','c',-3), ('c','b',1), ('b','d',1)], 'a') is None
True
>>> bellman_ford([(('x','y',7)], 'y')
{'y': 0}
>>> bellman_ford([(('a','b',-2), ('b','a',3)], 'a') == {'a': 0, 'b': -2}
True
```

Solution.

```
INF = float("inf")

def bellman_ford(edges, src):
    n_nodes = len({n for u, v, _ in edges for n in (u, v)} | {src})

    def relax(state):
        steps, dist = state
        new = dict(dist)
        for u, v, w in edges:
            if new.get(u, INF) + w < new.get(v, INF):
                new[v] = new[u] + w
        return (steps + 1, new)

    def settled(state):
        steps, dist = state
        return steps >= n_nodes or relax(state)[1] == dist

    steps, dist = until(settled, relax, (0, {src: 0}))
    return None if relax((steps, dist))[1] != dist else dist
```

Remark. Shortest paths *are* a fixed point: the least map satisfying every edge's triangle inequality. `until` iterates the relax step to stability; a change on pass $|V|$ proves a negative cycle. Dijkstra is the same relax with a clever order; Bellman-Ford orders nothing and pays in passes.

19.8 course_schedule: NeetCode: Course Schedule (cycle detection via Kahn)

```
Contract:
    can_finish(n: int, prereqs: list[tuple]) -> bool
```

Courses 0..n-1; (a, b) means b must precede a. True iff every course can be taken, i.e. the prerequisite graph is acyclic.
`find_order(n: int, prereqs: list[tuple]) -> list | None`
 A valid course order (smallest-numbered course first on ties), or None if impossible.

Hint:

Kahn's insight inverted: topo sort doesn't FAIL on a cycle, it just leaves the cycle behind -- nodes on a cycle never reach indegree 0. So: run 06's Kahn, then `len(topo) == n` is the whole cycle test. Heap instead of deque only to make ties deterministic.

Doctests:

```
>>> find_order(2, [(1, 0)])
[0, 1]
>>> find_order(4, [(1, 0), (2, 0), (3, 1), (3, 2)])
[0, 1, 2, 3]
>>> find_order(2, [(1, 0), (0, 1)]) is None
True
>>> find_order(3, [])
[0, 1, 2]
>>> can_finish(2, [(1, 0)])
True
>>> can_finish(2, [(1, 0), (0, 1)])
False
>>> can_finish(5, [(1, 4), (2, 4), (3, 1), (3, 2)])
True
```

Solution.

```
def find_order(n, prereqs):
    adj, indeg = Tree(1, list), Tree(1, int)
    for a, b in prereqs:
        adj[b].append(a)
        indeg[a] += 1
    heap = []
    for c in range(n):
        if indeg[c] == 0:
            heappush(heap, c)
    topo = []
    while heap:
        u = heappop(heap)
        topo.append(u)
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                heappush(heap, v)
    return topo if len(topo) == n else None

def can_finish(n, prereqs):
    return find_order(n, prereqs) is not None
```

Chapter 20

Class VIII — Dynamic Programming and Control

Choices now change what is possible later – so memoise the future’s needs. The craft is naming the *state*: index pairs for alignments (`lcs`, `edit_distance`), remaining capacity for knapsacks, a rectangle for guillotine cuts, a bitmask plus room for the partition problem. If you can say “the future only needs *X*” in one sentence, `memo` over *X* writes the rest. The parser closes the class: recursive descent is control flow as grammar. *Practice files live in g8_dp_and_control/; each carries the prelude snippets it needs inline. Attempt every problem there before reading on – the doctests are the referee.*

20.1 coin_change: NeetCode: Coin Change

Contract:

```
coin_change(coins: list[int], amount: int) -> int
Fewest coins summing to amount (unlimited supply), -1 if impossible.
amount >= 0, coins positive.
```

Hint:

```
Knapsack, dp[remaining]. Top-down with memo on the single
index `amount` -- the future only needs what remains.
```

Doctests:

```
>>> coin_change([1, 2, 5], 11)
3
>>> coin_change([2], 3)
-1
>>> coin_change([1], 0)
0
>>> coin_change([186, 419, 83, 408], 6249)
20
```

Solution.

```
INF = float("inf")

def coin_change(coins, amount):
```

```

@memo
def go(rem):
    if rem == 0:
        return 0
    return min([1 + go(rem - c) for c in coins if c <= rem],
               default=INF)
best = go(amount)
return -1 if best == INF else best

```

20.2 lcs: NeetCode: Longest Common Subsequence

Contract:

```

lcs(a: str, b: str) -> int
Length of the longest common subsequence (not substring).

```

Hint:

```

Alignment pattern, dp[i][j] -- same skeleton as 03's edit
distance, but max instead of min. Index-based args for the cache.

```

Doctests:

```

>>> lcs("abcde", "ace")
3
>>> lcs("abc", "abc")
3
>>> lcs("abc", "def")
0
>>> lcs("", "abc")
0

```

Solution.

```

def lcs(a, b):
    @memo
    def go(i, j):
        if i == len(a) or j == len(b):
            return 0
        if a[i] == b[j]:
            return 1 + go(i + 1, j + 1)
        return max(go(i + 1, j), go(i, j + 1))
    return go(0, 0)

```

20.3 edit_distance: Levenshtein distance

Minimum single-character insertions, deletions, or substitutions turning a into b.

Contract:

```

edit_distance(a: str, b: str) -> int

```

Hint:

The alignment DP again -- 02's skeleton with min-of-three plus one. Matching heads cost nothing; otherwise pay 1 and recurse on the three repairs. memo over the index pair.

Doctests:

```
>>> edit_distance("kitten", "sitting")
3
>>> edit_distance("", "abc")
3
>>> edit_distance("same", "same")
0
>>> edit_distance("flaw", "lawn")
2
>>> edit_distance("a", "b")
1
```

Solution.

```
def edit_distance(a, b):
    @memo
    def go(i, j):
        if i == len(a):
            return len(b) - j
        if j == len(b):
            return len(a) - i
        if a[i] == b[j]:
            return go(i + 1, j + 1)
        return 1 + min(go(i + 1, j), go(i, j + 1), go(i + 1, j + 1))
    return go(0, 0)
```

20.4 knapsack: 0/1 knapsack

Each (weight, value) item usable at most once; maximize value within capacity.

Contract:

```
knapsack(capacity: int, items: list[tuple[int, int]]) -> int
```

Hint:

memo over (i, room): take item i or skip it. Two integers of state -- the cache keys on the args tuple, so keep them hashable.

Doctests:

```
>>> knapsack(10, [(5, 10), (4, 40), (6, 30), (3, 50)])
90
>>> knapsack(0, [(1, 100)])
0
>>> knapsack(5, [])
0
>>> knapsack(3, [(4, 100), (2, 7)])
7
```

Solution.

```
def knapsack(capacity, items):
    items = list(items)
    @memo
    def go(i, room):
        if i == len(items) or room == 0:
            return 0
        w, v = items[i]
        skip = go(i + 1, room)
        if w > room:
            return skip
        return max(skip, v + go(i + 1, room - w))
    return go(0, capacity)
```

20.5 calculator: Integer expression evaluator (capstone)

+, -, * and parentheses, optional spaces, normal precedence, left-associative. No unary minus, no division.

Contract:

```
evaluate(expr: str) -> int
```

Grammar (one function per rule, shared position cursor):

```
expression := term (('+' | '-') term)*
term       := factor ('*' factor)*
factor     := NUMBER | '(' expression ')'
```

Hint:

Recursive descent. Lex numbers with `takeWhile str.isdigit` on the remaining input -- `span` IS the lexer. And remember the trap you certified earlier: `'` in `"+-"` is `True`; test membership against a tuple.

Doctests:

```
>>> evaluate("2*(3+4)")
14
>>> evaluate("2*(3+4)-5")
9
>>> evaluate("10-2-3")
5
>>> evaluate("2+3*4")
14
>>> evaluate("((1+2))*3")
9
>>> evaluate(" 12 * 2 ")
24
```

Solution.

```
def evaluate(expr):
    s = expr.replace(" ", "")
```

```

pos = [0]

def peek():
    return s[pos[0]] if pos[0] < len(s) else ""

def number():
    digits = takeWhile(str.isdigit, s[pos[0]:])
    pos[0] += len(digits)
    return int("".join(digits))

def factor():
    if peek() == "(":
        pos[0] += 1
        v = expression()
        pos[0] += 1
        return v
    return number()

def term():
    v = factor()
    while peek() == "*":
        pos[0] += 1
        v *= factor()
    return v

def expression():
    v = term()
    while peek() in ("+", "-"):
        op = peek()
        pos[0] += 1
        v = v + term() if op == "+" else v - term()
    return v

return expression()

```

20.6 max_height_cuboids: LC 1691: Maximum Height by Stacking Cuboids

Any rotation allowed; cuboid A sits on B only when ALL THREE of A's dims are $\leq$ B's, compared side to side. Each cuboid used at most once. Maximize stack height.

Contract:

```
max_height(cuboids: list[tuple[int, int, int]]) -> int
```

Hint:

The rotation freedom COLLAPSES: sort each cuboid's own dims ascending -- the biggest dim as height is always at least as good, and sorted-vs-sorted is the only comparison that can succeed (the photo_lineup exchange argument, twice). Then sort the cuboids and run weighted LIS: best ending at $i = h_i + \text{best over } j \text{ fitting}$

under i. Fold-with-history, $O(n^2)$ -- the g4_07 shape with sum instead of length. (g8_06's mask version is the ground truth oracle for THIS file: same ≥ 17 -style surprises, different fit rule -- all three dims here, base only there.)

Doctests:

```
>>> max_height([(50, 45, 20), (95, 37, 53), (45, 23, 12)])
190
>>> max_height([(38, 25, 45), (76, 35, 3)])
76
>>> max_height([(7, 11, 17), (7, 17, 11), (11, 7, 17), (11, 17, 7), (17, 7, 11), (17,
    ↪ 11, 7)])
102
>>> max_height([(1, 1, 1)])
1
```

Solution.

```
def max_height(cuboids):
    cs = sorted([sorted(c) for c in cuboids])
    fits = lambda a, b: all(zipWith(lambda x, y: x <= y, a, b)) # a under-or-equal
    ↪ b, all three dims
    best = []
    for i, c in enumerate(cs):
        best.append(c[2] + max([best[j] for j in range(i) if fits(cs[j], c)],
                               default=0))
    return max(best, default=0)
```

20.7 guillotine__cut: Max value cut from a sheet, guillotine cuts only

A $W \times H$ sheet; piece types (pw, ph, value), unlimited supply, fixed orientation (add rotated copies yourself if rotation is allowed). Every cut runs edge to edge, so each cut splits one rectangle into two rectangles. Unused offcuts score 0.

Contract:

```
guillotine(W: int, H: int, pieces: list[tuple[int, int, int]]) -> int
```

Hint:

Guillotine is what re-admits DP: free-form 2D packing has no small state (the frontier goes irregular), but edge-to-edge cuts keep every subproblem a RECTANGLE -- (w, h), two ints. Interval pattern: $\text{best}(w, h) = \max$ of a piece that fits the sheet EXACTLY, every vertical split x , every horizontal split y . A smaller piece never needs a direct case: the cut that isolates it is one of the splits. memo over (w, h); $O(W*H*(W+H))$ -- say it.

Doctests:

```
>>> guillotine(4, 4, [(1, 1, 1)])
16
```

```

>>> guillotine(4, 4, [(3, 3, 7), (1, 1, 1)])
16
>>> guillotine(4, 4, [(3, 3, 20), (1, 1, 1)])
27
>>> guillotine(4, 4, [(4, 2, 10), (2, 2, 3)])
20
>>> guillotine(3, 3, [(2, 2, 5)])
5
>>> guillotine(2, 2, [(3, 1, 99)])
0

```

Solution.

```

def guillotine(W, H, pieces):
    pieces = list(pieces)

    @memo
    def best(w, h):
        exact = [v for pw, ph, v in pieces if (pw, ph) == (w, h)]
        vsplit = [best(x, h) + best(w - x, h) for x in range(1, w // 2 + 1)]
        hsplit = [best(w, y) + best(w, h - y) for y in range(1, h // 2 + 1)]
        return max(exact + vsplit + hsplit, default=0)

    return best(W, H)

```

Remark. Guillotine cuts are what re-admit dynamic programming: free-form 2-D packing has no small state, but edge-to-edge cuts keep every subproblem a rectangle – two integers. A smaller piece never needs a direct case; the cut that isolates it is one of the splits.

20.8 partition_k_subsets: LC 698: Partition to K Equal Sum Subsets

Contract:

```

can_partition(nums: list[int], k: int) -> bool
True iff nums splits into k non-empty groups of equal sum.
Positive ints, small n (say the bound: 2^n states).

```

Hint:

The mask family, pure form: the future needs (which numbers remain, how much room is left in the group being filled). When a group closes (room hits 0) the room resets to target -- so the state is just (used, room): the number of CLOSED groups is implied by the used-sum. memo over that pair; fill greedily into one group at a time to avoid counting group orderings. (g8_06's go is this skeleton plus geometry.)

Doctests:

```

>>> can_partition([4, 3, 2, 3, 5, 2, 1], 4)
True
>>> can_partition([1, 2, 3, 4], 3)
False
>>> can_partition([2, 2, 2, 2], 2)

```

```
True
>>> can_partition([1], 1)
True
>>> can_partition([1, 1], 3)
False
```

Solution.

```
def can_partition(nums, k):
    nums = list(nums)
    total, n = sum(nums), len(nums)
    if k <= 0 or n < k or total % k:
        return False
    target = total // k

    @memo
    def go(used, room):
        if used == (1 << n) - 1:
            return room == target          # last group closed cleanly
        return any(go(used | (1 << i),
                    target if room == nums[i] else room - nums[i])
                  for i in range(n)
                  if not used & (1 << i) and nums[i] <= room)

    return go(0, target)
```

Remark. The mask family in pure form: the future needs only (which numbers remain, room left in the open group), because a closed group's room resets to the target. Fill one group at a time so orderings are not counted twice. 2^n states – say the bound before you code.

20.9 min_path_sum: Cheapest right/down path through a grid

```
Contract:
    min_path_sum(grid: list[list[int]]) -> int
    Top-left to bottom-right, moving only right or down.
```

```
Hint:
    Grid DP: state (r, c) -- the future only cares where you stand.
    memo over the index pair; base case is the goal cell.
    Compare g7_02: uniform steps there needed BFS; weighted cells
    with only right/down moves make a DAG, so plain DP suffices.
```

Doctests:

```
>>> min_path_sum([[1, 3, 1], [1, 5, 1], [4, 2, 1]])
7
>>> min_path_sum([[5]])
5
>>> min_path_sum([[1, 2], [3, 4]])
7
```

Solution.

```
def min_path_sum(grid):
```

```
R, C = len(grid), len(grid[0])

@memo
def go(r, c):
    if (r, c) == (R - 1, C - 1):
        return grid[r][c]
    opts = ([go(r + 1, c)] if r + 1 < R else []) + \
           ([go(r, c + 1)] if c + 1 < C else [])
    return grid[r][c] + min(opts)

return go(0, 0)
```


Chapter 21

Class IX — Stack Folds

The prelude has no stack section because a plain Python list already is one: `append` pushes, `pop` pops, both $O(1)$. What this class adds is the *fold* discipline: the stack is the accumulator, and in `balanced_brackets` the failed state is `None` – a poison value that propagates untouched, Maybe as an accumulator. `next_greater` graduates to the monotonic stack, the pro tier, with its lovely amortised $O(n)$ argument. *Practice files live in `g9_stack_folds/`; each carries the prelude snippets it needs inline. Attempt every problem there before reading on – the doctests are the referee.*

21.1 `balanced_brackets`: Matching pairs: valid bracket nesting

Only `()[]{}` appear.

Contract:

`balanced(s: str) -> bool`

Hint:

The stack fold, purest form: a plain list IS a stack, and the fold's accumulator is one. The twist worth learning: use `None` as the FAILED state -- once a closer mismatches, the poison propagates through the rest of the fold untouched (Maybe as accumulator). `Balanced == the fold ends on an empty stack.`

Doctests:

```
>>> balanced("([{}])")
True
>>> balanced("([])")
False
>>> balanced("(((")
False
>>> balanced("")
True
>>> balanced("]")
False
```

Solution.

```
MATCH = {"(": ")", "[": "]", "{": "}"}
```

```
def balanced(s):
    def step(stack, c):
        if stack is None:
            return None                # poison: already failed
        if c in "([{":
            return stack + [c]
        if not stack or stack[-1] != MATCH[c]:
            return None
        return stack[:-1]
    return foldl(step, s, []) == []
```

Remark. A fold whose accumulator is a stack – and whose failure state is `None`, propagating untouched through the rest of the fold. That is `Maybe` as an accumulator, and it removes every flag and early return from the classic solution.

21.2 eval_rpn: Evaluate Reverse Polish notation

Integer operands, operators `+` `-` `*`.

Contract:

```
eval_rpn(tokens: list[str]) -> int
```

Hint:

One fold over the tokens: operands push; an operator pops two and pushes the result. The classic slip: `stack[-2]` is the LEFT operand. The answer is whatever the fold leaves on the stack.

Doctests:

```
>>> eval_rpn(["2", "3", "+", "4", "*"])
20
>>> eval_rpn(["5", "1", "2", "+", "4", "*", "-"])
-7
>>> eval_rpn(["7"])
7
```

Solution.

```
OPS = {"+": add, "-": sub, "*": lambda a, b: a * b}
```

```
def eval_rpn(tokens):
    def step(stack, tok):
        if tok in OPS:
            return stack[:-2] + [OPS[tok](stack[-2], stack[-1])]
        return stack + [int(tok)]
    return foldl(step, tokens, [])[0]
```

21.3 next_greater: First later element that's larger

For each element, the next strictly greater element to its right;

```
-1 if none.
```

Contract:

```
next_greater(xs: list[int]) -> list[int]
```

Hint:

The MONOTONIC stack: fold over `enum(xs)`; the newcomer pops every stacked index whose value it beats (that newcomer is their answer), then pushes itself. Each index pushes and pops at most once -- $O(n)$ despite the inner while. The pro tier of stacks.

Doctests:

```
>>> next_greater([2, 1, 2, 4, 3])
[4, 2, 4, -1, -1]
>>> next_greater([5, 4, 3])
[-1, -1, -1]
>>> next_greater([])
[]
```

Solution.

```
def next_greater(xs):
    out = [-1] * len(xs)
    def step(stack, ix):
        i, x = ix
        while stack and xs[stack[-1]] < x:
            out[stack.pop()] = x
        return stack + [i]
    foldl(step, enum(xs), [])
    return out
```

Remark. The monotonic stack: the newcomer pops every index it beats (it is their answer) and then waits for its own. Each index pushes and pops at most once, so the nested `while` is still $O(n)$ – amortised analysis in one picture.

Part IV

Appendices

Appendix A

prelude.py, Complete

The whole file, exactly as it ships – 15 sections, define-before-use, no line over 105 columns.

```
1  """prelude.py -- Haskell-style Prelude in pure Python, in pedagogical order.
2
3  Pascal discipline: strict define-before-use. Read top to bottom and every name is already defined
4  when you meet it. Sections build on each other:
5
6      1 combinators          2 pairs          3 arithmetic
7      4 list basics          5 higher-order lists  6 folds
8      7 scans                8 streams (lazy)  9 slicing & spans
9      10 sorting & searching 11 strings       12 containers
10     13 nodes               14 grids & windows 15 control
11
12  Within a section names run alphabetically, except that a definition is hoisted ahead of the first
13  name that uses it (foldl before compose, span before break_), so the discipline holds inside
14  sections too.
15
16  Import qualified (import prelude as P): Haskell names shadow builtins by design. Beyond the Prelude
17  proper: the working parts of Data.List, Data.Map, Data.Maybe, Data.Function and friends are folded
18  into the same sections -- grouped by what they do, not by the module they came from. None is
19  Nothing throughout. foldr = foldl(flip(f), reversed(xs), base): iterative, stack-safe, valid only
20  because Python is strict.
21  """
22
23  # ===== 1. combinators ===== (functions about functions)
24
25  const      = lambda x: lambda _: x                # always returns x, ignoring the second
                ↪ argument
26  # curry: converts a two-argument function into a chain of two one-argument functions, so instead
27  # of calling f(x, y), you call curry(f)(x)(y).
28  curry      = lambda f: lambda x: lambda y: f(x, y)
29  flip       = lambda f: (lambda x, y: f(y, x))      # flip f x y = f y x
30  NOTHING    = object()                            # Maybe's Nothing: "no arg given"; test with
                ↪ `is`
31
32                                     # hoisted ahead of foldl, which needs it as
                ↪ a sentinel
33
34  def foldl(f, xs, base=NOTHING): # THE left fold; Python buried its own in functools as reduce
35      it = iter(xs)
36      if base is NOTHING:
37          try:
38              acc = next(it)
```

```

38     except StopIteration:
39         raise TypeError("fold of empty sequence with no initial value")
40     else:
41         acc = base
42     for x in it:
43         acc = f(acc, x)
44     return acc
45
46 foldr = lambda f, xs, base: foldl(flip(f), reversed(list(xs)), base) # THE right fold, via foldl
47 compose = lambda *fns: lambda x: foldr(lambda f, acc: f(acc), fns, x) # foldr (.) id: RIGHTMOST
    ↪ first
48 fromMaybe = lambda d, x: d if x is None else x # Data.Maybe: default for every
    ↪ None-returning tool
49 id = lambda x: x # shadows builtin id() by design
50 isJust = lambda x: x is not None # Data.Maybe: None test as a handable
    ↪ predicate
51 isNothing = lambda x: x is None # Data.Maybe: true when x is Nothing (None)
52 on = lambda f, g: lambda x, y: f(g(x), g(y)) # Data.Function: combine via a key -- cmp
    ↪ `on` fst
53 partial = lambda f, *bound: lambda *args: f(*bound, *args) # fix leading args of f, wait for
    ↪ the rest
54 uncurry = lambda f: lambda p: f(*p) # inverse of curry: call f with a pair, not
    ↪ two args
55
56 # ===== 2. pairs =====
57
58 fst = lambda p: p[0] # first element of a pair
59 snd = lambda p: p[1] # second element of a pair
60 swap = compose(tuple, reversed) # Data.Tuple
61
62 # ===== 3. arithmetic & logic =====
63
64 add = lambda a, b: a + b # (+) as a value:
    ↪ scanl1(add, xs)
65 div = lambda a, b: a // b # floors, like Haskell div
66 even = lambda n: n % 2 == 0 # true for even numbers
67 gcd = lambda a, b: abs(a) if b == 0 else gcd(b, a % b) # >= 0 like Haskell;
    ↪ depth <= 90
68 halve = lambda n: n // 2 # e.g. iterate(halve, 1024) -> 1024, 512, 256, ...
69 hypot = lambda x, y: (x*x + y*y) ** 0.5 # straight-line distance
70
71 def isqrt(n): # floor sqrt without floats (Newton)
72     if n < 0:
73         raise ValueError("isqrt of negative")
74     x = n
75     y = halve(x + 1)
76     while y < x:
77         x, y = y, halve(y + n // y)
78     return x if n else 0
79
80 lcm = lambda a, b: abs(a // gcd(a, b) * b) if a and b else 0 # >= 0 like Haskell
81 mod = lambda a, b: a % b # remainder, sign follows
    ↪ b
82 not_ = lambda b: not b # `not` is a Python
    ↪ keyword
83 odd = lambda n: n % 2 == 1 # true for odd numbers
84 otherwise = True # guard fallback: always
    ↪ true
85 pred = lambda x: chr(ord(x) - 1) if isinstance(x, str) else x - 1 # Enum: numbers and Chars

```

```

86 quot      = lambda a, b:  $-(a // b)$  if (a < 0) != (b < 0) else a // b      # truncates, like Haskell
    ↪ quot
87 rem       = lambda a, b: a - b * quot(a, b)                               # remainder, sign follows
    ↪ a
88 quotRem   = lambda a, b: (quot(a, b), rem(a, b))                         # (quot, rem) in one call
89 signum    = lambda x: (x > 0) - (x < 0)                                  # -1, 0, or 1 by the sign
    ↪ of x
90 sub       = lambda a, b: a - b                                           # (-) as a value:
    ↪ zipWith(sub, a, b)
91 succ      = lambda x: chr(ord(x) + 1) if isinstance(x, str) else x + 1    # next: increments,
    ↪ advances chars
92
93 # ===== 4. list basics =====
94
95 drop       = lambda n, xs: list(xs)[max(n, 0):]                          # finite lists; n<0
    ↪ drops none
96 elem      = lambda x, xs: x in xs                                         # true if x is in xs
97 zip_      = lambda a, b: list(zip(a, b))                                  # shortest wins, any
    ↪ iterable
98 enum      = lambda xs, start=0: zip_(list(range(start, start + len(xs))), xs) # pairs each x
    ↪ with its index
99 head      = lambda xs: xs[0]                                              # first element
100 init     = lambda xs: xs[:-1]                                             # all but the last
    ↪ element
101 isPrefixOf = lambda p, xs: list(xs[:len(p)]) == list(p)                  # Data.List; strings or
    ↪ lists
102 isSuffixOf = lambda p, xs: list(xs[len(xs) - len(p):]) == list(p)        # not [-len(p):]: [-0:]
    ↪ is ALL
103 last     = lambda xs: xs[-1]                                              # last element
104 lookup    = lambda k, pairs: next((v for kk, v in pairs if kk == k), None) # Nothing -> None
105 notElem   = lambda x, xs: x not in xs                                     # true if x is not in xs
106 nub      = compose(list, dict.fromkeys)                                    # unique; first seen
    ↪ wins
107 null     = lambda xs: len(xs) == 0                                        # true for an empty list
108 pairwise = lambda xs: zip_(xs, xs[1:])                                    # zip xs (tail xs)
109 replicate = lambda n, x: [x] * n                                          # x repeated n times
110 splitAt  = lambda n, xs: (xs[:n], xs[n:]) if n >= 0 else (xs[:0], xs)    # n<0 splits at the
    ↪ front
111 stripPrefix = lambda p, xs: xs[len(p):] if isPrefixOf(p, xs) else None    # Just the rest, or
    ↪ Nothing
112 tail     = lambda xs: xs[1:]                                              # all but the first
    ↪ element
113 def transpose(xss):                                                         # Data.List: rows <-> cols, RAGGED-SAFE like Haskell
114     xss = [list(xs) for xs in xss]    # any iterables in (rows may be one-shot); materialise once
115     n = max(map(len, xss), default=0) # short rows just drop out of later columns (no truncation)
116     return [[xs[i] for xs in xss if i < len(xs)] for i in range(n)]
117 unzip    = lambda ps: tuple(map(list, zip(*ps))) if ps else ([], [])      # inverse of zip: pairs
    ↪ -> two lists
118 zip3     = lambda a, b, c: list(zip(a, b, c))                             # zip three lists
    ↪ together
119
120 # ===== 5. higher-order lists =====
121
122 filter_   = lambda cond, xs: [x for x in xs if cond(x)]                   # keep elements where cond
    ↪ holds
123 catMaybes = partial(filter_, isJust)                                       # mapMaybe id
124 concat    = lambda xss: [x for xs in xss for x in xs]                    # flatten one level of
    ↪ nesting
125 concatMap = lambda f, xs: [y for x in xs for y in f(x)]                  # map f over xs, then flatten

```

```

126 cross      = lambda a, b: [(x, y) for x in a for y in b]           # cartesian; `product` is
    ↪ the fold
127 # Data.List find: lazy first-match -- folds can't stop early, find can
128 find        = lambda cond, xs: next((x for x in xs if cond(x)), None) # first match, else None
129 map_         = lambda f, xs: [f(x) for x in xs]                     # apply f to every element
130 # Data.Maybe mapMaybe: map and drop the Nothings in ONE pass -- the parse-and-filter shape
131 mapMaybe    = lambda f, xs: [y for y in (f(x) for x in xs) if y is not None]
132 def partition(cond, xs):                                           # (keepers, rest) in ONE pass; cond called once per
    ↪ element
133     yes, no = [], []
134     for x in xs:
135         (yes if cond(x) else no).append(x)
136     return (yes, no)
137 starmap      = lambda f, pairs: [f(*p) for p in pairs]             # map . uncurry
138 zipWith      = lambda f, a, b: [f(x, y) for x, y in zip(a, b)]      # combine two lists
    ↪ elementwise
139 zipWith3     = lambda f, a, b, c: [f(x, y, z) for x, y, z in zip(a, b, c)] # combine three lists
    ↪ elementwise
140
141 # ===== 6. folds ===== (collapse a list to one value)
142 # foldl, foldr and compose are hoisted up to section 1 (combinators): compose needs foldr, foldr
    ↪ needs
143 # foldl, and swap (section 2) needs compose -- so all three must exist before section 2 even
    ↪ begins.
144
145 foldl1       = foldl                                               # foldl seeded by the
    ↪ first element
146 foldr1       = lambda f, xs: foldl(flip(f), reversed(list(xs)))    # foldr seeded by the last
    ↪ element
147 product      = lambda xs: foldl(lambda a, x: a * x, xs, 1)         # the numeric fold
148
149 # enumeration folds -- brute-force licenses for small n (say the bound out loud):
150 # Control.Monad replicateM: every length-n word over xs -- cross, n times; |xs|^n
151 replicateM    = lambda n, xs: foldl(lambda acc, _: [s + [x] for s in acc for x in xs], range(n),
    ↪ [[]])
152 # Data.List subsequences: the powerset; each element DOUBLES acc -- 2^n, fine to n ~ 20
153 subsequences  = lambda xs: foldl(lambda acc, x: acc + [s + [x] for s in acc], list(xs), [[]])
154
155 # ===== 7. scans ===== (a fold that keeps its history)
156
157 def accumulate(xs, f=None, initial=NOTHING):                       # scanl / scanl1
158     if f is None:
159         f = add
160     it = iter(xs)
161     if initial is NOTHING:
162         try:
163             acc = next(it)
164         except StopIteration:
165             return
166     else:
167         acc = initial
168     yield acc
169     for x in it:
170         acc = f(acc, x)
171         yield acc
172
173 scanl         = lambda f, z, xs: list(accumulate(xs, f, initial=z)) # foldl, keeping every
    ↪ accumulator

```

```

174 scanl1 = lambda f, xs: list(accumulate(xs, f))                # scanl with no explicit
    ↪ base
175 scanr  = lambda f, z, xs: list(reversed(scanl(flip(f), z, list(reversed(list(xs))))) # foldr,
    ↪ keeping every acc
176 scanr1 = lambda f, xs: list(reversed(scanl1(flip(f), list(reversed(list(xs)))))    # scanr
    ↪ with no base

177
178 # ===== 8. streams ===== (lazy, possibly infinite)
179
180 chain    = lambda *iterables: (x for it in iterables for x in it)  # concat, lazily
181
182 def count(start=0, step=1):                # [start, start+step ..]
183     n = start
184     while True:
185         yield n
186         n += step
187
188 def cycle(xs):                             # repeat xs forever
189     xs = list(xs)
190     if not xs:
191         raise ValueError("cycle: empty list")  # Haskell errors too; the alternative is a silent
    ↪ hang
192     while True:
193         yield from xs
194
195 def iterate(f, x):                         # iterate f x = [x, f x, f (f x), ..]
196     while True:
197         yield x
198         x = f(x)
199
200 def repeat(x, n=None):                     # yield x forever, or n times if given
201     if n is None:
202         while True:
203             yield x
204     else:
205         for _ in range(n):
206             yield x
207
208 def unfoldr(f, seed):                      # Data.List unfoldr: f(seed) -> None | (value, seed')
209     out = []                               # iterate's finite twin: grows a list until f says
    ↪ stop --
210     step = f(seed)                         # no more threading (state, acc) tuples through until
211     while step is not None:
212         val, seed = step
213         out.append(val)
214         step = f(seed)
215     return out
216
217 # ===== 9. slicing & spans ===== (finite views of streams)
218
219 def span(cond, xs):                        # split at first failure, ONE pass; safe on one-shot
    ↪ iters
220     xs = list(xs)
221     i = next((i for i, x in enumerate(xs) if not cond(x)), len(xs))
222     return (xs[:i], xs[i:])
223 break_ = lambda cond, xs: span(lambda x: not cond(x), xs)      # split at first hit
    ↪ (opposite of span)
224 chunksOf = lambda n, xs: [xs[i:i + n] for i in range(0, len(xs), n)] # Data.List.Split; short
    ↪ last ok

```

```

225
226 def dropwhile(cond, xs):                                # generator: drop the leading run where cond holds
227     it = iter(xs)
228     for x in it:
229         if not cond(x):
230             yield x
231             break
232     yield from it
233
234 dropWhile = lambda cond, xs: list(dropwhile(cond, xs))    # drop the leading run where cond holds,
    ↪ as a list
235
236
237 def islice(iterable, stop):                                # take
238     it = iter(iterable)
239     for _ in range(stop):
240         try:
241             yield next(it)
242         except StopIteration:
243             return
244
245 tails      = lambda xs: [xs[i:] for i in range(len(xs) + 1)] # every suffix; substrings start here
246 take       = lambda n, xs: list(islice(iter(xs), n))         # works on infinite streams
247
248 def takeuntil(cond, xs):                                   # like takewhile(not . cond), but INCLUDES the first
    ↪ hit
249     for x in xs:
250         yield x
251         if cond(x):
252             return
253
254 def takewhile(cond, xs):                                   # generator: yield the leading run where cond holds
255     for x in xs:
256         if not cond(x):
257             return
258     yield x
259
260 takeWhile = lambda cond, xs: list(takewhile(cond, xs))    # take the leading run where cond holds,
    ↪ as a list
261
262 # ===== 10. sorting & searching =====
263
264 def bisect_left(a, x):                                     # first index where a[i] >= x
265     lo, hi = 0, len(a)
266     while lo < hi:
267         mid = halve(lo + hi)
268         if a[mid] < x:
269             lo = mid + 1
270         else:
271             hi = mid
272     return lo
273
274 def bisect_right(a, x):                                    # first index where a[i] > x
275     lo, hi = 0, len(a)
276     while lo < hi:
277         mid = halve(lo + hi)
278         if a[mid] <= x:
279             lo = mid + 1
280     else:

```

```

281         hi = mid
282     return lo
283
284 def cmp_to_key(cmp):                                # resurrect Python 2's cmp for sortBy
285     class K:
286         def __init__(self, x): self.x = x
287         def __lt__(self, other): return cmp(self.x, other.x) < 0
288     return K
289
290 maxOn = lambda f, xs: max(xs, key=f)                # maximumBy (comparing f) in O(n); first wins ties
291
292 def merge(a, b):                                    # merge two sorted lists, stable; the heart of merge
293     ↪ sort
294     out, i, j = [], 0, 0                            # (section 13's merge_lists is this same loop on
295     ↪ ListNodes)
296     while i < len(a) and j < len(b):
297         if a[i] <= b[j]:
298             out.append(a[i]); i += 1
299         else:
300             out.append(b[j]); j += 1
301     return out + a[i:] + b[j:]
302
303 minOn = lambda f, xs: min(xs, key=f)                # minimumBy (comparing f): sortOn + head without the
304     ↪ sort
305
306 sortBy = lambda cmp, xs: sorted(xs, key=cmp_to_key(cmp)) # comparator sort, Python 2 style
307 sortOn = lambda f, xs: sorted(xs, key=f)            # sortOn (schwartzian, f called once per element)
308
309 # ===== 11. strings =====
310
311 lines = str.splitlines                               # split on newlines
312 unlines = lambda ls: "".join(l + "\n" for l in ls)   # join lines back with trailing newlines
313 unwords = " ".join                                  # join words with single spaces
314 words = str.split                                    # split on whitespace
315
316 # ===== 12. containers =====
317
318 # multiset ops on Counters (use .get, not [] -- indexing a Counter autovivifies zeros):
319 bag_diff = lambda a, b: {k: a[k] - b[k] for k in a if a[k] > b.get(k, 0)} # clipped at 0
320 bag_inter = lambda a, b: {k: v for k in a.keys() & b.keys() if (v := min(a.get(k, 0), b.get(k,
321     ↪ 0)))} # min count per shared key
322 bag_sub = lambda a, b: {k: a.get(k, 0) - b.get(k, 0) for k in a.keys() if a.get(k, 0) > b.get(k, 0)} # a <= b (inclusion)
323 bag_union = lambda a, b: {k: max(a.get(k, 0), b.get(k, 0)) for k in a.keys() | b.keys()}
324
325 class defaultdict(dict):                             # dict that manufactures a value via factory() on
326     ↪ first miss
327     def __init__(self, factory=None, *args, **kw):
328         super().__init__(*args, **kw)
329         self.factory = factory
330     def __missing__(self, key):
331         if self.factory is None:
332             raise KeyError(key)
333         self[key] = self.factory()
334     return self[key]
335
336 def Counter(xs):                                     # count-by-value; a defaultdict(int) fold
337     d = defaultdict(int)
338     for x in xs:
339         d[x] += 1
340     return d

```

```

335
336 class deque:                                # two-stack; all four ends amortized O(1)
337     def __init__(self, xs=()):
338         self._in, self._out = [], list(reversed(list(xs)))
339     def append(self, x): self._in.append(x)
340     def appendleft(self, x): self._out.append(x)
341     def pop(self):
342         if not self._in:
343             self._in, self._out = list(reversed(self._out)), []
344         return self._in.pop()
345     def popleft(self):
346         if not self._out:
347             self._out, self._in = list(reversed(self._in)), []
348         return self._out.pop()
349     def __len__(self): return len(self._in) + len(self._out)
350     def __iter__(self): return iter(self._out[::-1] + self._in)
351
352 def dsu(n):                                    # union-find, path halving; union -> False if already
    ↪ joined
353     parent = list(range(n))
354     def find(x):
355         while parent[x] != x:
356             parent[x] = parent[parent[x]]
357             x = parent[x]
358         return x
359     def union(a, b):
360         ra, rb = find(a), find(b)
361         if ra == rb:
362             return False
363         parent[ra] = rb
364         return True
365     return find, union
366
367 def fromListWith(f, pairs):                    # Data.Map fromListWith: THE dict-building fold;
    ↪ f(new, old)
368     d = {}                                    # like insertWith -- so grouping with concat REVERSES
    ↪ each
369     for k, v in pairs:                        # group (the classic Haskell gotcha); use add for
    ↪ counts
370         d[k] = f(v, d[k]) if k in d else v
371     return d                                  # Counter == fromListWith(add) over (x, 1);
372                                             # grouping == fromListWith(++) over (k, [v])
373
374 def getpath(t, ks, default=None):              # read along ks WITHOUT autovivifying
375     for k in ks:
376         if not isinstance(t, dict) or k not in t:
377             return default
378         t = t[k]
379     return t
380
381 def groupBy(eq, xs):                          # Data.List groupBy: runs of CONSECUTIVE elements;
    ↪ each new
382     out = []                                  # element is compared via eq against the run's FIRST
    ↪ element,
383     for x in xs:                              # exactly like Haskell (span-based) -- not its neighbor
384         if not and eq(out[-1][0], x):
385             out[-1].append(x)
386         else:
387             out.append([x])

```

```

388     return out
389
390 group = partial(groupBy, lambda a, b: a == b)    # Data.List group: runs of equals, [[a]] -- no keys
391
392 def heappop(h):                                # min-heap on a plain list: sift-down
393     h[0], h[-1] = h[-1], h[0]
394     top = h.pop()
395     i, n = 0, len(h)
396     while True:
397         s, l, r = i, 2*i + 1, 2*i + 2
398         if l < n and h[l] < h[s]: s = l
399         if r < n and h[r] < h[s]: s = r
400         if s == i:
401             return top
402         h[i], h[s] = h[s], h[i]
403         i = s
404
405 def heappush(h, x):                             # min-heap on a plain list: sift-up
406     h.append(x)
407     i = len(h) - 1
408     while i and h[halve(i - 1)] > h[i]:
409         h[halve(i - 1)], h[i] = h[i], h[halve(i - 1)]
410         i = halve(i - 1)
411
412 insertWith = lambda f, k, v, d: {**d, k: f(v, d[k]) if k in d else v}    # Data.Map insertWith:
413     ↪ PERSISTENT                                                         # (fresh dict); f(new, old)
414
415 ITree = lambda: defaultdict(ITree)          # infinitely-nestable defaultdict: a tree that grows
416     ↪ on access
417
418 def paths(t):                                # cata for Tree/ITree: [(keypath, leaf)]
419     # recursion depth = tree height (word length / len(nums)) -- well under the ~1000 limit
420     if not isinstance(t, dict) or not t:    # leaf = non-dict value, or empty node
421         return [([], t)]
422     return [[([k] + p, leaf) for k, sub in t.items() for p, leaf in paths(sub)]]
423
424 leaves = compose(partial(map_, snd), paths)    # every leaf value in a Tree/ITree, in path order
425
426 def setpath(t, ks, v):                       # write leaf v at key path ks (autovivifies interior)
427     for k in ks[:-1]:
428         t = t[k]
429     t[ks[-1]] = v
430     # NB: clobbers any subtree already at ks
431
432 Tree = lambda depth, leaf: (defaultdict(leaf) if depth == 1           # fixed-depth nested
433     ↪ defaultdict;
434     else defaultdict(lambda: Tree(depth - 1, leaf))) # leaves default to
435     ↪ leaf()
436
437 unionWith = lambda f, a, b: {**a, **{k: f(a[k], v) if k in a else v for k, v in b.items()}}
438     # Data.Map unionWith: f(a's value, b's value) on shared
439     ↪ keys;
440     # bag_union == unionWith(max), merge-sum == unionWith(add)
441
442 # ===== 13. nodes ===== (the LeetCode givens: binary trees & linked lists)
443
444 class ListNode:                               # the LeetCode linked list
445     def __init__(self, val=0, nxt=None):
446         self.val, self.next = val, nxt
447

```

```

442 class TreeNode:                                # the LeetCode binary tree
443     def __init__(self, val=0, left=None, right=None):
444         self.val, self.left, self.right = val, left, right
445
446 from_list = lambda xs: foldr(ListNode, xs, None)    # build; foldr of ListNode
447
448 def has_cycle(node):                             # Floyd: fast laps slow iff a cycle exists
449     slow = fast = node
450     while fast and fast.next:
451         slow, fast = slow.next, fast.next.next
452         if slow is fast:
453             return True
454     return False
455
456 # tfold: the tree cata -- f(val, l_acc, r_acc), z for the empty tree; recursion depth = tree height
457 tfold      = lambda f, z, t: z if t is None else f(t.val, tfold(f, z, t.left), tfold(f, z, t.right))
458 inorder    = partial(tfold, lambda v, l, r: l + [v] + r, [])    # left, root, right
459
460 def levelorder(t):                                # BFS: the deque earning its keep
461     out, q = [], deque([t] if t else [])
462     while len(q):
463         n = q.popleft()
464         out.append(n.val)
465         if n.left: q.append(n.left)
466         if n.right: q.append(n.right)
467     return out
468
469 def merge_lists(a, b):                             # merge two sorted lists; dummy-head idiom
470     dummy = tail_ = ListNode()
471     while a and b:
472         if a.val <= b.val:
473             tail_.next, a = a, a.next
474         else:
475             tail_.next, b = b, b.next
476         tail_ = tail_.next
477     tail_.next = a or b
478     return dummy.next
479
480 def middle(node):                                  # fast/slow: second middle for even lengths
481     slow = fast = node
482     while fast and fast.next:
483         slow, fast = slow.next, fast.next.next
484     return slow
485
486 postorder = partial(tfold, lambda v, l, r: l + r + [v], [])    # left, right, root
487 preorder  = partial(tfold, lambda v, l, r: [v] + l + r, [])    # root, left, right
488
489 def reverse_list(node):                            # three-pointer; prev is a fold accumulator
490     prev = None
491     while node:
492         node.next, prev, node = prev, node, node.next
493     return prev
494
495 tdepth     = partial(tfold, lambda _, l, r: 1 + max(l, r), 0)    # height of the tree
496 to_list    = compose(list, partial(unfoldr, lambda n: None if n is None else (n.val, n.next))) #
497             ↪ ListNode chain -> list
498 tsize      = partial(tfold, lambda _, l, r: 1 + l + r, 0)        # number of nodes in the tree
499 # ===== 14. grids & windows =====

```

```

500
501 def longest_window(xs, valid, add, shed):    # sliding-window skeleton; state lives in the closures
502     lo = best = 0
503     for hi in range(len(xs)):
504         add(xs[hi])
505         while not valid():                    # shrink until legal again
506             shed(xs[lo])
507             lo += 1
508         best = max(best, hi - lo + 1)
509     return best
510
511 def neighbors4(r, c, R, C):                  # in-bounds orthogonal neighbors
512     for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
513         if 0 <= r + dr < R and 0 <= c + dc < C:
514             yield r + dr, c + dc
515
516 def neighbors8(r, c, R, C):                  # ... plus diagonals
517     for dr in (-1, 0, 1):
518         for dc in (-1, 0, 1):
519             if (dr or dc) and 0 <= r + dr < R and 0 <= c + dc < C:
520                 yield r + dr, c + dc
521
522 # ===== 15. control =====
523
524 def memo(f):                                # unbounded memoizer; enough for DP (no eviction by
525     ↪ design)
526     cache = {}
527     def wrapped(*args, **kw):
528         key = (args, frozenset(kw.items())) if kw else args
529         if key not in cache:
530             cache[key] = f(*args, **kw)
531         return cache[key]
532     wrapped.cache = cache                    # peek at the DP table if curious
533     return wrapped
534
535 def until(cond, f, x):                       # until cond f x -- loop, not recursion (unbounded
536     ↪ depth)
537     while not cond(x):
538         x = f(x)
539     return x

```


Appendix B

Haskell Provenance

Where each borrowed name comes from, for the day you meet the originals.

Module	Names
Prelude	id, const, flip, curry, uncurry, fst, snd, succ, pred, even, odd, signum, div, mod, quot, rem, gcd, lcm, head, tail, init, last, null, elem, replicate, drop, splitAt, lookup, zip, unzip, map, filter, concat, concatMap, zipWith, foldl, foldr, product, iterate, repeat, cycle, take, takeWhile, dropWhile, span, break, words, unwords, lines, unlines, until
Data.List	find, partition, nub, sortOn, sortBy, transpose, unfoldr, subsequences, inits, tails, isPrefixOf, isSuffixOf, stripPrefix, minimumBy/maximumBy (as minOn/maxOn), groupBy, group
Data.Map	insertWith, fromListWith, unionWith
Data.Maybe	fromMaybe, isJust, isNothing, mapMaybe, catMaybes (None is Nothing)
Data.Function	on
Data.Tuple	swap
Control.Monad	replicateM
Data.List.Split	chunksOf
Python itertools (names)	count, islice, takewhile, dropwhile, chain, accumulate, starmap, pairwise – kept under Python’s own names so both vocabularies stay warm

Deliberately not ported: laziness-dependent knot-tying (`fix`), typeclass machinery, and monadic control flow – Python is strict and untyped, and the prelude stays honest about it.

Appendix C

How to Practice

The course directory is the textbook's other half:

- `fpython/prelude.py` – the toolkit. Study loop: read a section, close the file, retype its definitions from memory, diff.
- `fpython/g1_... g9_...` – 74 practice files. Each is self-contained: statement, contract, hint, the needed snippets copied inline, a `# solution goes here` slot, and a doctest runner. Run with `python3 file.py` until it reports *N/N*.
- `fpython/solutions/` – the same files solved, flat, named `g2_08_chunks.py`. For *after* the fight: compare shapes, not before.
- `fpython/prelude_flowchart.svg` – the recognition ladder of Chapter 2 as a printable picture.

Work the classes in order; within a class, any order. Narrate out loud as you solve – shape, tools, complexity – because the interview grades the narration as much as the code. When a solution of yours beats the book's, trust the doctests and keep yours.

